# Safeguarding Users' Web Interactions

by

Faezeh Kalantari

A Dissertation Presented in Partial Fulfillment
of the Requirement for the Degree
Doctor of Philosophy

Approved October 2024 by the
Graduate Supervisory Committee:

Adam Doupé, Chair
Yan Shoshitaishvili
Ruoyu Wang
Tiffany Bao
Jason Polakis

ARIZONA STATE UNIVERSITY

December 2024

ABSTRACT

The Web plays an indispensable role in modern life, enabling countless interactions, from social engagement to critical business operations. However, these interactions expose users to a variety of security and privacy threats. This dissertation focuses on safeguarding users' Web interactions by addressing three key challenges: content injection vulnerabilities in web applications, privacy risks from browser extension fingerprinting, and Account Takeover (ATO) attacks carried out by fraud browsers, all of which directly impact users' safety.

First, Context-Auditor introduces a novel technique to mitigate content injection vulnerabilities, including Cross-Site Scripting (XSS), scriptless attacks, and command injections. By detecting unintended context switches in the browser's parsing engine, Context-Auditor provides robust protection for web applications, ensuring that users' interactions with them remain secure.

Second, Simulacrum enhances privacy protection by defending against DOM-based extension fingerprinting using the DOM Reality Shifting concept. It conceals browser extension behaviors from websites, preventing over 95% of vulnerable extensions from being fingerprinted. This approach directly addresses user privacy concerns, shielding them from tracking and profiling based on browser extensions.

Finally, Browser Polygraph provides a scalable, privacy-preserving solution to detect fraud browsers used in ATO attacks. By leveraging coarse-grained browser fingerprints, it identifies suspicious sessions, improving the accuracy of risk-based authentication systems and protecting users from fraudulent account compromises.

In summary, this dissertation presents practical, deployable solutions that enhance the security and privacy of users' Web interactions. By safeguarding web applications, protecting user privacy, and defending against ATO fraud, these contributions play a key role in ensuring the safety of users in the increasingly adversarial Web ecosystem.

*To my wonderful Mom and Dad and my incredible husband Mohammad.*

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my advisor, committee members, mentors, fellow students, industry collaborators, and my family and friends for their unwavering support throughout these years.

First and foremost, I am profoundly thankful to my advisor, Dr. Adam Doupé, for his exceptional guidance and encouragement during my PhD journey. I still recall my first email to you during my master's program, and how your supportive response gave me the confidence to begin my PhD at Sefcom. Your patience, mentorship, and inspiring words, especially those moments when you said "I am proud of you," have been instrumental in my growth.

I also extend my sincere thanks to my mentors at Sefcom, Dr. Yan Shoshaitashvili, Dr. Fish Wang, and Dr. Tiffany Bao, for your invaluable support and wisdom over the years. A special thanks to Dr. Gail-Joon Ahn for your kindness and support during my PhD.

I am deeply grateful to Dr. Jason Polakis for serving on my committee and for your guidance on our collaborative project. Dr. Gianluca Stringhini, I also greatly value your advice and the insights you provided during our collaboration.

To my wonderful husband, Mohammad—words cannot fully express how grateful I am for you. Your unwavering love, support, and belief in me kept me going through the hardest and most stressful years of my PhD. The final stretch was filled with challenges, but you stood by me, helping me find my way when I was lost. Thank you for listening to my endless concerns and being my rock.

To my incredible Maman, my first and greatest teacher—thank you for your boundless love, sacrifices, and support. I owe every achievement to the countless sacrifices you made. To my dear Baba, the kindest soul I know—thank you for your constant love and for always encouraging my education. I will forever be grateful for

all you've done for me. I want to thank my amazing siblings for always supporting me: Zohreh, you set the path for me and Zeinab to follow in computer science; you've always been an inspiration and my role model. Thanks for being a smart, patient, and supportive big sister. Zeinab, thank you for your emotional and professional support throughout the years and always being so patient and understanding. Ali, thank you for your strength and always being there for me, from childhood through adulthood. Your unwavering support means the world to me. Fatemeh, I attribute my academic success to the days in high school when I pushed myself to keep up with you while you prepared for medical school—you thought me the true meaning of dedication and perseverance and I thank you for that.

I am incredibly grateful to my dear friend and collaborator, Mehrnoosh Zaeifi, whose continuous support has been invaluable to my PhD projects. To my other friends and collaborators in the Sefcom lab—Xavier Jeremy Maso, Yeganeh Safaei, Erik Trickel, Adam Oest, Marzieh Bitaab, and many others—thank you for your help and friendship. I also extend my appreciation to my collaborators from UIC, Soroush Karami and Panagiotis Ilia.

Lastly, a heartfelt thank you to my friends from ASU, who played a pivotal role in my PhD journey. I cherish all the wonderful memories we've made. To my incredible roommate, Mansooreh Karami, thank you for being a brilliant, strong sister to look up to. I also want to acknowledge my other dear friends—Tahora Nazer, Parisa Mahmoudi, Zahra Zahedi, Fatemeh Zahedi, Mona Khoddam, Zeinab Ranjbar, Rana Pourmohammad, Zahra Derakhshandeh, Ghazal Shams, and Hyun-Jung Lim. Your friendship has been a tremendous source of joy and support.

TABLE OF CONTENTS

Page

LIST OF TABLES . . . . . x

LIST OF FIGURES . . . . . xi

CHAPTER

1 INTRODUCTION . . . . . 1

1.1 Protecting Web Applications . . . . . 3

1.2 Preserving User Privacy . . . . . 4

1.3 Enhancing the Security of User Accounts . . . . . 6

1.4 Contributions . . . . . 7

1.5 Organization of the Dissertation . . . . . 9

2 BACKGROUND . . . . . 11

2.1 Cross-Site Scripting . . . . . 11

2.1.1 XSS Prevention . . . . . 12

2.1.2 XSS Detection . . . . . 13

2.1.3 Context-Awareness . . . . . 14

2.1.4 Conclusion . . . . . 15

2.2 Browser Fingerprinting . . . . . 15

2.3 Extension Fingerprinting . . . . . 16

2.3.1 DOM-Based Extension Fingerprinting . . . . . 16

2.4 DOM-Based Extensions Anti-Fingerprinting via Simulacrum . . . . . 18

2.4.1 Background and Threat Model . . . . . 20

2.4.2 DOM Reality Shifting with Simulacrum . . . . . 23

2.4.3 System Implementation . . . . . 24

2.4.3.1 Primitives . . . . . 24

2.4.3.2 DOM-Accessing APIs . . . . . 26

CHAPTER Page


2.4.3.3 Function Overriding . . . . . 28

2.4.3.4 Additional Security Precautions . . . . . 36

3 CONTEXT-SENSITIVE CONTENT INJECTION MITIGATION IN WEB APPLICATIONS . . . . . 41

3.1 Introduction . . . . . 41

3.2 Background . . . . . 44

3.2.1 Content Injection Vulnerabilities . . . . . 44

3.2.2 Mitigations of Content Injection Exploits . . . . . 47

3.3 Overview . . . . . 49

3.3.1 Motivation . . . . . 50

3.3.2 Context Switching . . . . . 51

3.3.3 Context-Auditor Overview . . . . . 52

3.3.4 Vulnerability Model . . . . . 55

3.3.5 Threat Model . . . . . 57

3.3.6 Comparison with Current Mitigations . . . . . 57

3.4 Building the Model . . . . . 59

3.4.1 Modeling Web Languages Parsing . . . . . 59

3.4.2 Modeling Shell Command Parsing . . . . . 62

3.5 Implementation . . . . . 62

3.6 Evaluation . . . . . 64

3.6.1 Experiment Design . . . . . 65

3.6.2 Data Sets . . . . . 65

3.6.3 E1: Detecting Web-Based Content Injection Exploits . . . . . 67

3.6.4 E2: Detecting XSS Exploits Generated by w3af . . . . . 67

CHAPTER Page


3.6.5 E3: Detecting Command Injection Exploits . . . . . 68

3.6.6 E4: Measuring the Likelihood of Context Switching in Exploits of a Public Data Set . . . . . 69

3.6.7 E5: Performance Overhead and False Positive on Top Alexa Websites . . . . . 71

3.6.8 E6: Measuring False Positives on a Blog Site . . . . . 72

3.6.9 E7: Measuring Runtime Overhead . . . . . 73

3.7 Limitations . . . . . 74

3.8 Related Work . . . . . 79

3.9 Conclusion . . . . . 80

4 EVALUATION RESULTS OF SIMULACRUM . . . . . 82

4.1 Experimental Evaluation . . . . . 82

4.2 Discussion, Limitations, and Guidelines . . . . . 93

4.3 Related Work . . . . . 95

4.4 Conclusions . . . . . 97

5 EFFICIENT DEPLOYMENT OF COARSE-GRAINED BROWSER FINGERPRINTS FOR WEB-SCALE DETECTION OF FRAUD BROWSERS 99

5.1 Introduction . . . . . 99

5.2 Background . . . . . 102

5.2.1 Fraud Browsers . . . . . 103

5.2.2 Fraud Browsers' Behavior . . . . . 104

5.3 Web-Scale Fingerprinting Requirements . . . . . 106

5.4 Threat Model of Browser Polygraph . . . . . 108

5.5 Overview of Browser Polygraph . . . . . 109

CHAPTER Page


5.6 Design . . . 111

5.6.1 Candidate Fingerprint Generation . . . 111

5.6.2 Real-World Data Collection . . . 114

5.6.3 Data Pre-Processing . . . 114

5.6.4 Training of Machine-Learning Model . . . 117

5.6.4.1 Pre-Processing . . . 117

5.6.4.2 Feature Selection . . . 118

5.6.4.3 Clustering . . . 118

5.6.5 Fraud Detection . . . 122

5.6.6 Drift Detection . . . 123

5.7 Evaluation . . . 125

5.7.1 Real-World Experiment . . . 125

5.7.2 Fraud Browsers' Detection . . . 127

5.7.3 Drift Analysis . . . 128

5.7.4 Privacy Analysis . . . 131

5.7.5 Performance Analysis . . . 132

5.8 Discussion . . . 132

5.9 Related Work . . . 135

5.10 Ethics . . . 137

5.11 Conclusion . . . 138

6 CONCLUSION . . . 139

6.1 Future Research Directions . . . 140

CHAPTER Page

REFERENCES . . . . . 143

APPENDIX . . . . . 156

A SIMULACRUM DETAILS . . . . . 157

B CONTEXT-AUDITOR DETAILS . . . . . 163

C BROWSER POLYGRAPH FEATURES . . . . . 167

D ALL FEATURES USED IN THE REAL-WORLD DATA COLLECTION STAGE . . . . . 169

E BROWSER POLYGRAPH WITH DIFFERENT NUMBER OF CLUSTERS . . . . . 175

F SENSITIVITY ANALYSIS OF BROWSER POLYGRAPH . . . . . 177

G COMPARISON BETWEEN BROWSER POLYGRAPH AND FINE-GRAINED FINGERPRINTING TECHNIQUES . . . . . 181

LIST OF TABLES

Table Page

3.1 Comparison of XSS mitigation techniques . . . . . 58

3.2 Details of each experiment in the evaluation. . . . . 66

3.3 Context-Auditor's command injection detection capability . . . . . 69

3.4 False negative and true positive rate of Context-Auditor . . . . . 73

4.1 Breakdown statistics for Simulacrum's effectiveness. . . . . 85

4.2 Average Time-To-Interactive overhead in seconds. . . . . 91

4.3 Breakage in 50 randomly-chosen popular websites. . . . . 92

5.1 Fraud browsers categories . . . . . 105

5.2 Performance comparison of fingerprinting tools . . . . . 108

5.3 `user-agents` assigned to clusters with k=11. . . . . 122

5.4 Real-world evaluation of Browser Polygraph . . . . . 125

5.5 Fraud browsers' detection capability via Browser Polygraph. . . . . 128

5.6 Drift Analysis of Browser Polygraph . . . . . 130

5.7 Entropy information of Browser Polygraph features . . . . . 132

A.1 The number of overridden functions. . . . . 159

A.2 Examples of different functions and our strategy for wrapping them. . . 160

C.1 All features . . . . . 168

E.1 `user-agents` distribution with k=6 . . . . . 176

F.1 Clusters number sensitivity analysis . . . . . 178

F.2 PCA components sensitivity analysis . . . . . 179

F.3 features number sensitivity analysis . . . . . 179

G.1 Comparison of fingerprinting techniques on Windows . . . . . 183

G.2 Comparison of fingerprinting techniques on macOS . . . . . 183

LIST OF FIGURES


Figure Page

2.1 Most prevalent prototypes across top 10k websites. . . . . 27

2.2 Different stages in a webpage's lifecycle. . . . . 38

3.1 Context-Auditor's design . . . . . 53

3.2 Context switching in a browser parser . . . . . 54

3.3 Context-Auditor's deployment . . . . . 56

3.4 Potential deployment locations for Context-Auditor . . . . . 59

3.5 Context-Auditor as a command detection module . . . . . 63

3.6 Context-Auditor as an nginx module . . . . . 64

4.1 DOM-interacting APIs calls in the top 10K sites. . . . . 86

4.2 Interface invocations in the top 10K websites . . . . . 88

4.3 DOM loading time difference caused by Simulacrum. . . . . 89

5.1 Overview of Browser Polygraph. . . . . 109

5.2 Cumulative variance based on the number of components in PCA. . . . . 119

5.3 Elbow method to select the optimal number of clusters. . . . . 121

5.4 Relative WCSS vs. clusters. . . . . 121

5.5 Percentage of fingerprints in anonymity sets. . . . . 131

A.1 Inheritance hierarchy in the DOM API. . . . . 161

A.2 Evaluation results of Simulacrum with Celtic Kane. . . . . 161

A.3 Example of site with added noise elements. . . . . 162

B.1 A simplified 2PDA for shell scripts. . . . . 165

B.2 A simplified 2PDA for HTML docs. . . . . 166

# Chapter 1

# INTRODUCTION

The Web has evolved from static HTML pages to sophisticated applications that deliver dynamic and interactive content. These modern applications, known as web applications, have become an integral part of daily life, serving critical functions in fields such as medicine, banking, and social networking. Given that many of these applications handle sensitive user information, they have become prime targets for cybercriminals seeking to exploit security and privacy vulnerabilities.

Modern web applications consist of both client-side and server-side components. The server-side module (such as a PHP program) dynamically generates HTML pages in response to HTTP requests, often interacting with back-end databases to retrieve additional content. Once generated, this content is sent to the client-side, typically a web browser, where it is rendered and can be further manipulated.

A significant advancement in Web technology is the ability of modern applications to offload parts of their business logic to the client-side using JavaScript. JavaScript enables rich client-side functionality, allowing operations to be performed without constant communication with the back-end. Through asynchronous JavaScript (AJAX) requests, web content can be dynamically updated seamlessly. However, the power and broad access granted to JavaScript, such as the ability to manipulate sensitive client-side data or make remote requests, also exposes web applications to significant risks. Attackers can exploit vulnerabilities in web applications to execute unwanted or malicious JavaScript code on a user's browser. Additionally, users' privacy can be compromised by websites that extract sensitive user information using fingerprinting techniques. Furthermore, the security of user accounts is threatened by attackers

spoofing browser fingerprints to perform Account Takeover (ATO) attacks.

While restricting JavaScript execution on certain websites may seem like a viable solution, it is not practical for all web applications. Blocking JavaScript indiscriminately can break essential functionality, rendering many web applications unusable. Therefore, it is crucial to safeguard users' Web interactions without compromising the dynamic nature of modern web applications.

In this dissertation, I explore safeguarding users' Web interactions across three distinct areas:

(1) **Protecting web applications** from the execution of unwanted code, with a focus on mitigating JavaScript-related vulnerabilities (later generalized to other forms of content injection) and preventing the exploitation of web applications.

(2) **Preserving user privacy** by defending against advanced tracking mechanisms, particularly those deployed by websites using DOM-based extension fingerprinting techniques to identify users.

(3) **Enhancing the security of user accounts** by defending against ATO attacks, which compromise user account security through fingerprint spoofing—techniques often employed by fraud browsers.

This dissertation addresses these core Web security challenges by proposing novel approaches and tools designed to secure users' Web interactions. These include detecting and mitigating content injection attacks, protecting user privacy by countering browser extension fingerprinting, and strengthening account security by identifying sessions of fraud browsers used in ATO attempts. By advancing the field of Web security and privacy, this dissertation aims to create a safer, more secure environment for users within the Web ecosystem.

## 1.1 Protecting Web Applications

Web security experts have long been concerned about vulnerabilities in web applications that lead to the execution of unwanted JavaScript code in a victim's browser. JavaScript execution holds broad functionality and access rights within the browser environment, and failure to detect and prevent JavaScript exploits can result in privacy breaches and sensitive information leakage.

Browsers implement security policies to restrict the loading and running of scripts from external resources. For example, Same-origin policy (SOP) [1], isolates scripts from one origin, preventing them from accessing resources from other origins, while Content Security Policy (CSP) [2] defines trusted sources of scripts and other content for a website. In many cases, these policies are either improperly deployed (will be discussed in Section 2.1), or they are so restrictive that they interfere with necessary functionalities in modern web applications. Specifically, while SOP restricts resource sharing between websites, the Cross-Origin Resource Sharing (CORS) [3] policy relaxes some of these restrictions to facilitate such sharing.

Traditional JavaScript-based exploits, such as those targeting *Cross-Site Scripting (XSS)* [4] vulnerabilities, are somewhat mitigated by browser security policies, but these policies do not guarantee absolute protection. Due to the prevalence and persistence of JavaScript exploits, this dissertation focuses on Cross-Site Scripting vulnerabilities.

Cross-Site Scripting (XSS) is a vulnerability that allows an attacker to bypass the SOP [1] and execute arbitrary JavaScript code within a website's execution environment. Once an XSS vulnerability is exploited, attackers gain broad access to sensitive authentication tokens or authorization data and can perform unauthorized actions by sending malicious requests to the web application [4]. British Airway's website, for

example, was compromised with an XSS exploit in 2018, leading to the leakage of sensitive information for approximately 380k transactions [5]. Similarly, in 2019, an XSS vulnerability in Fortnite, a popular online video game, exposed the accounts of roughly 200 million users to attackers [6].

The emergence of advanced XSS exploitation techniques inspired a deeper examination of the root cause of XSS vulnerabilities: Web applications embed into an HTML page untrusted user input as *pure data*, such as strings and text, with the intent that no part of the untrusted user input shall be interpreted by the web browser's HTML parser as code. However, this important developer-intent is lost once the HTML code is generated and sent to the browser: client-side parsers must *re-discover* the meaning of every part of the page by parsing the HTML response. In exploiting XSS vulnerabilities, attackers leverage this loss of information and mislead client-side parsers to transition from HTML parsing to JavaScript parsing when parsing the user input, which *violates the developer's original intent.* I will explore this issue in greater detail in Chapter 3, where I present a single solution to mitigate XSS and similar vulnerabilities, which share the same root cause of changing parser behavior.

## 1.2 Preserving User Privacy

As the Web continues to evolve, preserving user privacy has become a central concern, particularly with the rise of advanced tracking mechanisms that allow websites to collect extensive information about users without their consent. These privacy-invasive practices often exploit the same capabilities that enable rich Web interactions, such as JavaScript execution and client-side processing. While privacy policies and browser-level security measures aim to limit tracking, attackers and malicious websites have developed increasingly sophisticated techniques to circumvent these defenses.

One of the most concerning trends is *browser fingerprinting*, a method by which websites collect identifiable information about a user's browser, device, and environment to create a persistent profile. Unlike traditional tracking methods, such as cookies, browser fingerprinting does not rely on stored data. Instead, it leverages client-side characteristics—such as the the user-agent [7], geolocation, timezone, screen resolution [8], available fonts [7; 9; 10], the JavaScript execution environment [11; 12; 13]. Many of these fingerprinting techniques heavily rely on JavaScript Web APIs[14] to collect detailed information from the user's browser.

Beyond standard fingerprinting techniques, some methods focus on *browser extension fingerprinting*, where attackers target specific user-installed browser extensions. Extensions, designed to enhance browser functionality and user experience, can, however, leave behind detectable traces that attackers can use to infer which extensions are installed. Browser extension fingerprinting is indeed a real-world threat, as LinkedIn was found trying to detect 38 different extensions [15]. This type of fingerprinting poses significant risks to user privacy, as certain extensions can expose sensitive information about a user's preferences or affiliations [16].

Prior studies have proposed mitigations that target different aspects of extension fingerprinting, namely preventing techniques that exploit Web Accessible Resources (WARs) [17], stylesheets (CSS) [18], and node attributes [19]. However, these defenses have not been effective in fully preventing *DOM-based extension fingerprinting* [16], since many extensions intentionally modify pages in ways that makes them uniquely identifiable. The root cause of behavior-based extension fingerprinting is that any JavaScript running within the context of the web page can observe all the changes and modifications made by installed extensions, enabling fingerprinting.

To address this problem, the concept of DOM Reality Shifting was proposed to hide extension-related DOM modifications from fingerprinting websites. This con-

cept was designed and implemented in Simulacrum[20]. In this dissertation, I will specifically evaluate the practicality and effectiveness of Simulacrum in mitigating DOM-based extension fingerprinting (detailed in Chapter4).

## 1.3 Enhancing the Security of User Accounts

As web applications become more integral to business operations, organizations face growing threats from cybercriminals seeking to exploit user accounts. One of the most significant threats is *Account Takeover (ATO)* fraud, where attackers gain unauthorized access to user accounts by stealing credentials and using them to impersonate legitimate users. This form of fraud results in substantial financial losses, particularly for industries like e-commerce and financial services. In 2022, the FBI reported that victims of business email compromise, a subset of ATO, suffered losses exceeding 2.7 billion USD [21].

To counter these threats, organizations have increasingly adopted *risk-based authentication* methods, which analyze various aspects of a login attempt to assess its legitimacy. A common technique is to match the current browser's characteristics against previously established browser fingerprints for the user. However, attackers have adapted by using *anti-detect browsers*, referred to as *fraud browsers* in this dissertation, which are specifically designed to mimic browser profiles and could bypass these authentication systems. Fraudsters can easily steal browser profiles and use them to impersonate legitimate users. These stolen profiles are often sold on underground marketplaces, such as Genesis Market, where over 80 million stolen browser profiles were traded before the site was taken down by the FBI in 2023 [22]. As a result, detecting ATO fraud becomes particularly challenging for organizations.

Although *fine-grained browser fingerprinting* techniques could be used to detect fraud browsers, they face significant challenges in large-scale deployments due to

high computational overhead and privacy concerns. Fine-grained fingerprinting often relies on detailed user tracking, which can slow down Web interactions and introduce substantial overhead, making continuous fraud detection inefficient.

To address these challenges, I propose a machine-learning-based tool designed to detect fraud browsers using *coarse-grained fingerprinting*. By, coarse-grained browser fingerprints, I refer to a simplified and less detailed set of fingerprinting features tailored specifically for real-world fraud detection and safeguarding user accounts. In Chapter 5, I will discuss the design, implementation, and evaluation of this tool.

## 1.4 Contributions

In this dissertation, I make the following contributions to safeguard users' Web interactions.

- **Context-Auditor: A Content Injection Mitigation Tool**

  To improve security of web applications against injection vulnerabilities: (1) I reformulated the problem of defending against command injections, XSS, and scriptless attacks as a content injection mitigation problem, and I focused on the root cause of content injection exploitation, which is untrusted user input triggering a context switch in parsers. (2) I built a custom parser model that supports HTML, CSS, JavaScript, and Bash scripts (and is extensible to other languages). Context-Auditor uses this model to identify context switches caused by untrusted user input, which indicates the exploitation of a content injection vulnerability. (3) I demonstrated that Context-Auditor can mitigate exploits that state-of-the-art XSS mitigation techniques cannot detect.

- **Evaluation of Simulacrum: Defending Against DOM-Based Extension Fingerprinting**

To protecting users from privacy-invasive DOM-based extension fingerprinting: (1) I experimentally evaluated Simulacrum's defensive and performance impact, and demonstrated that Simulacrum effectively protects extensions while incurring a negligible performance overhead and limited website breakage. (2) I presented guidelines for extension developers that allow them to eliminate problematic fingerprintable behaviors without affecting the extension's functionality. These guidelines require straightforward changes that are trivial to implement, and can contribute to completely eliminating DOM-based fingerprinting.

- **Browser Polygraph: Efficient Deployment of Coarse-Grained Browser Fingerprints for Web-Scale Detection of Fraud Browsers**

  To enhance the security of user accounts against rising threat of ATOs driven by fraud browsers (1) I proposed a new privacy-preserving idea of detecting lying browsers while maintaining the performance characteristics necessary for real-world deployment at a major financial company (called FinOrg) using efficient coarse-grained fingerprints. (2) I leveraged this idea into a clustering model, called Browser Polygraph, that outputs risk factor of suspicious user sessions, which can be used as part of risk-based authentication. (3) I deployed Browser Polygraph at FinOrg, where I have demonstrated its effectiveness at detecting real-world fraud. (4) I demonstrated Browser Polygraph 's effectiveness in detecting a subset of fraudulent browser fingerprints created by fraud browsers.

During my PhD, I have published the following three papers, which form the primary contents of this dissertation:

1. **Context-Auditor: Context-sensitive Content Injection Mitigation**. Published in the Proceedings of the 25th International Symposium

on Research in Attacks, Intrusions, and Defenses (RAID).

2. **Unleash the Simulacrum: Shifting Browser Realities for Robust Extension Fingerprinting Prevention**. Published in the Proceedings of the 31st USENIX Security Symposium.

3. **Browser Polygraph: Efficient Deployment of Coarse-Grained Browser Fingerprints for Web-Scale Detection of Fraud Browsers**. Published in the Proceedings of the 24th ACM Internet Measurement Conference (IMC).

## 1.5 Organization of the Dissertation

This dissertation is organized as follows:

In Chapter 2, I provide the background knowledge necessary to understand the rest of the dissertation. Section 2.1 introduces Cross-Site Scripting (XSS), the various categories of XSS, and the prevention and detection methods used to combat it, as well as the concept of context-awareness, which is central to the development of Context-Auditor.

Regarding privacy in Web interactions, Section 2.2 discusses browser fingerprinting, while Section 2.3 focuses on extension fingerprinting, specifically DOM-based extension fingerprinting. In Section 2.4, Simulacrum is presented as a countermeasure to DOM-based extension fingerprinting: the contents of this Section are drawn from our collaborative paper [20] and, while they represent the contribution of another author, Soroush Karami, they are necessary for understanding my own contribution, which is discussed in Chapter 4.

Chapter 3 details the development of Context-Auditor, including the motivation, design, implementation, deployment, and evaluation of this content injection detection tool.

Chapter 4 presents the evaluation results of Simulacrum and provides guidelines for extension developers to enhance privacy protection.

Chapter 5 introduces Browser Polygraph, covering its motivation, design and evaluation as a Web-scale tool for fraud detection and safeguarding user accounts.

Finally, Chapter 6 concludes the dissertation and outlines potential future research directions in Section 6.1.

# Chapter 2

# BACKGROUND

## 2.1 Cross-Site Scripting

Cross-Site Scripting (XSS) is a vulnerability that allows an attacker to bypass the Same-origin policy (SOP) [1] and execute arbitrary JavaScript code within a website's execution environment. Once an XSS vulnerability is exploited, attackers gain broad access to sensitive authentication tokens or authorization data and can perform unauthorized actions by sending malicious requests to the web application [4].

XSS vulnerabilities are commonly categorized into different types. Below, I discuss the most prevalent classes of XSS.

**Reflected XSS.** Reflected XSS is the traditional form of XSS, where the web application reflects user inputs from an HTTP request back into the corresponding HTTP response without proper sanitization. This results in the execution of malicious JavaScript code injected by the attacker.

**Stored XSS.** In this class of XSS, the unwanted JavaScript code is not *directly* from an HTTP request. Instead, it originates from a vulnerable data set of the web application [4]. Attackers populate this vulnerable data set with an exploit via a specially crafted HTTP request, which is later executed when the data is retrieved and displayed to users.

**DOM-based XSS.** DOM-based XSS specifically compromise vulnerabilities in the client-side code (an attacker does not send an HTTP request to the back-end web application for the exploitation). A common example of vulnerable client-side code is the usage of `eval` or `write` JavaScript functions when an attacker can control the

input sent to these critical functions. In this case, she can insert unwanted JavaScript code into a web page.

**Other typs of XSS.** Researchers introduced other variants of XSS vulnerability: *Mutation-based XSS* (mXSS) occurs when certain browser features (often undocumented and specific to particular browser implementations) transform benign markup into dangerous characters that form part of an XSS exploit [23]. Another variant, known as *persistent client-side XSS* exploits [24] arises when a DOM-based exploit is stored in client-side storage mechanisms like local storage or cookies.

### *2.1.1 XSS Prevention*

Since the introduction of XSS, researchers have proposed techniques to prevent the injection of JavaScript code at *server-side*: Balzarotti et al. implemented an input sanitization tool called Saner [25] which performed a combination of static and dynamic analysis on PHP code, to identify XSS vulnerabilities. XSS-Guard [26] performed cross-validation between any HTML response and a corresponding shadow HTML response (a response triggered by benign inputs) to prevent potential script injections. Automatic sanitizer placement [27] and code and data separation [28] were also some other server-side XSS prevention methods.

A few years after the discovery of conventional server-side XSS exploits, *client-side* XSS also gained the attention of Web security researchers: ScriptProtect [29] is a client-side prevention technique that forbids string to code conversions originating from third party JavaScript code, and DOMPurify [30] is a JavaScript library used to sanitize HTML code and prevent XSS exploitation.

**Content Security Policy (CSP)** is a white-listing mechanism that specifies the legitimate sources of external resources for a web page; CSP adds directives into HTTP headers to determine these legitimate sources [2]. In this way, when used

correctly, CSP can prevent XSS exploits originating from external resources. While all modern browsers support CSP, it is not deployed correctly by most web applications in the wild [31], and also previous work has shown it to be insufficient to prevent all XSS exploits [32].

Google has been using **Trusted Types** [33] since 2019 as a measure against DOM-based XSS. Trusted Types has a set of policies to help a web application distinguish trusted data from untrusted. These policies proxy inputs to sensitive JavaScript functions so that they accept special types instead of strings; these types have already been sanitized using proper functions from DOMPurify [30] library. Enforcement of Trusted Types policies is also controlled inside CSP HTTP response headers [34].

**Testing** also plays a critical role in XSS prevention; it can reveal many XSS exploits targeting a web application in different stages of development and deployment. **Scanners** are popular vulnerability analysis tools that test a list of exploit payloads against a web application. Prior research has already explored the efficacy of web scanners [35]. Scanners mostly use a predefined list of exploits to test a target web application, leading to the failure of some of these test cases. Therefore, automatic generation of exploits might create exploits that are more adjusted to the context of a vulnerable web application and have higher success rate: Lekies et al. implemented a context-sensitive XSS exploit generation approach [36]. Navex [37] combines static and dynamic analysis of PHP web applications to perform automatic exploit generation for a broad spectrum of web attacks, including XSS.

### *2.1.2 XSS Detection*

**Browser filters** are a common client-side XSS detection method: NoScript [38] (for Firefox) is a powerful browser extension that is widely used as an XSS filter. There is also a No-Script Chrome extension [39]. Chrome browser used XSS-Auditor [40] as

a browser filter until 2019 which is not as powerful as NoScript.

Researchers have also investigated client-side **taint-tracking** to detect XSS exploits. Taint-tracking encompasses untrusted input with taint metadata and then analyzes whether the taint has reached a sensitive output of the web application. In 2013, Lekies et al. created a taint-aware JavaScript engine to detect DOM-based XSS exploits [36]. Later, in 2018, Melicher et al. [41] proposed a more comprehensive taint tracking approach to detect vulnerabilities caused by errors in the implementation of **HTML templating**. HTML templating is a similar approach to parameterized SQL queries focused on preventing XSS exploits. Steffens et al. also introduced another taint-tracking method in 2019 that aimed to detect flows from client-side sources such as local storage to sinks; their method detects persistent client-side XSS exploits [24].

**Web application firewalls (WAFs)** (e.g., ModSecurity [42]) attempt to detect web requests that include an XSS exploit and prevent them from reaching back-end web applications. WAFs must be manually configured with a comprehensive set of rules or directives and its detection capability is closely related to the comprehensibility of these rules and directives.

### *2.1.3 Context-Awareness*

I believe XSS has been a trivial web attack for around 20 years since most detection and prevention techniques in this area lack a proper understanding of their underlying context. As discussed in Section 1.1, XSS exploits succeed since they can trick client-side parsers into parsing JavaScript code instead of HTML markup (that was the developer's original intent). Besides, the detection and prevention techniques fail as they are not aware of these switches of context from HTML to JavaScript. A comprehensive XSS mitigation technique needs to be precisely aware of the *expected context* in the critical entry points of the application and then flag any *unexpected*

*context switch* in those points.

Few papers are trying to reflect the context-awareness concept in their work: Saxena et al. has discussed the issues related to the inconsistent placement of sanitizers in web applications [43] that includes either wrong selection of input sanitizers or application of them in the wrong order. Stock et al. [44], used context switches in the JavaScript parser to detect DOM-based XSS exploits using a taint-tracking browser. Lekies et al. considered different break-in and break-out sequences to create exploits in either HTML or JavaScript contexts [36].

### *2.1.4 Conclusion*

As discussed in Section 2.1.3, security researchers have underestimated the importance of context-awareness in detection and prevention of JavaScript-based injection exploits in web applications. I believe, with a focus on switches of context, triggered by untrusted inputs, we can design a comprehensive method to mitigate JavaScript-based code injections and any other code injection exploits targeting a web application. Therefore, in this dissertation, I aim to enhance the security of web browsers with a focus on the detection of unwanted context switches; I precisely determine such context switches by tracking untrusted inputs and their impact on parsing analysis of web contents. The proposed context-awareness injection detection method, is a general solution for the general category of content injection exploitations against web applications. I will demonstrate this detection technique in Chapter 3.

## 2.2 Browser Fingerprinting

Browser fingerprinting is an important aspect of protecting users privacy in the modern Web. Browser fingerprinting research extensively probes the attributes and features of browsers that can divulge details about a user's underlying system—both

software and hardware. These attributes and features can be used to uniquely identify the user's browser. JavaScript accessible attributes such as the `user-agent` [7], geolocation, timezone, screen resolution [8], and available fonts [7; 9; 10] have been classic sources of fingerprintable data. Recent advancements have added depth to this field by exploring more intricate parameters; for instance, APIs such as Canvas [45; 10], WebGL [46; 47], and WebRTC [10] have been explored. Beyond these, even nuanced elements such as AudioContext [10], CSS properties [48], specific JavaScript objects such as `navigator` and `screen` [49], Crypto API [50], the list of extensions and plugins [17; 18; 19; 51], and HTTP headers [51] serve as potential fingerprinting vectors.

## 2.3 Extension Fingerprinting

Browser fingerprinting involves extracting unique attributes from a user's browser and device. Among these, the list of installed browser extensions can be combined with other information to create highly reliable fingerprints. Several methods exist to perform extension fingerprinting, such as exploiting Web Accessible Resources (WARs)[17], CSS styles[18], or the communication patterns of extensions [16].

In this dissertation, however, the focus is on fingerprinting browser extensions by analyzing the modifications they make to the Document Object Model (DOM) of a webpage. This technique will be explained in more detail in the remainder of this chapter.

### *2.3.1 DOM-Based Extension Fingerprinting*

DOM-based extension fingerprinting refers to the process of identifying browser extensions by analyzing how they modify the DOM of a webpage, including the addition, removal, or alteration of DOM elements and attributes. Starov and Nikiforakis

developed XHOUND [52], the first fully automated system for fingerprinting browser extensions by tracking their modifications to the DOM. They used XHOUND to evaluate popular extensions and demonstrated that between 9% and 23% of these extensions were fingerprintable.

In response to this vulnerability, Trickel et al. developed CloakX [19], a countermeasure designed to obfuscate DOM-based fingerprints. CloakX works by intercepting DOM modification requests, diversifying their responses, and injecting random tags and attributes into the DOM to complicate fingerprint detection.

Carnus, proposed by Karami et al. [16], introduced a behavior-based approach to fingerprint extensions, focusing on how they modify the DOM by inserting, deleting, or altering nodes. Carnus demonstrated strong results, with approximately 84% to 88% of its behavior-based fingerprints remaining effective even against CloakX's defenses.

Recognizing the persistent threat posed by behavior-based extension fingerprinting, our collaborative project aimed to address the root cause of DOM-based extension fingerprinting: the attacker shares the same DOM view as the user, allowing easy detection of DOM-based fingerprints. To mitigate this, the concept of DOM Reality Shifting was introduced in the Simulacrum extension [20], a technique designed to prevent DOM-based extension fingerprinting by separating the user's experience of a webpage from what the webpage can observe, thereby reducing the amount of extension-related information exposed.

The design and implementation of Simulacrum—the system that applies DOM Reality Shifting—were carried out by Soroush Karami and detailed in our collaborative paper [20]. For clarity, the relevant sections from that paper [20] (though not my contribution) are presented verbatim in the remainder of this background section to support understanding of my own contributions, which will be discussed in Chapter 4.

In Chapter 4, I will present my evaluation of Simulacrum, focusing on its effectiveness in preventing behavior-based extension fingerprinting, as well as its practicality and efficiency as a privacy solution in real-world web environments.

## 2.4 DOM-Based Extensions Anti-Fingerprinting via Simulacrum

Text of this subsection is from [20].

The modern Web has permeated numerous aspects of our everyday lives and, thus, reshaped how we conduct many sensitive and critical operations. At the heart of users' online experience lie web browsers, mediating a wide range of sensitive communications and activities. Unfortunately, while browsers are a portal to limitless potential, their rich set of features and complex functionality can also enable or facilitate privacy-invasive behaviors [53; 54; 55; 56; 57; 58]. As a result, in recent years web tracking has garnered significant attention from researchers and practitioners alike.

Due to the stateless nature of the HTTP protocol, web tracking has traditionally relied on the presence of cookies. However, with users becoming more privacy-cautious and browsers continuing to deploy anti-tracking defenses that hinder cookie-based tracking [59; 60; 61], trackers have also evolved accordingly. In fact, a wide range of techniques have been demonstrated by researchers or found in the wild; from "supercookies" and "evercookies" (e.g., using HSTS policies [62], internal storage [45], or favicons [63]) to DNS-based trickery [64; 65; 66], these techniques highlight the feasibility and creativity of tracking techniques that bypass existing defenses. While browsers may gradually adapt and prevent such emerging techniques, one of the most alarming modern approaches is that of browser fingerprinting. Prior research has demonstrated many browser fingerprinting vectors targeting underlying system and hardware characteristics [51; 7; 67; 46; 11; 49; 9; 10; 68; 69; 70; 71; 72]. Additionally,

a more recent line of research has focused on how installed browser extensions can be used for fingerprinting and tracking, as each user will install a unique set of browser extensions. To make matters worse, installed extensions carry semantic information that can be used to infer sensitive user traits such as religion, sexual orientation, and medical issues [16].

Prior studies have proposed mitigations that target different aspects of extension fingerprinting, namely preventing techniques that target Web Accessible Resources (WARs) [17], stylesheets (CSS) [18], and node attributes [19]. However, no existing defense can effectively prevent DOM-based fingerprinting [16], since many extensions intentionally modify pages in varied and diverse ways and these changes can be uniquely identifiable. Essentially, the root cause of behavior-based extension fingerprinting is that any JavaScript running in the context of the web page can see all the changes and modifications that the installed extensions make to the page.

In [20], we address the root cause of this robust fingerprinting technique, by approaching the problem from a fundamentally different perspective. We propose the notion of *DOM Reality Shifting*, wherein we split the reality that a user experiences when browsing a page from the reality that the page can actually observe. By separating the page's DOM, the user sees the changes that an extension makes to the page, while the page's JavaScript cannot see those changes. In more detail, we create a *Parallel DOM* in addition to the *User DOM*, and mediate access such that extensions query, edit and interact with the User DOM while page JavaScript is limited to interacting with the Parallel DOM. While DOM Reality Shifting is conceptually straightforward it is fundamentally effective against DOM-based behavior fingerprinting, yet correctly implementing it requires handling a myriad of JavaScript idiosyncrasies, corner-cases and real-world complexities that can undermine the security of such a system or the functionality of web applications. To demonstrate the

feasibility of our approach we develop a prototype extension called Simulacrum that implements DOM Reality Shifting *without the need to change the browser*, through a targeted instrumentation of Web API interfaces.

To assess the practicality of our approach we experimentally evaluate Simulacrum across multiple dimensions. First, we demonstrate our system's effectiveness by deploying it against a state-of-the-art automated DOM-based fingerprinting system [16]. Out of 5,793 fingerprintable extensions our system effectively hides the presence of 95.37% of the extensions. We then measure the overhead introduced by our defense and find that it is less than 390ms for half of the websites and $\sim 895ms$ on average. Finally, we manually assess how Simulacrum affects extensions' and websites' functionality and find that all extensions remain unaffected, while major and minor breakage occurs in 12% and 10% of sites respectively. To avoid breakage, our extension's users can allowlist trusted sites.

To further reproducibility in science, we make the source code of our system as well as the list of all domains and extensions used in our experiments available [73].

### *2.4.1 Background and Threat Model*

Browser fingerprinting relies on extracting unique attributes of the user's browser and device. Among those, the list of installed browser extensions can be coupled with other information to build reliable fingerprints. Browser extension fingerprinting is indeed a real-world threat, as LinkedIn was found trying to detect 38 different extensions [15].

Extensions rely heavily on the JavaScript language and customize web pages by modifying the page's DOM. An invasive web page can observe this behavior (i.e., the modifications made to the page's DOM) and use it to construct a set of behavior-based fingerprints (i.e., signatures). When a user visits the page, the invasive page

can use the fingerprints and the changes made to determine the extensions the user has installed. Since the extension modifies the page's DOM, effectively hiding the extension's behavior from the page without breaking the page's functionality is a challenging problem that prior defenses failed to truly address [16].

**Threat model.** We assume the attacker controls a specially crafted web page or iframe that implements DOM-based fingerprinting to uncover the extensions installed in the user's browser. In a nutshell, DOM-based fingerprinting relies on JavaScript that leverages the Web API [74] for direct read and write access to the DOM; by observing and interacting with it, the attacker can deduce which extensions are installed based on how the extensions modify the DOM. We note that social engineering attacks that trick the user into divulging which extensions they have installed (e.g., following a similar strategy to [75]) and side-channel attacks (e.g., timing-based) are considered out of scope for our defense.

**Browser extensions.** Users install extensions in their browsers to expand the browser's functionality and improve their browsing experience [76]. Extensions offer such features via a bundle consisting of HTML, CSS, JavaScript code, and a configuration file called the `manifest`.

*Security model.* To reduce the threat of malicious pages compromising browser extensions [77; 78], browsers employ privilege separation for extensions. Specifically, extensions' `background scripts` have powerful privileges but do not have access to the DOM, and extensions use `content scripts` to access the DOM, which lack the extension's full capabilities. Although `content scripts` have full access to the DOM, they do not run in the `website scripts'` execution environment. That is, neither `website scripts` nor `content scripts` are able to directly access one another because each runs in its own isolated world, while sharing the same DOM.

*DOM-based fingerprinting.* Due to the sharing of the DOM across the isolated

worlds, invasive pages can construct DOM-based fingerprints by observing the extension's modifications to the page's DOM [52]. Prior work [16] has demonstrated how an attacker can automatically construct an extension's fingerprint by capturing the extension's DOM modifications that alter specific fields (e.g., username, password), DOM elements (e.g., images), or text keywords. Then, when a victim visits a specially crafted web page that contains a comprehensive set of elements and features, the fingerprinting framework can compare the DOM modifications to the previously captured fingerprints to identify the extension.

**JavaScript** uses prototypes to share properties (values and functions) and define hierarchical relationships between objects. Prototype-based languages do not use classes to generalize the characteristics of a set of objects. Instead, any object can share its properties with others as part of its prototype chain [79].

*Prototypal inheritance.* Effectively, the prototype chain describes the inheritance hierarchy of an object, and is used when code tries to access an object's attribute. During execution, JavaScript looks for the attribute within the object itself, and returns the attribute if found. Otherwise it recursively searches the prototype chain until it finds (and returns) the attribute or reaches the end of the chain (and returns `undefined`).

*Function overriding.* JavaScript can dynamically update properties of existing objects. Programmers leverage this behavior to *wrap* existing functionalities and customize their behavior. Additionally, JavaScript can override the `getter` and `setter` functions of a prototype's properties to customize prototype modifications themselves. In the Appendix we provide code detailing how Simulacrum overrides prototype values and functions, including getters and setters.

### *2.4.2 DOM Reality Shifting with Simulacrum*

We present Simulacrum, a novel countermeasure against DOM-based extension fingerprinting. Here, we provide an overview of our design and the techniques employed by our system for hiding the presence of extensions from malicious or invasive webpages. The core strategy behind our defense is to create a *split reality* between what a user experiences when browsing a page and what the page can actually observe. Specifically, Simulacrum hides the artifacts created by extensions modifying the DOM by creating an alternate reality for the page that does not include any of the DOM modifications of installed extensions. The user's reality includes not only the page but also all DOM modifications performed by extensions, thus offering users the same browsing experience with enhanced privacy protections.

**DOM reality shifting.** Simulacrum places the webpage JavaScript in a parallel reality that does not contain extensions' DOM fingerprints by creating a simulacrum of the User DOM that omits the extensions' DOM-fingerprints, the *Parallel DOM*, and routing all the webpage's DOM requests to the Parallel DOM. To isolate the altered reality, Simulacrum prevents the webpage from accessing the *User DOM*, the browser-created DOM that the user sees and that contains all the modifications made by the webpage and the user's extensions. In addition, Simulacrum maintains the webpage's altered reality by mediating all access to and ensuring consistency between the two DOMs.

**Modification mirroring.** Simulacrum replicates changes between the User and Parallel DOMs depending on the origin of the request. For modifications originating from the website or user input, Simulacrum replicates the modifications in the Parallel DOM. However, Simulacrum limits modifications made by an extension to the User DOM.

**Query routing.** Simulacrum automatically routes queries to either the User or Parallel DOMs depending on the query's origin. When an extension requests access to the DOM, it automatically accesses the User DOM. However, when the webpage accesses the DOM, our system seamlessly routes the request to the Parallel DOM. As a result, the website is unable to detect DOM-fingerprints left behind by extensions because in the website's DOM reality the extension's changes simply do not exist and, thus, no fingerprintable artifacts are left behind by the extensions' functionality.

**Interception.** Simulacrum controls all requests and performs the necessary replication functionality by wrapping the DOM interface with an intelligent router. We achieve this by being the first script to execute and overriding the appropriate DOM interfaces before the website's JavaScript executes.

### *2.4.3 System Implementation*

This section covers Simulacrum's implementation details and the technical challenges that it addresses to effectively achieve DOM Reality Shifting. Here, we use the following notation: `e` refers to a JavaScript object of type (i.e., respecting the interface) `Element`, and `n` refers to a JavaScript object of type `Node`.

#### 2.4.3.1 Primitives

**Creating the Parallel DOM.** To preserve compatibility, the Parallel DOM created and maintained by Simulacrum must include every aspect of the User DOM *except* for the DOM modifications introduced by the user's extensions. Simulacrum instantiates the Parallel DOM by cloning the User DOM after it receives the `DOMContentLoaded`, which is triggered by the browser after it finishes loading and parsing the page's HTML and without waiting for the stylesheets, images, and subframes to load. Simulacrum maintains the Parallel DOM throughout the page's life cycle by continually

(1) ensuring the Parallel DOM's consistency with the User DOM and (2) preventing the propagation of extension-driven changes to the Parallel DOM.

**Equivalent elements.** Propagating DOM modifications between the User and Parallel DOMs requires a method for accurately mapping equivalent elements across them. We create this mapping using `id` attributes, which are unique within a `Document` [80]. That is, for an element with an `id` value of `"node_id"` in the User DOM, Simulacrum calls `parallelDOM.getElementById("node_id")` to access the equivalent element in the Parallel DOM. When DOM elements lack an `id` attribute, we assign them a unique random value before creating the Parallel DOM. In addition, we handle elements created after the Parallel DOM is instantiated. To handle this, Simulacrum wraps functions that create new DOM elements (e.g., `document.createElement()`) with logic that assigns them a unique `id`.

Simulacrum implements a function, `getEquivalent(e)`, to return the element equivalent to `e` from the other DOM. First, the function checks whether the provided `e` originates from the User DOM or Parallel DOM; if `e` does not originate from either (i.e., `e` is not attached to either DOM), the function returns `e`. Next, if the `id` of `e` exists in both DOMs, then the function returns the element with the same `id` (i.e., the equivalent element) from the non-originating DOM. Otherwise, the function returns `null`. In practice, websites might use the same ID for multiple elements. If the `getEquivalent()` function detects non-unique IDs, it uses `querySelectorAll("[id='node_id']")` to get the list of all elements with `id='node_id'`. Then, if the element is the i-th element with `node_id` in one DOM, the equivalent will be the i-th element with `node_id` in the other DOM. For simplicity, in the remainder of this dissertation we will only mention `getElementById` for finding equivalent elements.

**Cloning.** The standard JavaScript method for making node copies, `cloneNode()`,

does not copy custom properties or the source object's event listeners. To fix the limitations of `cloneNode()`, Simulacrum uses a custom `deepClone()` function. After making a clone with `cloneNode()`, `deepClone()` uses `Object.keys()` to get all the custom properties and transfer them one by one to the cloned node.

Unfortunately, JavaScript does not provide an API for accessing the list of all event listeners for each node. The `Element` and `HTMLElement` interfaces provide APIs for setting and getting event listeners (such as `HTMLElement.onclick`), and `deepClone()` uses these APIs to check for and clone event listeners on the node. However, JavaScript does *not* provide a direct way for accessing the event listeners that are set using `EventTarget.addEventListener()`. Simulacrum overrides this function to intercept all invocations of it and collect (and, later, clone) the event listeners that are set for each node.

**Referencing original functions.** Simulacrum needs access to the original versions of the functions it overrides to affect its operations on the DOMs. Before overriding them, we store the methods and functions of these interfaces in a hash table (accessible only by the Simulacrum content script).

#### 2.4.3.2 DOM-Accessing APIs

The DOM API is incredibly complex, comprised of hundreds of interfaces and thousands of functions. However, we only need to override APIs that have either read or write access to the DOM; Simulacrum wraps the original functions with logic to restrict the webpage's access to the User DOM.

**Read access.** Simulacrum wraps functions to ensure that JavaScript code run by a webpage reads from the Parallel DOM. For example, `Element.querySelector()` has read access to the DOM, and we override it to query the parallel DOM.

**Write access.** Simulacrum wraps functions with write access. The write access

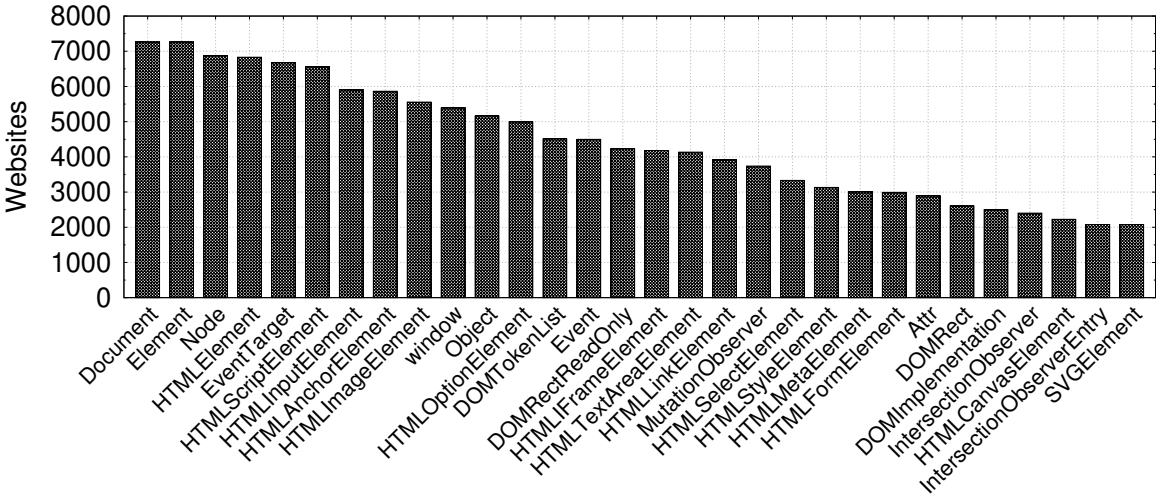


**Figure 2.1:** Most prevalent prototypes across top 10k websites.

wrappers apply the necessary DOM modifications that originate from website scripts to both DOMs and ensure that they are synchronized (i.e., consistent). For example, `Element.append(<newElement>)` has write access and the wrapper appends `<newElement>` to both DOMs.

**Identifying APIs.** While `Document`, `Element` and `Node` are obvious items to be added to our overriding list, an automated and systematic approach was necessary for identifying other commonly used interfaces to read from or write to the DOM. We developed a Chrome extension, called `VisibleJS`, that records all function invocations that occur during the execution of JavaScript code on a page. `VisibleJS` overrides all the functions implemented by JavaScript and logs the functions each time the page's code invokes them, using the code shown in Listing 7 (Appendix). For each function invocation, `VisibleJS` analyzes the arguments, returned value, and receiver. If at least one of those is an instance of `Node` or a DOM-related object (e.g., `DOMTokenList` and `DOMStringList`), then it records the function as one with access to the DOM.

Inspired by prior approaches on quantifying the prevalence of browser features [81], we chose to use the 10K most popular sites (according to the Alexa ranking) to determine commonly used interfaces. We use Selenium, ChromeDriver, and `VisibleJS`

to capture the functions used by each site that access the DOM. Figure 2.1 shows the number of websites that use at least one function of the 30 most popular interfaces. Overall, our experiment uncovered a total of 135 interfaces (and 1,532 functions) that interacted with the DOM among the top 10K websites. However, as one might expect, not all interfaces are equally popular, as more than half (70) of the interfaces are used by less than 1% of the websites. Balancing the popularity of the functions and the effort required for manually overriding each function, we decided to override the 75 most popular interfaces. In addition, we override all interfaces that inherit from the `Node` interface (some of them are not used by the top 10K websites). In total, Simulacrum overrides 156 interfaces.

VisibleJS lead us to certain interesting findings, which shed light on the intricacies and complexities of the DOM API's interfaces. For instance, we found it surprising that `XMLHttpRequest` interacts with the DOM. `XMLHttpRequest` is a commonly used interface for interacting with servers. However, `XMLHttpRequest`'s `responseXML` argument returns an object that implements the `Document` interface and can then be appended to the DOM.

#### 2.4.3.3 Function Overriding

Here, we detail the processes Simulacrum employs for overriding read and write functions available through the DOM API.

**Categorization.** We first categorize functions depending on their type of DOM interactions. Table A.2 (Appendix) displays different function examples for each category with their overridden counterpart. We provide simplified versions of the actual call invocations throughout the text to improve readability.

*Simple getter.* This category of functions returns static information about the DOM. For example, `document.getElementsByTagName('div')` returns all the `div`

elements of the DOM, and `e.hasAttribute('src')` returns `true` if the object `e` has a `src` attribute. Simulacrum overrides these functions so they return the result of their execution on the parallel DOM. Accordingly, it replaces the examples given above with `parallelDOM.getElementsByTagName('div')` and `parallelDOM.getElementById(e.id).hasAttribute('src')`.

*Active getter.* This category of functions cannot be executed on the Parallel DOM. For example, `e.scrollTop` returns the number of pixels that the element's content is scrolled vertically. However, since the Parallel DOM is not visible to users, this value will always be zero for any element tested. As such, for this type of function we first check if `e` has an equivalent in the Parallel DOM (thus ensuring that it was not the product of an extension), and then run this function on the User DOM to obtain the appropriate value that should be returned.

As another example, `document.activeElement` returns the element in the DOM that currently has focus. Since the Parallel DOM is not visible, its elements cannot have focus. Thus, we run this function on the User DOM and return the equivalent element on the Parallel DOM. If the element that is currently focused was created by an extension, we return its first ancestor not added by an extension (i.e., its first ancestor that has an equivalent in the Parallel DOM).

*Simple setter.* Functions in this category modify the structure of the DOM or a node's attribute. To replicate such DOM modifications, we run the function on both the function receiver and the equivalent of the receiver. For instance, when a webpage's script invokes `e.innerHTML = "text"`, the wrapper checks that the `id` exists in the Parallel DOM. If it does exist, then the wrapper sets the `innerHTML` of `e` and its equivalent object. For example, if the Parallel DOM owns `e`, then the wrapper runs `e.innerHTML = "text"` and `document.getElementById(e.id).innerHTML="text"`.

*Active setter.* Functions in this category do not directly modify the DOM. As such,

we simply direct these to the User DOM. For example, `e.requestFullscreen()` results in the element being displayed in full-screen mode. The wrapper for this function first checks the element's owner document. If the owner document is the Parallel DOM, we find its equivalent element in the User DOM and run this function on it. Otherwise, we simply execute the function.

**Forwarding arguments.** It is important to note that any of the functions in the mentioned categories might have arguments. Almost all arguments can simply be passed to the corresponding functions. However, if an argument is a DOM object (an object of type `Node`, `Element`, etc.), the wrapper might need to invoke the function with the equivalent of arguments.

For instance, `parentNode.insertBefore(newNode, refNode)` inserts the `newNode` before the `refNode`, which is a child of `parentNode`. In this case, Simulacrum uses the overriding strategy used for the *simple setter* category. That is, it runs this function two times, once for `parentNode` and once for its equivalent one. Unlike the *simple setter* category, the `insertBefore` function receives two DOM objects as arguments and, as a result, the wrapper locates and passes the arguments `equivalentNewNode` and the `equivalentRefNode`. While the wrapper uses `getEquivalent(refNode)` to find the equivalent version of `refNode`, the `newNode` does not have an equivalent node because it is not connected to a DOM yet. Thus, the wrapper creates a `deepClone()` of `newNode` and uses that for `equivalentNewNode`.

**Interfaces of the DOM API.** The DOM API is structured around different *Interfaces* effectively grouping JavaScript objects with common state and behavior together. Note that all objects in JavaScript are instances of the `Object` class, which is on the top of the prototype chain. For the interested reader, Figure A.1 (Appendix) shows a partial representation of the DOM API with some of its interfaces.

**Node interface.** Every node of the DOM tree is represented by an object of type

`Node`, which also includes any interfaces inheriting from it (notably `Attr`, `Document`, `Element` [80]). The `Node` interface describes properties (e.g., `nodeName`, `parentNode`) and methods (e.g., `cloneNode()`, `normalize()`) that are shared amongst all DOM objects [80]. We wrapped 14 properties and 15 methods in `Node`'s prototype.

*Document interface.* This interface represents the result of parsing the page, and grants access to the DOM. It describes the common properties (e.g., `title`, `bgColor`) and methods (e.g., `querySelector()`, `createElement()`) for any kind of document [80]. We note that this interface exposes properties for setting event handlers (e.g., `onclick`,) [80]; these are in the active setter category, and we set these event listeners in the User DOM. In practice, the User DOM fires the events because the Parallel DOM is in the background. During the execution of callback functions from event handlers, as with any other JavaScript function, Simulacrum applies modifications to both the DOMs. The `Document` interface also allows the creation of specific DOM objects [80]. In particular, Simulacrum needs to wrap the `createElement()`, `createElementNS()`, and `createDocumentFragment()` methods to forcefully assign a value to the `id` attribute of new elements created through this interface (see §2.4.3.1). Finally, `createTreeWalker()` and `createNodeIterator()` instantiate objects that help JavaScript perform DOM traversal [80]; Simulacrum uses the *simple getter* strategy to prevent them from reading the User DOM. More specifically, Simulacrum's wrapper for `document.createTreeWalker(root)` converts it to `parallelDOM.createTreeWalker(equivalentRoot)`. This limits the webpage's access to the Parallel DOM. In the end, we override 264 properties and 40 methods.

*Element interface.* All elements in the DOM inherit a set of methods and properties common to all types of elements from `Element`. As shown in Figure A.1, each element object derives from either `HTMLElement` or `SVGElement`. Both of them count several descendant interfaces: 72 for `HTMLElement`, and 71 for `SVGElement` [80]. `HTM-`

LElement serves as the base interface for HTML elements. Some elements directly derive from this interface, while others implement this interface using another interface that inherits it. For example, <footer> elements directly implement HTMLElement, while <video> elements implement HTMLVideoElement, which inherits from HTMLMediaElement, which inherits from HTMLElement. Additionally, all the SVG elements of the SVG language inherit the SVGElement interface. Simulacrum wraps all the methods and properties of the Element interface and its descendants with an appropriate overriding strategy. To that end, we manually analyzed all 1,532 methods and properties, so as to choose the correct overriding strategy for each case.

**Observer interfaces.** The Web APIs provide different observers allowing JavaScript executed in the browser to observe, be notified, and react (using callback functions) to updates in the state of DOM objects. Simulacrum needs to override three observers: ResizeObserver, IntersectionObserver and MutationObserver [82; 83; 80]. Listing 8 (Appendix) gives examples that demonstrate the three observers.

*ResizeObserver interface.* Objects of this interface monitor changes to the size of elements (size may change for various reasons, such as changes in the size of the browser window). The ResizeObserver's constructor receives a callback function. Each time the size of an observed element changes, the browser notifies the observer by executing the provided *callback* function. The observe method starts the observation of the specified element and identifies the target of interest. We use the active setter strategy for overriding the observe function. That is, Simulacrum prevents a webpage from observing an element that does not exist in the Parallel DOM.

*IntersectionObserver interface.* This interface provides the ability to observe changes in the intersection of a target element with another object called the *root*. The IntersectionObserver's constructor receives a callback and an optional configuration variable which, when provided, defines the root. The root descends from a spe-

cific target or the viewport. If the constructor does not include the configuration argument, the observer assigns the viewport as the root. Simulacrum wraps two functions: first, the `IntersectionObserver`'s constructor to configure a proper root for its object. To do so, the wrapper checks that the configuration argument contains a root and that it exists in the Parallel DOM. If so, it uses `userDOM.getElementById(root.id)` to replace the root with the one from the User DOM. Second, we wrap the `observe` function using the active setter strategy, which prevents pages from using this API to detect extensions.

*MutationObserver interface.* This interface allows scripts to observe changes made to the DOM. The `MutationObserver`'s constructor receives the callback function. The `observe` method receives the target node and the observer configuration. The configuration determines the types of DOM changes the observer will react to. For example, by passing a configuration with `childList: true` and `subtree: true`, the observer triggers the callback for DOM modifications that change the list of children attached to the target node or the nodes in its subtree. The list of children changes through the addition or removal of nodes. The `observe` wrapper uses a slightly more complicated version of the active setter strategy. The `MutationObserver` invokes the callback when it observes modifications to the target node and the target node's descendants. For instance, if the target is the `<html>` element, the observer receives notifications for new elements that the browser appends to any location in the page. Subsequently, the wrapper adds functionality to avoid invoking the callback when an extension mutates the DOM. To that end, Simulacrum overrides `MutationObserver`'s constructor to modify the incoming callback. The callback modifications cause it to skip DOM mutations originating from an extension and only invoke the original `callback` for those DOM changes that originate from the webpage's scripts. Listing 9 in the appendix shows the wrapper code. The wrapper replaces the orig-

inal callback with `newCallback`. The `argument[0]` for `newCallback` is an array of mutation records observed by the `MutationObserver`. The wrapper uses the `filterMutations()` function to filter out the mutations caused by an extension. If any mutations remain, the wrapper invokes the original `callback` with the remaining webpage-only mutations.

The `filterMutations()` function operates based on the attributes of each mutation's record. For example, if an extension appends a new `<p>` element to the User DOM, `mutation.type` will be `"childList"`, the value of `mutation.target` will be the reference to the parent of the `<p>` element (i.e., the element that it has been appended to) and `mutation.addedNodes` refers to the `<p>` element. The `filterMutations()` function then checks the Parallel DOM for its equivalent of `<p>` and `mutation.target`. In this example, since `<p>` is injected by an extension and is not a child of the Parallel DOM's version of `mutation.target`, the function filters out the current mutation from the mutation array.

In another example, if one of the webpage's script modifies the `src` attribute of an `<img>` element, the `mutation.type` will be `"attributes"`, the `mutation.target` will be a reference to the `<img>` element, and the `attributeName` will be `"src"`. Based on `mutation.type`, the `filterMutations()` investigates the attributes by comparing the `src` attribute of the `<img>` element with the `src` of its equivalent from the non-originating DOM. In this case, it does not filter out the current mutation out of the mutation array because the `src` value from both DOMs are equal. Thus, the wrapper invokes the original `callback` with the remaining mutations.

**Other interfaces.** We use an element's `id` to find equivalents in the two DOMs. Objects that do not implement the `Element` interface and lack an `id` require a different approach; the key observation for these objects is that they are connected to an element. Thus, to find their equivalent we leverage the element they are connected to.

E.g., `element.classList` returns an object that implements the `DOMTokenList` interface, which has functions like `add()` for modifying the object. When the page calls `tokenList.add("name")`, we also call `equivalentTokenList.add("name")` to propagate this to the other DOM. Since `tokenList` does not implement `Element`, we use `element` to find the equivalent, and `equivalentElement.classList` to set `equivalentTokenList`. Accessing the connected element differs for different non-element objects. Some non-element objects implement properties, for example `ownerElement`, `parentElement`, or `parentNode`, which provide direct access to the connected element. However, other objects do not provide a direct mechanism for accessing their connected element (e.g., `DOMTokenList`). For indirect cases, Simulacrum automatically adds an owner element property to these objects the first time they are called. For example, `element.classList.add("name")` modifies the object that is returned by `element.classList`. Indeed, the call results in two API calls: 1) `element.classList` returns a `DOMTokenList`, 2) `tokenList.add("name")` adds the string to the list. In the wrapper function of `classList`, the wrapper sets `element` as the owner element of the returned `DOMTokenList` object. Therefore, in the `add()` function, Simulacrum can get the owner element, which it can then use to find the owner of non-element's equivalent.

It is important to emphasize that knowing the owner element is not sufficient for finding the equivalent object. In addition to the owner element, Simulacrum needs to know the function that was called for getting this object. For example, the `Element.classList`, `HTMLLinkElement.relList`, and `HTMLIframeElement.sandbox` properties return an object that implements the `DOMTokenList`. After finding the equivalent owner element, we need to run the proper function to get the equivalent non-element object. To do this, the wrapper function stores the API call for accessing the object, which the wrapper later uses to get the equivalent non-element object.

#### 2.4.3.4 Additional Security Precautions

During our design and development process we accounted for potential attacks against Simulacrum and incorporated appropriate defenses to prevent circumvention of our protections.

**IIFE.** As mentioned in §2.4.3.1, Simulacrum stores the original version of all the interfaces in a hash table. If attackers are able to access the original interfaces, they can replace the overridden functions with the original ones and access the User DOM. To prevent this attack, Simulacrum leverages Immediately-invoked Function Expressions (IIFE) for isolating functions and variables. That is, we encapsulate our system in an anonymous function that the JavaScript environment executes immediately. In essence, this allows us to define our logic within another function while the inner functions have access to the variables in the outer function's scope. Using this approach Simulacrum has access to the global scope of the webpage but prevents the webpage's JavaScript code to access objects like the hash table of original interfaces that are used by Simulacrum.

**Position-based attacks.** Simulacrum executes *active getter* functions on the User DOM. While they cannot be executed on elements injected by extensions, they can be used to get the position of other elements. Since injected elements can affect other elements' position, attackers could potentially identify installed extensions by observing changes in the positions of elements. To prevent this, Simulacrum adds `<div>` and `<span>` elements with random sizes at different locations throughout the User DOM. Since these do not exist in the Parallel DOM they are not visible to the websites' scripts and, thus, attackers cannot infer whether changes to the positions of elements occur due to injections by extensions or the random noise introduced by our system. It is important to note that these noise elements do not really impact

the user's browsing experience as they simply introduce empty spaces around other elements (see Figure A.3 in the Appendix).

While adding noise prevents attacks against extensions that inject elements, this might not work for extensions that remove elements. If an extension removes an element from the User DOM, the element will not be removed from the Parallel DOM; however, from the changes in the location of other elements an attacker might be able to infer that extension's presence. There are two categories of extensions that remove elements from websites. The first category is extensions that block scripts which are responsible for creating elements (e.g., ad-blockers). The second category includes extensions that explicitly remove elements which are already connected to the DOM. While Simulacrum cannot observe the behavior of the first category, the behavior of the second category is DOM-based and can be detected by Simulacrum. To that end, when an element is removed by an extension, Simulacrum replaces the removed element with another element of the same size; this prevents changes to the location of other elements. We have experimentally found that such extensions are very rare (only 59 fingerprintable extensions from §4.1 remove an element). As such, Simulacrum only activates this defensive mechanism against element removal when the user has one of these 59 extensions installed; in our prototype, this is done through a hard-coded list but can easily be enforced based on a list that is fetched from a server (similarly to EasyList [84]).

**Order of execution.** If the attacker's code executes before Simulacrum, they can access the original interfaces. To prevent this, Simulacrum injects its script into the webpage's execution context prior to DOM construction and before the browser executes any other script. Simulacrum achieves execution priority by setting `run_at: "document_start"` in the extension's manifest, which causes the browser to execute the extension's content scripts at the beginning of DOM construction and before

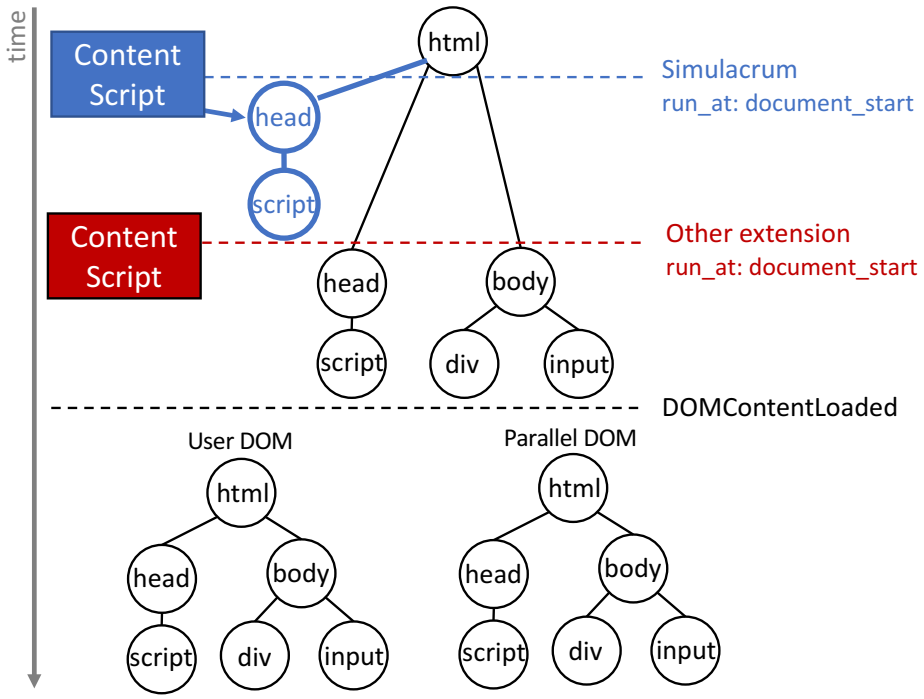


**Figure 2.2:** Different stages in a webpage's lifecycle.

any of the webpage's scripts. In this way, we guarantee that the webpage cannot recover the originals of the functions that we override by preemptively executing before Simulacrum's content scripts

However, when more than one extension sets `run_at: "document_start"`, the execution order is decided by the browser, based on the order of their installation time: the extension that was installed first will be executed before later-installed ones [85]. To ensure its prioritization, Simulacrum leverages the reordering technique proposed by Picazo-Sanchez et al. [85]. This approach uses the *management* permission to change the order of extensions in the execution pipeline by disabling and re-enabling them, resulting in these extensions being treated as if they had been installed after Simulacrum.

Figure 2.2 shows the different stages of loading a website when Simulacrum is present. The content script of Simulacrum runs at `document_start`. At this point,

the browser has not started to parse the HTML page or construct the DOM. Simulacrum's content script creates a `head` element and appends its script to it. This script overrides the interfaces mentioned previously and then removes the `head` element from the page to prevent any conflict with the website's head element. Simulacrum reorders the execution pipeline of content scripts and will be the first script that executes in the webpage. Moving to the next area of the diagram in Figure 2.2, the browser starts to construct the DOM. Right after constructing the DOM, it fires the `DOMContentLoaded` and Simulacrum clones the DOM to generate the Parallel DOM. From this point forward, extensions' fingerprints remain on the User DOM and they will not be accessible by website scripts. Simulacrum's event listeners also fire before the webpage's and the other extensions. JavaScript orders the firing of event handlers based on when the listener was set. That is, even if multiple scripts set a listener for the `DOMContentLoaded` event, the browser invokes the Simulacrum's callback first (creating the Parallel DOM) because it is the first script that set a listener for the `DOMContentLoaded` event. In addition, JavaScript executes as a single thread; therefore, Simulacrum's event handler blocks other events from firing until Simulacrum's event handler completes (i.e., there is no race condition).

**iframes.** Simulacrum uses the `"all_frames": true` manifest file setting to inject its content scripts into all `iframes` so that it can protect against fingerprinting by code running in these `iframes`. However, due to browser implementation subtleties, `iframes` with a blank or undefined `src` attribute are not covered by the `all_frames` setting. Thus, an attacker can append `<iframe id="ifrm"></iframe>` to the DOM, retrieve the *original* version of `getElementById` through the `ifrm.contentWindow.Document.prototype.getElementById` lookup chain, and bypass Simulacrum. To inject Simulacrum's content scripts into such `iframes` (as well as `iframes` with src attributes of `javascript:*` or `about:*`), Simulacrum includes `"match_about_blank":`

`true` in its manifest. This setting successfully injects the content scripts into all `iframes`. Interestingly, this bypass vector has been overlooked by some previously proposed countermeasures (e.g., [18]).

Another `iframe`-related circumvention can occur when fingerprinting code running in the parent page of an `iframe` exploits a race condition between the time points when an `iframe` is created and when Simulacrum's scripts override its DOM. The resulting attack is identical to the previously-stated lookup chain, depending on quick access to the original elements. Though this was reported in 2017 [86], it was never fixed, and popular privacy extensions (e.g., Privacy Badger [87]) are vulnerable to this attack. We prevent this attack by blocking all accesses to the `iframe`'s `contentWindow` and `contentDocument` from the parent page before Simulacrum has wrapped the `iframe`'s functions. To that end, the wrappers of `contentWindow` and `contentDocument` check Simulacrum's execution state inside the `iframe`; if it has completed, they return the function's result or `undefined` otherwise.

**Overriding prototypes.** The overriding approach from [88] would allow attackers to delete functions and revert them to the original function pointers. To prevent this and protect objects the authors used `Object.freeze` after the virtual machine layering process. In Simulacrum, as shown in Listing 5 (Appendix), we override prototypes and not objects. Our approach has two advantages; first, deleting functions will not revert them to the original ones. Second, since functions are not frozen, Simulacrum allows websites and extensions to wrap functions (i.e., they can add another layer on top of ours) which allows them to maintain their functionality.

Chapter 3

# CONTEXT-SENSITIVE CONTENT INJECTION MITIGATION IN WEB APPLICATIONS

## 3.1 Introduction

Though rich, interactive, Web 2.0 applications are critical in enabling the modern Web, they are also a critical attack vector on the Internet. Web application vulnerabilities have significantly contributed to the financial loss from cybersecurity issues over the past years. Among vulnerabilities in web applications, Cross-Site Scripting (XSS) is among the *most common* type of vulnerabilities based on reports such as Bugcrowd's 2022 [89] and Hackerone's 2021 [90] reports. In fact, since the release of the initial US-CERT Advisory in 2000 [91], XSS is among the most critical web application security threats every year, consistently appearing in "worst of" lists, such as the OWASP Top 10 [92] and MITRE top 25 [93].

Research into XSS prevention and mitigation has continued since XSS was first discovered. Existing approaches attempt to statically identify XSS vulnerabilities in server-side code [94; 95; 96], analyze the use of server-side sanitization functions [25; 27], filter out JavaScript code on the server side [28; 97], or attempt to detect the presence of vulnerabilities from the client's viewpoint [41; 36]. Yet, XSS vulnerabilities continue to manifest in web applications. A different class of solutions attempt to *mitigate XSS exploits* rather than detect the underlying vulnerabilities. These solutions include XSS filters in browsers (e.g., NoScript for Firefox [38]), Content Security Policy (CSP) [2], web application firewalls (e.g., ModSecurity [42]), and HTML sanitizers [30]. These mitigations are widely adopted in practice (as I will

discuss in Section 3.2.2), however research has shown that it is possible to bypass these defenses.

One particularly interesting bypass of XSS mitigations, called scriptless attacks, generalizes the concept of XSS *beyond* the injection of JavaScript code [98]. Scriptless attacks inject data (such as CSS, HTML 5, SVG, and font files) to compromise the security of web applications, which, as a result, allows attackers to steal sensitive information, even in a restricted environment without JavaScript execution. Prior solutions that check if untrusted user input is used in sensitive output functions (e.g., `echo()` or `system()`) [95; 99; 100], or identify the mixing of code (HTML) and data (JavaScript) in the same channel [28], do not work for scriptless attacks, as the notion of code-data mixing is not fine-grained enough.

These exploit techniques inspired me to examine the root cause of XSS and scriptless attacks: Web applications embed into an HTML page untrusted user input as *pure data*, such as strings and text, with the intent that no part of the untrusted user input shall be interpreted by the parser as non-data. However, this important developer-intent is lost once the HTML code is generated and sent to the browser: client-side parsers must *re-discover* the meaning of every part of the page by parsing the HTML response. In carrying out XSS exploits, attackers exploit this loss of information and mislead client-side parsers to transition from HTML parsing to JavaScript or CSS parsing when parsing the user input, which *violates the developer's original intent.* For example, in an XSS exploit, an untrusted user input may cause a browser to transition from parsing plain text in HTML to parsing JavaScript code with a `<script>` tag. I term these context transitions in a parser *context switches*, where "context" refers to the functionality of the token being parsed. In fact, I argue that unintended context switches are the root cause of a series of vulnerabilities including (among others) XSS, scriptless attacks, command injection, SQL injection, XML in-

jection, and template injection. I will refer to them as *content injection vulnerabilities* throughout this dissertation.

My insight is that, because these vulnerabilities share a common root cause, exploits against them can be mitigated using a common approach. In this dissertation, I introduce Context-Auditor, a novel, general technique that detects content injection exploits by identifying unintended context switches, caused by untrusted input, during parsing. This idea is inspired by Stock et al. [44], who used context switches (tokenization-based) in the JavaScript parser to detect DOM-based XSS exploits using a taint-tracking browser and by other string-based [101] and taint-based [102] injection prevention methods. Context-Auditor expands these ideas, generalizing these concepts to the broader category of content injection exploits.

I have implemented multiple prototypes of Context-Auditor in various forms—including a shell wrapper, an nginx module, a web proxy, and a Chrome extension—to detect content injection exploits in shell commands, HTML, CSS, and JavaScript. I tested these prototypes on reputable testing suites and comprehensive real-world data sets, where Context-Auditor successfully detected and blocked all reflected XSS, scriptless, and command injection exploits in a number of web applications. Additionally, Context-Auditor demonstrated negligible false positive rate in a live crawl of the Alexa top-1000 websites. In the spirit of open science, I have open source Context-Auditor and published the evaluation data and configurations to guarantee experiment reproducibility [103].

This chapter is organized as follows: I will introduce content injection vulnerabilities and their prior mitigation techniques in Section 3.2. In Section 3.3, I will demonstrate the context switching concept with a closer look into the expected behavior of client-side parsers and how a piece of untrusted input can change their behavior. With that, I will propose Context-Auditor's idea at high level by this section's end. I

have designed Context-Auditor as a two-way pushdown automaton, and I will dig more into Context-Auditor's theoretical details in Section 3.4. Since I have proposed a stand-alone detection module by Context-Auditor, I deployed it in multiple configurations demonstrated in Section 3.5. Section 3.6 showcases evaluation results of Context-Auditor in three aspects, including effectiveness, practicality, and efficacy, and Section 3.7 discusses limitations of our mitigation approach. Related work is presented in Section 3.8. Finally, I will conclude this chapter in Section 3.9.

## 3.2 Background

In this section, I provide an overview of content injection exploits that Context-Auditor aims to address along with an overview of existing mitigation techniques.

### *3.2.1 Content Injection Vulnerabilities*

A web application that allows the usage of user data with unintended semantics in generating dynamic content is susceptible to content injection vulnerabilities. Therefore, an attacker can compromise the security of the web application by sending an exploit that leverages this vulnerability to perform developer unintended actions. For instance, if user input is used to construct a string that is then passed to a shell script command, an adversary can construct an exploit using shell script control characters, such as semicolon, to escape out of the current command and execute arbitrary commands. In another case, an attacker may alter the intended semantics of a web page or execute arbitrary JavaScript code if a web application uses user input to construct HTML response without proper sanitization.

**Web-based exploits.** We classify content injection exploits that manipulate the structure of an HTML response as *web-based exploits*, which are categorized into two groups: *scripting* exploits and *scriptless* exploits. Listing 1 is an example of vulnerable

PHP code with three web-based content injection vulnerabilities (and DOM-based XSS) in different contexts (HTML, JavaScript, and CSS), and we will refer to it throughout the dissertation.

- **Scripting exploits.** Scripting exploits are a traditional and well-known attack vector that involve the injection of malicious JavaScript code into web pages. An example of scripting exploits is a Cross-Site Scripting (XSS) exploit. Two of the vulnerabilities in the PHP code in Listing 1 can be exploited with scripting exploits. Both are reflected XSS vulnerabilities: One is in the HTML context on Line 20, and the other is in the JavaScript context at Line 17. The first vulnerability on Line 20 can be exploited by a classic HTML-context XSS exploit: `<script>alert('injection');</script>`. The second vulnerability on Line 17 can be exploited by a JavaScript-context XSS exploit: `Hi"; alert('injection');"There`.

- **Scriptless exploits.** Scriptless exploits are another type of content injection exploits where attackers embed into the DOM tree non-scripting elements (e.g., images or style sheets) that violate security policies in browsers, most notably, CORS policies [98]. Listing 1 has a vulnerability on Line 5 that can be exploited with a scriptless content injection exploit. Attackers can inject a malicious CSS context exploit, such as the example exploit in Listing 2, to leak the characters used in the secret Cross-Site Request Forgery (CSRF) token.

**Non-web-based exploits.** Content injection exploits happen not only in client-side web languages, but also in many other contexts, such as SQL queries and shell commands. For the `exec` function (Line 24 of Listing 1): the shell command includes a user-controlled string which enables an attacker to execute arbitrary shell commands:

```
1  <html>
2  <body>
3  <style>
4  body{
5      background-color: <?php echo($_GET["color"])?> }
6  </style>
7  <form action="index.php">
8      background Color :
9      <input type="name" name="color" />
10     <input type="submit" value="Change Color" />
11     <input type="hidden" name="CSRFToken" value="SECRET">
12 </form>
13 <script>
14     document.write("Username is: ");
15     var str = "<?php
16     echo($_GET["id"])?>" ;
17     document.write("<text>"+str+"</text>");
18 </script>
19 <h2> You were searching for:
20     '<?php echo(£_GET['term']) ?>'
21 </h2> Here is the result:
22      <?php
23      £command= 'cat userinfo.txt | grep '. £_GET['term'];
24      echo(exec(£command)); ?>
25 </body>
26 </html>
```

Listing 1: PHP code with three content injection vulnerabilities (Line 5, Line 17, and Line 20 represent CSS, JavaScript and HTML contexts respectively) in the server-side HTML response, one DOM-based XSS vulnerability on Line 18 and a content injection vulnerability in the form of command injection on Line 24.

```
} input[name="CSRFToken"][value*='A'] {
  background: url(attacker.com?A); } ...
input[name="CSRFToken"][value*='S'] {
  background: url(attacker.com?S); } ...
input[name="CSRFToken"][value*='Z'] {
  background: url(attacker.com?Z);}  ...
```

Listing 2: An example CSS context exploit used to exploit the scriptless content injection vulnerability shown in Listing 1 (idea from [98]).

sending `Auditor; rm userinfo.txt` to the web application would cause it to remove the `userinfo.txt` file.

### *3.2.2 Mitigations of Content Injection Exploits*

Generally, there are four types of mitigations for web-based content injection exploits (although thus far they have mostly focused on XSS exploits), and all are bypassable [100]:

**Browser-based XSS Filters.** Some browsers have built-in XSS detection and mitigation mechanisms to block malicious-looking HTML requests and responses. NoScript [38] (for Firefox) is a powerful browser XSS filter that is widely used. There is also a No-Script Chrome extension [39]; it could be used as a temporary XSS mitigation measure after the retirement of XSS-Auditor [40] from the Chrome browser. Detection capability of XSS filters is often limited to pattern matching solutions that are defined by regular expressions and they might become ineffective in the face of new exploits.

**HTML Sanitizers.** HTML sanitizers (e.g., DOMPurify [30]) are libraries used by web application developers to sanitize HTML text and filter (potentially malicious)

content. The black-listing approach of sanitizers is usually very conservative, as characters are sanitized without a thorough understanding of their actual impact on the semantics of a web page. For instance, not all < characters inside HTML text are potentially harmful and need filtering. However, sending a string containing < character to DOMPurify, results in filtering of < and all of its following characters; although the original string might not eventually cause any security violation in the web application. Unsurprisingly, sanitization approaches cannot cover all possible content injection exploits (e.g., DOMPurify does not provide sanitization for JavaScript and CSS).

**Content Security Policy (CSP).** CSP is a white-listing mechanism that adds directives into HTTP headers or meta tags, which specify (among other things) the legitimate source of external resources that a web page can embed. In this way CSP, when used correctly, can mitigate many XSS exploits. While all modern browsers support CSP, it is not deployed properly by most web applications in the wild, and also previous work has shown it to be insufficient to prevent all XSS exploits [32]. The application of strict CSP policies, however, does not guarantee the security of a web application in case the JavaScript code is already vulnerable. Hayes [104] demonstrated a bypass of a nonce-based CSP policy for a vulnerable JavaScript code with a flow from an HTML tag's attribute to a sensitive sink. Similar to the exploitation of JavaScript gadgets [100], an attacker can exploit such vulnerabilities via injection of specially-crafted HTML tags into a webpage: JavaScript gadgets misuse existing pieces of JavaScript code in the web application to achieve their aim (alike return-oriented programming [105]). In this vulnerable script, the attacker can modify the source of an existing script tag (that has already been protected by nonce-based CSP tokens) to the attribute value of an injected input tag shown in Listing 3 and execute arbitrary JavaScript code. Our context switching-based analysis prevents such

```
<input id="RecaptchaClientUrl-" value="//portswigger-labs.net/xss/xss.js" />
```

Listing 3: An HTML tag injection proposed by Hayes [104]: This exploit miss-uses a JavaScript gadget in a vulnerable JavaScript code and bypasses nonce-based CSP policies.

```
GET /?id=Admin";alert(1);" HTTP/1.1
Host: vulnerable.com
```

Listing 4: An HTTP request towards an nginx web server with ModSecurity enabled via OWASP core rule set (CRS).

HTML tag injections; on the contrary, CSP prevents the injection of a limited set of HTML tags.

**Web Application Firewall (WAF).** WAFs (e.g., ModSecurity [42]) attempt to detect malicious web requests and prevent them from reaching back-end web applications. WAFs need to be manually configured with a comprehensive set of rules or directives, comprehensibility of which correlates with WAFS detection capability. To demonstrate this, we configured the ModSecurity with the OWASP core rule set (CRS) [106], and it failed to detect a simple injection inside JavaScript code (Listing 4) against the web page of Listing 1. WAFs are also ineffective in preventing JavaScript gadget exploitations similar to Listing 3 since WAFs' directives do not usually consider such HTML tag injections malicious.

## 3.3 Overview

The failure of state-of-the-art content injection mitigation techniques (Section 3.2.2) is due to prior approaches not addressing the root cause of context switching vulnerabilities. Parsers (e.g., HTML parser, JavaScript parser, shell parser, etc.) are

the entities that eventually parse an exploit, yet they do not know if a context switch is triggered by attacker content or was developer intended. Therefore, in Context-Auditor, I suggest a fundamentally different approach: I model parsers using automata and detect any context switching caused by attacker-controlled input as a potential content injection exploit. Before discussing the details my context-switching-based detection approach, I first highlight the deficiencies of state-of-the-art techniques (motivating us to introduce a new detection approach) and the context switching concept. These prerequisites provide a high-level operational model of Context-Auditor.

### *3.3.1 Motivation*

State-of-the-art mitigations attempt to identify characteristics of common exploits as malicious: NoScript and XSS-Auditor operate based on regular expression matching. Similarly, ModSecurity searches for known malicious-looking directives inside HTTP traffic, and DOMPurify identifies known potentially harmful characters or patterns. CSP only allows the inclusion of external files from the same domain as the origin. These black-listing (browser filters, ModSecurity, DOMPurify) or white-listing (CSP) approaches operate based on previously known patterns among content injection exploits, which makes them unprepared for unfamiliar exploits. Still, detecting the known patterns requires preparation of extensive regular expressions or a comprehensive set of directives, which is an error-prone and tedious task. More specifically, I argue that many mitigation techniques do not provide a comprehensive detection approach against content injection exploits because they fail to address content switches caused by untrusted input.

Bypass of ModSecurity (Listing 4) occurs as a consequence of context switching from the double-quoted string context in the JavaScript parser to after assign-

ment, then to statement contexts. State-of-the-art techniques cannot detect context switches due to user-controlled data in web applications. Therefore, I recommend a fundamentally new content injection exploit detection approach in CONTEXT-AUDITOR that uses the context switching concept to detect content injections.

### *3.3.2 Context Switching*

I define *context switching* as when a parsing engine changes parsing context (i.e., from one context to the next) based on the input. The parsing context is defined by the grammar rules of the specific language. For instance, consider the following grammar rule in ECMA262 (JavaScript) specification used to interpret double-quoted string literal. `StringLiteral` and `DoubleStringCharacters` are non-terminals in the language and `"` is a control character deliminating `DoubleStringCharacters` token from double-quote token:

```
StringLiteral ::  "DoubleStringCharacters"
DoubleStringCharacter ::  SourceCharacter but not one of " or \
```

A web application developer writes code that generates dynamic HTML and JavaScript content with implicit assumptions about the parsing of it as a `DoubleStringCharacter` token. However, if user-controlled input is used to generate a `DoubleStringCharacter` token without sanitization, it can violate the developer's intent by including a double-quote character. This violation constitutes a potential content injection exploit. I use this developer unintended context switching to detect a content injection exploit. I extend this concept to other languages that are parsing dynamic content of web applications.

The focus on context switching as the root cause of content injection exploits allows us to provide a comprehensive and flexible detection technique. In CONTEXT-AUDITOR I model context switching for HTML, JavaScript, CSS, and Bash languages,

and this model can be used as a stand-alone detection module with fine-grained content injection detection coverage.

### *3.3.3* Context-Auditor *Overview*

At a high-level, Context-Auditor operates as a stand-alone detection module as depicted in Figure 3.1. The high-level operational model of Context-Auditor is as follows:

**Input.** Context-Auditor analyzes the content generated by web applications to determine the impact of attacker-controlled input on parsing. The input to a web application takes the form of an HTTP request, and generated content includes HTTP responses or dispatched shell commands. As its input, Context-Auditor consumes the application's generated content and the location of untrusted (attacker-influenced) bytes in that content. If not given knowledge of untrusted bytes, we can infer the location of untrusted data in the generated content by identifying where content from the HTTP request is used verbatim inside the generated content [1] .

**Parsing.** I designed automata to model context switches in the form of state transitions. These automata are made of states called *parsing states*: A parsing state includes syntactical and lexical information about the current character, which can determine the type of the token or the statement being parsed. These automata are also aware of the exact location in the nested structure of the language it is parsing. The parser is used to process the application output, and its design is detailed in Section 3.4.

[1] I discuss the implications of this choice on Context-Auditor's efficacy in Sections 3.3.4 and 3.7.

**Figure 3.1:** CONTEXT-AUDITOR is a detection black-box: given a specific content and byte offsets of untrusted input, it will return if the content is safe or is a content injection exploit.

**Detection.** The CONTEXT-AUDITOR automata parse the output content (HTTP response or shell command) using its parsing model from the *first character of the output* until the last character of untrusted (tainted) data. If a context switch occurs when parsing an untrusted data character in the application output, then CONTEXT-AUDITOR has detected an exploit. If no context switches occur as a result of untrusted data characters, then the untrusted input is benign.

**Mitigation.** When CONTEXT-AUDITOR detects an exploit, the user can choose to block the output from reaching its destination (i.e., does not deliver the HTTP response or does not allow the shell command to execute). This step can differ based on the user's deployment decision, and I discuss it in Section 3.5.

Figure 3.2 demonstrates the core idea of CONTEXT-AUDITOR, including simplified parsing automata of HTML, CSS, and JavaScript, along with the transitions between them. The bottom of Figure 3.2 has sample web application output (content input to CONTEXT-AUDITOR), and the arrows $\Uparrow$ indicate context switching of the parser on the index of the byte of the input (the transitions on the edges of the parsing automata are labeled with the same indices). Figure 3.2 also demonstrates two different scenarios. In the first scenario, the untrusted user input is `admin` (purple underline of the input), and the untrusted user input is identified based on the purple HTTP request. This input remains in the `Quoted Literal` state and does not cause any context switching, therefore it is benign. In the second scenario, the untrusted user

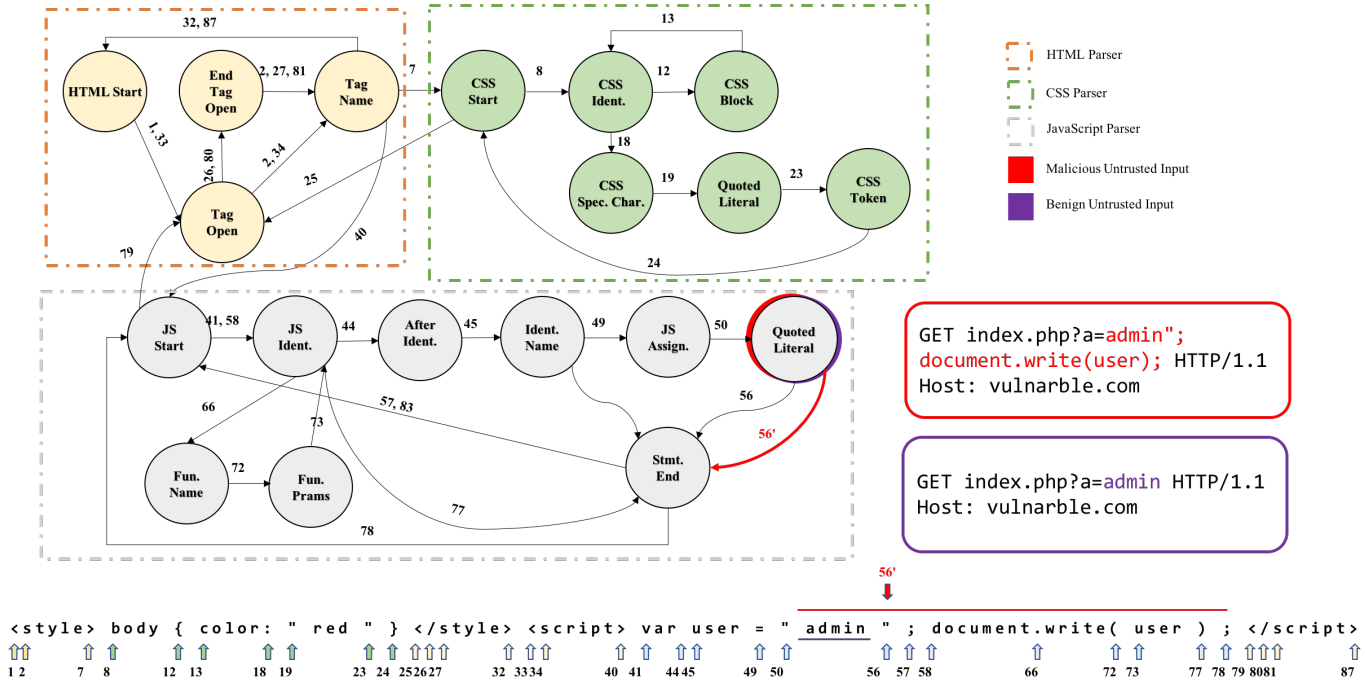


**Figure 3.2:** A simplified graph of context switching in a browser parser while parsing HTML input. If the untrusted input is `admin`, it does not trigger any context switch (parsing state is `Quoted Literal` for all characters of the input) and is therefore benign. However, if the untrusted input is `admin"; document.write(user);`, this triggers a context switch at edge 56′ (from `Quoted Literal` state to `Stmn. End` state) and is therefore malicious.

input is `admin"; document.write(user);` (red overline of the input), and the untrusted user input is identified based on the red HTTP request. This input triggers a context switch at byte 56 (edge $56'$) to `Stmt. End`, and because this context switch occurs while parsing untrusted user input CONTEXT-AUDITOR will detect this as malicious. As demonstrated, if untrusted user input does not cause a context switch (as is the case of the first scenario with `admin` as the untrusted user input in Figure 3.2), the untrusted user input is benign. A similar situation exists when parsing shell commands: if untrusted user input is used as an argument, then it will not cause a context switch. In this way, CONTEXT-AUDITOR uses context switching to detect content injection exploits.

### *3.3.4 Vulnerability Model*

Identifying where untrusted input occurs inside web application's dynamic content (e.g., HTML or shell commands), without knowledge of the server-side code, is a fundamentally difficult problem [107], and orthogonal to the detection of vulnerabilities by CONTEXT-AUDITOR (user-input offset is an input to CONTEXT-AUDITOR, as shown in Figure 3.1). Therefore, I outsource this orthogonal task to a module called `User-Input Detector`. Deployment of CONTEXT-AUDITOR inside an operational environment requires a `User-Input Detector` module in place, which in its simplest form performs string matching. To demonstrate the generality of CONTEXT-AUDITOR, I also deployed a heuristic from Buyukkayhan et al.'s work [107] as the `User-Input Detector` module in Section 3.6 (experiment E4): this heuristic identifies sensitive keywords of a payload and their counterpart inside an HTML response and then attempts to extend the matching offsets.

As an example of deploying an instance of CONTEXT-AUDITOR as a detection module, I developed a prototype in which untrusted input (coming from the HTTP

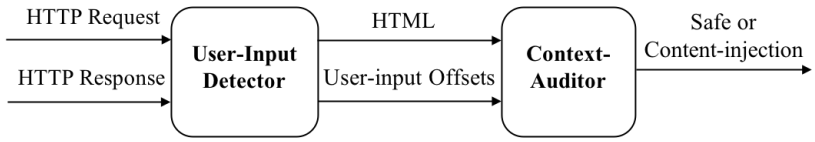


**Figure 3.3:** User-Input Detector identifies the location of untrusted input inside HTTP response and propagates inputs to CONTEXT-AUDITOR for further analysis.

request) used verbatim in generated HTML content are identified and propagated to CONTEXT-AUDITOR for further analysis (Figure 3.3). In another deployment, `User-Input Detector` is a wrapper around `/bin/sh` and invokes CONTEXT-AUDITOR in cases that are susceptible to command injection exploits. These deployments have several implications on the types of vulnerabilities that can be protected against by our prototype. The prototype can prevent exploits against *non-stored* content injection exploits: including reflected XSS, reflected scriptless exploits, and the vast majority of command injection exploits. If injected content is stored (for example, in a database) and later retrieved, our link of untrusted user input is lost. Without some additional tracking of this input (such as taint tracking and propagation), the prototype is not able to determine whether or not parsing context switches are caused by untrusted input.

The focus on reflected content injection allows CONTEXT-AUDITOR to operate without any knowledge about or modification to the underlying web application or web browser, and only concerns itself with the HTTP request and the content generated as a result of it. This allows CONTEXT-AUDITOR to be deployed in many configurations. For instance, similar to trusted-types [33] that uses DOMPurify to sanitize inputs to sensitive page's sinks (e.g. `document.write` and `innerHTML_setter`), we could integrate CONTEXT-AUDITOR in a browser to detect and limit unwanted context switching in sensitive sinks. Theoretically, it could also be deployed inside

frameworks that detect stored XSS because these frameworks can track untrusted user input.

### *3.3.5 Threat Model*

I designed my solution to protect against a sophisticated attacker. I assume that the attacker is aware of a content injection vulnerability in a target web application. The attacker can modify the content of any request parameter and send the request to the target web application to trigger the content injection vulnerability.

### *3.3.6 Comparison with Current Mitigations*

Table 3.1 demonstrates the shortcomings of prior web-based mitigations against content injection exploits: XSS-Auditor only protects against conventional XSS vectors and does not support any exploit from HTML, CSS, or JavaScript contexts [44]. NoScript supports these three languages and prevents scriptless exploits via CSP rules; however its regular expressions might be ineffective in the face of new exploitation techniques or complex nested JavaScript exploits. DOMPurify is mainly focused on sanitization of HTML context and as mentioned in Section 3.2.2 it could be conservative in input sanitization. CSP is mostly concerned with inclusion of files or tags from external domains, and it cannot protect against injection of JavaScript and CSS code into existing JavaScript code. ModSecurity is limited to its directives, and it can be insufficient to prevent simple injections inside JavaScript code, scriptless exploits, and many HTML injections. However, Context-Auditor provides a fine-grained content injection exploit detection solution for scripting, scriptless, and command injection exploits (in HTML, JavaScript, and CSS contexts).

For command injection exploits, I referred to code and data separation solutions (e.g., SMask [99]) or taint-enhanced prevention policies [108]. They provide a content

**Table 3.1:** Comparison of XSS mitigation techniques based on their detection capabilities and their applicability to mobile browsers. Missing detection capabilities are marked by numbers. 1: Depends on policy. 2: Pattern-matching-based. 3: Cannot detect complex nested JavaScript code. 4: It deploys CSP rules to prevent scriptless exploits. 5: Limited tags. 6: cannot detect content injected to CSS. In case of mobile support, we focused on whether the mitigation techniques introduced/considered any measures for content injection exploits that are sent from a mobile browser or not.

| Tool | Context-based Content Injection Exploit Mitigation | | | | Command Injection | Mobile Support | Location |
|---|---|---|---|---|---|---|---|
| | XSS | JS | HTML | CSS | | | |
| **XSS-Auditor** | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | client |
| **NoScript** | ✓ | Partial [2,3] | Partial [2,4] | Partial [2,4] | ✗ | ✓ | client |
| **DOMPurify** | ✓ | ✗ | ✓ | ✗ | ✗ | ✓ | client |
| **CSP** | Partial [1] | ✗ | Partial [5] | Partial [6] | ✗ | ✓ | client |
| **ModSecurity** | ✓ | Partial [2] | Partial [2] | ✗ | ✗ | ✗ | proxy |
| **SMask** | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | server |
| **Taint-enhanced policies** | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | server |
| **Context-Auditor** | ✓ | ✓ | ✓ | ✓ | ✓ | applicable | flexible |

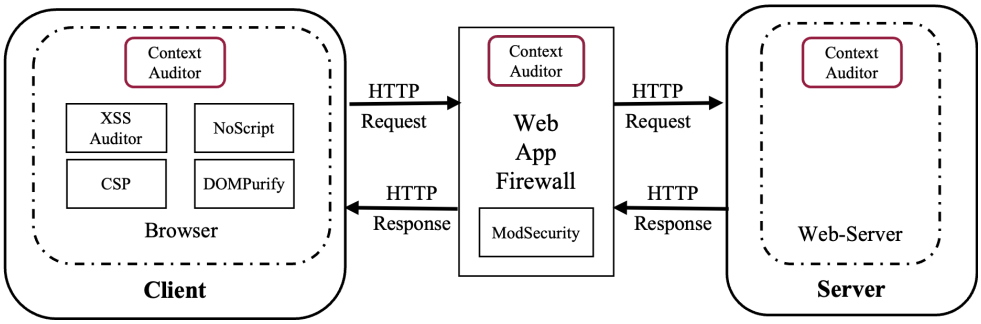


**Figure 3.4:** Possible deployment locations of all aforementioned XSS mitigations (XSS-Auditor, NoScript, CSP, ModSecurity, and WAF) and Context-Auditor in an HTTP client and server communication model.

injection measure against trivial XSS and command injection, and both required some knowledge of server-side deployment. As discussed in Section 3.3.4, despite server-side (SMask and taint-enhanced policies), client-side (XSS-Auditor, NoScript, CSP, and DOMPurify), and WAF (ModSecurity) mitigation techniques that are tied to a specific location, Context-Auditor has a flexible deployment. is also advantageous for mobile browsers because they lag behind in implementation of similar security measures proposed for desktop browsers [109].

## 3.4 Building the Model

The parsing engine, in the form of an automaton, is the core of Context-Auditor. This automaton uses its state transitions to identify context switches, which is the key feature used to detect content injection exploits. I construct this automaton by manually analyzing HTML, JavaScript, CSS, and Bash languages.

### *3.4.1 Modeling Web Languages Parsing*

To detect content injection exploits inside HTML content, we require an automaton to track parsing states of all characters in that content. If we design an automaton with precise parsing states encompassing syntactical and semantic information about

the underlying token or character in each location, we would be able to detect a broad spectrum of content injection exploits that could be inserted into any location inside HTML content. Such automaton will consequently help us in the detection of exploits with various granularities: we can detect coarse-grained exploits that cause a language transition inside HTML content, and we can also detect fine-grained (and short) exploits that only insert additional functionality into existing code. Furthermore, unlike prior approaches, this technique requires no prior knowledge of exploits.

To design an automaton with the state transition requirements which supports three major web languages (HTML5, JavaScript, and CSS) and the Bash language, I studied specifications of these languages: specifically, I analyzed the lexical analysis stage from the HTML 5 specification [110] (which is referred to as the *tokenization stage*), the grammar rules from the ECMAScript specification ECMA-262 [111], a tokenization procedure introduced by W3C in CSS language specification [112], and the Bash language manual [113]. Considering the specifications, I realized that our candidate automaton must meet the following requirements:

**(1)** The next state of the automaton should be based on the current state and input character (or lexical token).

**(2)** The automaton should be able to track history to support nested structures, branches, arrays, objects, tags, etc.

**(3)** The automaton should support revisiting. According to HTML 5 parsing specification, the parser needs to re-consume a character under certain conditions [110]. Revisiting is also necessary for parsing CSS and JavaScript, as for each of them, certain characters in individual states might be the indication of either a new token or a new statement. In this case, the character must be re-analyzed in another state later which requires the revisiting property.

Based on these constraints, I design the automaton in Context-Auditor as

a two-way finite pushdown automaton (2PDA): A pushdown automaton (PDA) is a finite-state automaton (FSA) with a stack, and a 2PDA complements PDA by supporting revisiting input characters that are already consumed. I designed the automaton in a similar fashion to the way that browsers parse an HTML document: the HTML parser is the primary parser shipped with any modern browser: It takes as input bytes representing an HTML document and starts parsing these bytes character by character. In the course of parsing, the HTML parser distinguishes among different tokens and special characters and even different languages in that document. For instance, while parsing a specific byte index, it determines whether the index is inside an opening tag, an attribute name/value or either inside a closing tag. The existence of special tags marking the beginning of style sheets or embedded scripts (e.g. `<script>` or `<style>` tags) is an indication of a new language for the HTML parser; it will invoke the corresponding parser (which is also included by the browser) to parse the embedded content between the opening and closing tags.

The automaton starts parsing the HTML document from the first input character and then it moves forward character by character, while moving along the sequence, it has detailed information about the underlying language, token, and the specific language statement being parsed. I refer to such information as *context* and I use *parsing states* to reflect this concept inside an automaton. I manually constructed the HTML 5 parsing automaton following tokenization stage in the HTML 5 specification [110]. It tracks the HTML tokens and tags for each character in the HTML sequence via parsing states' information. It also supports transition into states that are related to CSS and JavaScript language contexts by reading a `<script>` or `<style>` tag. I also manually constructed parsing states related to CSS and JavaScript languages by referring to the related specification and grammar rules respectively (W3C tokenization specification for CSS [112] and ECMA-262 [111]). Figure B.2 shows a

simplified representation of the 2PDA.

### *3.4.2 Modeling Shell Command Parsing*

Command injection exploits in web applications are the result of dynamic generation of shell commands using untrusted input. Similar to content injection detection for web languages, command injection exploits are also detectable via identifying context switching. To model shell command parsing, I focused on identifying different tokens in a shell command. I determine whether the current character is part of a command or operand, or if it is a special character which would change the type of a statement. The parser also considers the hierarchical structure of a command to keep track of quotes, back-ticks, parentheses, braces, etc. Figure B.1 shows a simplified 2PDA of the shell parsing automaton.

## 3.5 Implementation

As discussed in Section 3.3.2, our context switching-based content injection detection method provides us (among other benefits) with the advantage of flexible deployment. Despite dedication of other state-of-the-art content injection mitigation techniques to a specific location in the HTTP client–server communication model, as shown in Figure 3.4, Context-Auditor can be deployed anywhere along this model. To showcase the flexibility, I deployed Context-Auditor in four different operational models:

**Context-Auditor Shell Wrapper:** Figure 3.5 shows how Context-Auditor works on server side as a module to detect command injection exploits. This module is a wrapper around `/bin/sh` that implements the functionality of `User-Input Detector`: It generates byte offsets marking reflections of HTTP request parameters in a series of commands sent to `/bin/sh` and invokes Context-Auditor's shell parser module

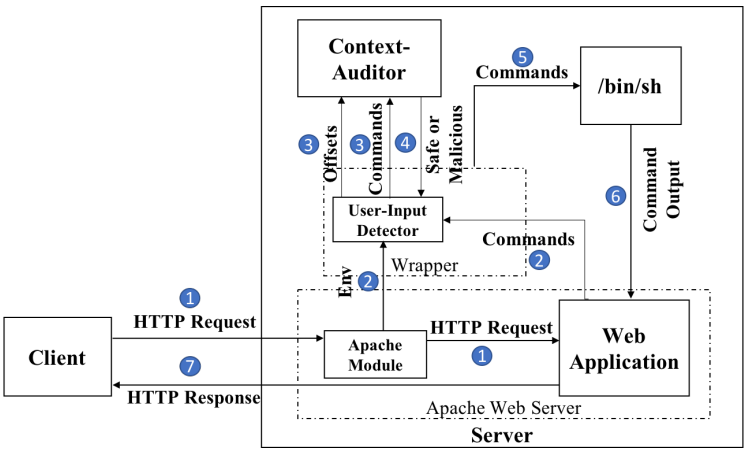


**Figure 3.5:** The communication diagram of a CONTEXT-AUDITOR-equipped HTTP client-server model to prevent command injection exploits.

with the input commands and byte offsets.

**Context-Auditor nginx Plugin:** Figure 3.6 illustrates how CONTEXT-AUDITOR works in an environment with nginx as the HTTP front-end, Apache httpd (or other web applications hosted by Apache httpd) as the service back-end, and CONTEXT-AUDITOR as a stand-alone detection module. The nginx plugin acts as an HTTP proxy and detects untrusted input by identifying any input data from the HTTP request in the HTML response. Then, nginx forwards the intercepted data to CONTEXT-AUDITOR, and, after receiving the detection results from CONTEXT-AUDITOR, the nginx plugin filters and blocks all responses that are content injection exploits.

**Context-Auditor Web Proxy:** In this case, I integrated CONTEXT-AUDITOR with *mitmproxy* [114]. This instance identifies byte offsets of user-controlled data in HTTP responses and invokes CONTEXT-AUDITOR for parsing analysis of the response. If a potential content injection exploit is detected, then CONTEXT-AUDITOR blocks the response.

**Context-Auditor Chrome Extension:** I modified an existing Chrome extension called *Tamper* [115], which allows us to intercept Chrome's HTTP requests and re-

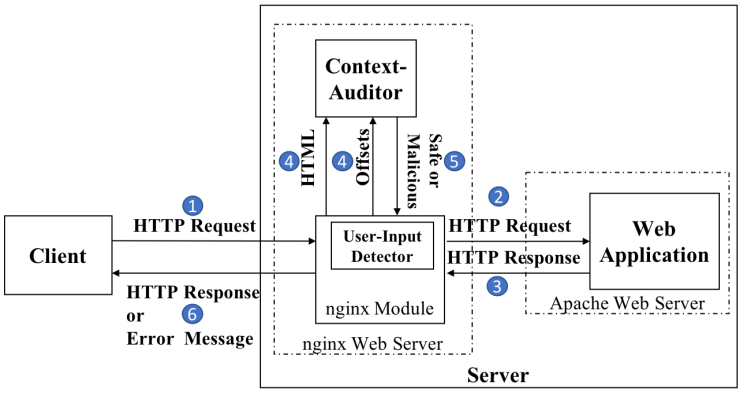


**Figure 3.6:** The communication diagram of a CONTEXT-AUDITOR-equipped HTTP client-server model to prevent injections inside a HTTP response.

sponses. From the request and response, I identify untrusted input and pass this to CONTEXT-AUDITOR to detect content injection exploits. Due to technical limitations of the Tamper extension, blocking responses was not possible and I logged these cases instead.

## 3.6 Evaluation

In evaluating CONTEXT-AUDITOR, I sought to answer the following research questions:

**Q1. Effectiveness.** How many content injection exploits in HTML, CSS, JavaScript, and shell scripts can CONTEXT-AUDITOR detect, and how many exploits does CONTEXT-AUDITOR miss?

**Q2. Practicality.** Does CONTEXT-AUDITOR exhibit a low false positive rate so that it can be deployed in real-world settings without raising excessive false alarms?

**Q3. Efficiency.** Does CONTEXT-AUDITOR exhibit a low runtime overhead in all evaluated scenarios to justify the deployment of it in real-world settings?

### 3.6.1 Experiment Design

I designed seven experiments under different scenarios with different data sets. Two experiments (E1 & E2) focus on the true positive rates of Context-Auditor in detecting XSS and scriptless exploits. To stress the generality of the proposed technique, I also designed an experiment (E3) demonstrating the effectiveness of Context-Auditor in detection of command injection exploits. E4 showcases the prevalence of the context switching phenomenon in a public data set of XSS exploits. Another three experiments (E5, E6, and E7) focus on showing the practicality (false positive rates) and efficiency (runtime overhead) of Context-Auditor in real-world settings. Table 3.2 shows an overview of these experiments.

### 3.6.2 Data Sets

I used both well-known public data sets and hand-crafted data sets in our experiments. Public data sets include the PortSwigger Cross-Site Scripting cheat-sheet (E1) [116], Buyukkayhan et al.'s [107] data set of reflected server-side XSS exploits (E4), and the OWASP XSS cheat-sheet (E7) [117]. These are all ground-truth content injection exploits that should be detected and blocked by a perfect XSS defense. I ran Context-Auditor on these data sets to evaluate its effectiveness. Additionally, as discussed in Section 3.2.2 and Section 3.3.6, I provide hand-crafted XSS exploits that bypass several state-of-the-art XSS defenses but can be detected by Context-Auditor, which I will not reiterate in this section. Because of the lack of publicly available data sets, I also built our own data sets of command injection exploits (E3) and web pages that are free of content injection exploits based on known CVEs and a crawl of top Alexa websites (E5), respectively.

**Table 3.2:** Details of each experiment in the evaluation. "Standalone" means CONTEXT-AUDITOR is used as a standalone Python library. The other four types of deployment are described in Section 3.5.

| ID | Description | Category | Deployment | Data set | Related Question |
|---|---|---|---|---|---|
| E1 | XSS and scriptless exploits detection | web | web proxy | PortSwigger XSS cheat-sheet [116] | Q1 |
| E2 | XSS detection, comparing against w3af | web | nginx plugin | Firing Range [118] | Q1 |
| E3 | Command injection exploits detection | shell | shell wrapper | web applications with known command injection CVEs | Q1 |
| E4 | Context switching probability in a public data set | web | standalone | [107] data set including exploits from xxsed [119] & obb [120] | Q1 |
| E5 | False alarms and runtime overhead | web | web proxy | Alexa Top 1,000 websites | Q2 and Q3 |
| E6 | False alarms | web | web proxy | WordPress and human-generated traffic | Q2 |
| E7 | Runtime overhead | web | nginx plugin<br>web proxy<br>Chrome extension | hand-crafted data set | Q3 |

### 3.6.3 E1: Detecting Web-Based Content Injection Exploits

To evaluate the effectiveness of CONTEXT-AUDITOR against real-world web-based content injection exploits, I ran CONTEXT-AUDITOR prototype against public data sets of XSS and scriptless exploits from PortSwigger XSS cheat-sheet [116]. Portswigger website provides URLs via link to vulnerable web applications for every exploit; I used both vulnerable web applications and exploits from the website in the experiment. I selected reflected XSS (including event handlers, consuming tags, file upload, restricted characters, frameworks, protocols, special tags, other useful attributes, encoding, obfuscation, and WAF bypass categories) and scriptless exploits specified to work on the Google Chrome browser. The exploits also cover both HTML and JavaScript contexts. Then, I use Chrome to access each URL, proxy the requests through the CONTEXT-AUDITOR Web Proxy (as described in Section 3.5), and report the number of detected exploits. Reflected exploits constitute the majority of exploits in the Portswigger dataset; therefore, I configured the experimentation environment via a User-Input Detection module that identifies request parameters used verbatim inside their consequent HTML response. CONTEXT-AUDITOR demonstrated 100% detection rate: It successfully detected all 242 XSS exploits and all 25 scriptless exploits.

### 3.6.4 E2: Detecting XSS Exploits Generated by w3af

In this experiment, I deployed CONTEXT-AUDITOR as an nginx plugin (Figure 3.6) and tested if it can prevent XSS exploits coming from attackers. I first hosted under nginx 32 web applications provided by Firing Range [2] [118] that have reflected XSS vulnerabilities. The content injection vulnerabilities in Firing Range may occur in all

[2]Firing Range is a web application test suite that contains a wide range of intentional vulnerabilities. I used version 0.48 in this experiment.

three contexts, which are HTML, CSS, and JavaScript. I then ran the open-source w3af web vulnerability scanner [121] against these web applications, and w3af successfully detected XSS vulnerabilities and generated XSS exploits on the 32 web pages. Then, I enabled the CONTEXT-AUDITOR nginx Plugin (as described in Section 3.5), and re-ran w3af against these applications. CONTEXT-AUDITOR correctly detected and blocked all requests with exploit payload generated by w3af, without blocking any benign requests. As a result, w3af did not report any XSS vulnerabilities. This shows that CONTEXT-AUDITOR can detect and block realistic XSS exploits.

### *3.6.5 E3: Detecting Command Injection Exploits*

Due to the lack of existing data set of web applications with command injection vulnerabilities, I manually scanned CVEs to build a data set that comprises PHP web applications with known command injection vulnerabilities. As dictated by the threat model (Section 3.3), CONTEXT-AUDITOR only supports the situation where the exploit payload is sent as a URL parameter and used verbatim in a shell command. From these, we sampled three vulnerable web applications. I additionally checked a vulnerability report published by RIPS [122] vulnerability scanner's website and added another vulnerable application to this list. With these four web applications, I verified the effectiveness of the CONTEXT-AUDITOR Shell Wrapper (as described in Section 3.5) to detect command injection exploits when it is deployed in a scenario similar to Figure 3.5. CONTEXT-AUDITOR successfully detected content injection exploits for all of these applications. Table 3.3 shows the list of these vulnerable web applications and the detection capability of CONTEXT-AUDITOR.

The command injection vulnerability in the `PHP File Manager` resides inside a backdoor that allows an attacker to execute arbitrary OS commands, which is an intended malice. Because malicious intention in a web application is outside the threat

**Table 3.3:** CONTEXT-AUDITOR's command injection detection capability. 1: Denotes multi-token commands.

| CVE | **Application** | **Version** | **Detection** |
|---|---|---|---|
| CVE-2015-5958 | PHP File Manager | 0.9.8 | Partial [1] |
| CVE-2010-4278 | Pandora FMS | 3.1 | ✓ |
| CVE-2008-6669 | nweb2fax | 0.2.7 | ✓ |
| 0-day by RIPS [122] | Oscommerce | 2.3.4 | ✓ |

model of CONTEXT-AUDITOR, whenever an attacker executes a single token command (such as *whomai* or *ls*), CONTEXT-AUDITOR does not detect it. However, CONTEXT-AUDITOR *can* block any command with more than one token sent to the web application.

### *3.6.6 E4: Measuring the Likelihood of Context Switching in Exploits of a Public Data Set*

Context switching concept is the core idea of CONTEXT-AUDITOR, to realize the importance of it, I analyzed its prevalence on a public data set of XSS exploits. I used a data set created by Buyukkayhan et al. [107]: They performed a longitudinal study on reflected server-side XSS exploits from XSSED [119] and OPENBUG-BOUNTY [120] data sets and consolidated those in their data set. This data set has an `attack` table including data from actual exploits; such data involves `payloads` and exploited HTML `responses`. However, each payload could have several reflections in its HTML response; all of those might not lead to actual exploits. They implemented a greedy heuristic to identify these candidate reflections (user-input detector module) and then used a series of methods to extract a single working exploit from each re-

sponse. Their heuristics was a proper user-input detector module, therefore I reached out to the authors and gained access to the heuristic's code. Afterwards, I ran an analysis on Buyukkayhan et al.'s [107] data set: For each payload and HTML response in the data set, I ran them through the heuristic and received a list of candidate exploit offsets. Context switching idea claims that at least one of these candidates triggers a context switching; therefore, I aimed to verify this claim in this experiment. In this regard, I analyzed the parsing of 170,667 entries of the data set. Context-Auditor successfully flagged 148,778 (87.17%) of those payloads that trigger a state transition in an HTML response; but, it did not report any state transition for 21,889 of the entries (12.82%) in the data set. I manually investigated some of these payloads to understand their nature. The inability of Context-Auditor in handling such exploits is for the following reasons:

(1) Context-Auditor is a research prototype to demonstrate the practicality of the context switching concept in detecting content injection exploits. Since I have manually designed and implemented this parsing automata, it is prone to parsing errors and incompleteness. I devoted significant engineering effort to improve its coverage and detection capability, however, there is still room for the implementation of methods to cover more corner cases and inputs via syntactical errors. (2) Some payloads do not trigger a context switching in any of the offsets. Such cases might occur due to the reflection of an exploit inside an HTML attribute value such as `onclick`, or it might lead to a second-order XSS exploitation. The current implementation of Context-Auditor could detect exploits that force the HTML parser to transfer (from `attribute name` state) into `attribute value` state; but it does not support interpretation of JavaScript context inside certain `attribute value` states (e.g. `onclick`'s value). However, this is not an inherent limitation of this approach, and it is a matter of engineering effort. In the case of second-order XSS exploits, if the exploit reflects

inside the assignment part of a JavaScript `Quoted Literal` for instance, it does not trigger a context switching at the time of parsing. But, it might lead to exploitation at runtime, which Context-Auditor fails to detect. Such exploitations are not in the scope of this experimentation setup, we could theoretically detect a few of those by deploying Context-Auditor at sensitive JavaScript sinks similar to Trusted Types [33] and defining injection detection policies (similar to [123]).

### *3.6.7 E5: Performance Overhead and False Positive on Top Alexa Websites*

To evaluate the performance overhead and false positive rate of Context-Auditor in real-world settings, I performed a crawl of the top 1,000 Alexa websites [124]. For each URL, two random links were followed (links with at least one parameter) and the average loading time (over two runs) for the random links was measured. Since Context-Auditor is not invoked for the main URLs (due to lack of URL parameters), I only considered the loading time overhead of random links and excluded the overhead of main links. Additionally, I monitored any potential false alarm triggered by Context-Auditor in this experiment. Similar to E1, I used Chrome to access each URL and proxy the requests through mitmproxy (as described in Section 3.5). Since the experiment involved benign traffic, I used a User-Input Detector module that recognizes request parameters used verbatim inside their consequent HTML response, and Context-Auditor is invoked if the length of those reflections is at least three characters. In case there are multiple reflections of the URL parameters inside the HTML response, I ran Context-Auditor for up to five reflections of each URL parameter value.

Since mitmproxy imposes significant overhead itself, I first crawled the random links in an experimentation setup with mitmproxy (Context-Auditor disabled) deployed, then I revisited the links in an environment with both mitmproxy and

Context-Auditor enabled. I filter out any request timed out (with a timeout of 20ms per request). Also, I did not involve links with negative loading time overhead in the loading time measurements. This measurements demonstrate an average of 4.7 seconds in loading time overhead while visiting random links that involve an average of three URL parameters. Impressively, Context-Auditor only flagged one of the visited random links as malicious. The loading time is caused to the following reasons: (1) In this experiment Context-Auditor analyzes up to five reflections for each request parameter value. For each, it parses the HTML response from the first character, which causes excessive delays. (2) The parsing automata cannot correctly handle some syntax errors or parsing of JavaScript statements not being deliminated by semicolons. I implemented a monitoring algorithm (timing-based) that detects such cases; it then adjusts the JavaScript code (by semicolon insertion) or forces the parser into particular states, all of which impose delays.

#### *3.6.8 E6: Measuring False Positives on a Blog Site*

Context-Auditor may theoretically produce false positives when user input causes context switches in the dynamic content of a web application, and this action is part of the expected functionality of the application. An example is a blogging website where users may embed JavaScript code into their blog posts. To understand the severeness of this scenario in a real-world setting, I performed a study on the WordPress platform, a widely used blogging web application, with a set of 10 human testers. I first deployed a WordPress instance on a server and put it behind a Context-Auditor web proxy. Then, given a unique user account and the URL to the WordPress instance, I allotted each tester 10 minutes and asked them to freely use WordPress and report any unresponsiveness of the application to me. Testers could create, add, delete, or modify any blog posts and input any data, including

**Table 3.4:** False negative and true positive rate of Context-Auditor with benign and malicious sets, along with performance analysis in terms of Latency (loading time in milliseconds) for the two lists, in four cases: without Context-Auditor or with any of three instantiations of Context-Auditor (nginx module, web proxy and Chrome extension).

| Case | Loading time without CA | CA loading time for nginx | CA loading time for proxy | CA loading time for extension |
|---|---|---|---|---|
| Benign Requests | 16ms | 20ms | 17ms | 35ms |
| Malicious Requests | 15ms | 16ms | 18ms | 28ms |

JavaScript code or CSS.

Throughout the experiment, testers did not report any case of unresponsiveness. Finally, a total of 1,680 HTTP request and response pairs were collected and analyzed by Context-Auditor. Context-Auditor did not report any content injection exploit, which means it did not raise any false alarms. I believe this is because normal WordPress users rarely need to insert custom JavaScript or CSS code into their blog posts despite the support in WordPress. Advanced WordPress users who need to insert JavaScript or CSS code into their blog posts may choose to ignore Context-Auditor warnings temporarily. Therefore, I argue that Context-Auditor can be applied in real world without causing excessive false alarms. In the unlikely case that a tester might have attempted to exploit WordPress, I further verified that these HTTP requests and responses do not contain any content injection exploits. So Context-Auditor also did not cause any false negatives in this experiment.

### *3.6.9 E7: Measuring Runtime Overhead*

To evaluate the performance overhead of Context-Auditor, I used the Selenium web driver [125] to request webpages. I extracted a list of content injection

exploits from the OWASP XSS cheat-sheet [117] categorizing them based on context. Then, I chose a proper vulnerable web page from the Firing Range application [118], for each exploit category and crafted corresponding links as the input to Selenium driver. I measured the time of fetching a webpage (1) without Context-Auditor, (2) with Context-Auditor as an nginx plugin, (3) with Context-Auditor as a proxy, and (4) with Context-Auditor as a browser extension. For the nginx plugin and the proxy, Context-Auditor returns a `404` response code when it detects a malicious request. This means that, depending on the deployment scenario, the latency to load the HTTP request may in fact, be faster for malicious requests. In case of the browser extension, the extension does not filter malicious responses, instead it only logs requests/responses including a content injection exploit.

Table 3.4 has the performance measurements of this evaluation. In the best case for benign requests, the Context-Auditor web proxy module, added 1ms (6%) of overhead. The worst case, the Context-Auditor extension, added 19ms (119%) of overhead. The best case of malicious requests, the Context-Auditor nginx module, added 1ms (7%) of overhead, while the extension added 13ms (87%) of overhead in the worst case. As all implementations of Context-Auditor are proof-of-concept research prototypes (Context-Auditor is implemented in Python), I did not focus on optimizing latency.

## 3.7 Limitations

As I showed in the evaluation section, Context-Auditor is a new approach to defend against content injection exploits. However, it does have a number of limitations, which I will discuss in this section.

**False positives.** There was one false positive in our experiments, however some circumstances could cause more: For instance, if a blogging application (that allows

blogging in raw HTML) reflects the user's newly created post (which is sent in as an HTTP parameter) in its response, CONTEXT-AUDITOR might report an exploit despite this behavior being legitimate and intended by the website's developers. This is not as common: blogging platforms typically send an HTTP redirect to the HTTP post request rather than reflecting the update itself, in which case there would be no false positive as the request will not be contained in the response. However, even if the platform does not use a redirect (and actually reflects the blog post), and CONTEXT-AUDITOR blocks the response, it will only block the response that includes the reflection: in this case, the submitted request will still be evaluated by the web application and the behavior would be correct (i.e., the blog post would appear).

**Second-order content injections.** I define second-order exploits as content injection exploits that do not trigger any parsing state transitions, yet still execute JavaScript. DOM-based XSS vulnerabilities are an example of second-order exploits: The content of an untrusted JavaScript string is interpreted—*during JavaScript execution*—as JavaScript code. These types of vulnerabilities are also called client-side XSS vulnerabilities, because the root cause of the vulnerability exists in the JavaScript code of the web page. In other words, there is no way to change the server-side code to fix the vulnerability (e.g., when untrusted input is used as an argument of `eval` function). Listing 1 contains a client-side XSS vulnerability on Line 18. The malicious input for the `id` parameter of the (JavaScript context) string literal `<script>alert(1)</script>`, will *not* cause a context switch. However, at runtime the browser's JavaScript execution engine will send the `str` variable (which is now untrusted data) to the `document.write` function, where it will interpret this string as HTML (thus causing a second-order content injection, where this parsing of the untrusted data by `document.write` will cause a context switch in the HTML parser). Unfortunately, CONTEXT-AUDITOR in its current deployment cannot detect such

exploits. Mitigating second-order (and, perhaps $n$-order) content injection exploits would require modifying the browser to track the untrusted data (e.g., via taint propagation), similar to the approach proposed by Stock et al. [44].

Steffens et al. [24] have released a taint-aware Chromium engine and a minimal extension, and we can integrate CONTEXT-AUDITOR with those to analyze second-order content injection exploits. Steffens et al. [24]'s tool defines a function called `DOMXSSFinderReport`, which is invoked along with each detection of a tainted flow into a sensitive client-side sink. We can further call CONTEXT-AUDITOR inside this function to perform parsing analysis on suspicious offsets of the inputs and raise flags if necessary.

**Transformations of user input.** Server-side code or client-side scripts may sometimes perform transformations on user-controlled input, yet our current deployment of CONTEXT-AUDITOR supports cases where input strings are reflected verbatim in response data. However, using a different `User-Input Detector` module such as the heuristic from Buyukkayhan et al. [107]'s work that allows a few character mismatches between a payload and a candidate offset in the HTML response, CONTEXT-AUDITOR could detect more cases of context-switching. The tracking of input through transformations is a generally complex problem. There *are* approaches, such as dynamic taint tracking, that could be used to track such transformations. However, as mentioned in Section 3.3.4, the task of user input tracking is orthogonal to the core concept of CONTEXT-AUDITOR. Any increase in the ability of untrusted input tracking, regardless of the techniques used, would increase the space of content injection vulnerability instances that CONTEXT-AUDITOR can protect.

**Stored content injection.** As discussed in Section 3.3.4, CONTEXT-AUDITOR only supports non-stored content injections. This is due to CONTEXT-AUDITOR's inability to track untrusted input through data stores used by an application. This is an

analogous issue to the transformation of user input: whereas that problem deals with the tracking of input through transformations, this must tackle the tracking of input through storage. One potential direction to address this limitation is through the use of proxies between the web application and its data stores: a very similar approach to SQL parsing analysis by libinjection [126], except that having a parsing automaton for query languages (an extension of Context-Auditor) we do not need to instrument web applications' source code, and we can delegate the injection detection task to a proxy. This proxy must implement a way of relating stored content and its corresponding HTTP request content (similar to how the `/bin/sh` wrapper is handled for shell injection) so that it could inform Context-Auditor of content containing untrusted input. This represents a significant engineering burden, but would expand Context-Auditor to yet additional content injection vulnerability classes.

**Syntax errors.** Context-Auditor is a research prototype manually built based on languages' grammars and specifications. We faced issues while parsing malformed HTML and JavaScript codes, which is the cause of some parsing failures in E4 (Section 3.6). We investigated numerous parsing errors and failures, resolving many of those. For instance, malformed JavaScript code or developers' negligence to deliminate statements via a semicolon could create syntax errors or indefinite parsing loops. Therefore, we implemented monitoring code to identify such cases and force the parser to `Syntax_Error` or `Automatic_Semicolon_Insertion` states when necessary. Another monitoring code modifies the JavaScript parser's input according to *automatic semicolon insertion* rules of ECMA-262 [111] specification when parser enters `Automatic_Semicolon_Insertion` state.

**Overhead.** As mentioned above, our current implementation of Context-Auditor can not handle some syntactical errors properly; we also made some simplification

assumptions for implementing our automaton, both of which might lead to infinite loops while parsing malformed inputs. Our monitoring code detects infinite loops via a timing constraint of 5 seconds on the last update of the `current_state`'s value of the automaton and forces the automaton into the `parsing_error` state. We first must find an optimum value for the timing constraint's threshold to decrease the automaton's overhead. Then, some simplifications of the automaton cause parsing errors in the face of malformed inputs and impose additional overhead. The simplifications were only for ease of design and implementation (fewer state transitions), and we can substitute those simplifications with more accurate state transitions in the future to reduce parsing errors and overhead.

**Browser quirks.** Attackers may utilize browser-specific parsing quirks when carrying out exploits. These quirks are notoriously difficult to model, and Context-Auditor's parsing model certainly does not handle all of them. However, Context-Auditor will report an exploit as long as the attacker's input triggers either a parsing context transition or a parsing error. In the presence of an attack utilizing a browser parsing quirk, the former case would represent Context-Auditor correctly handling the quirk, and the latter would represent the common case of Context-Auditor *not* handling the quirk. In either case, the attack will be detected. Of course, some browser quirks that do not trigger parsing errors almost certainly exist. Context-Auditor would not be able to detect the exploits, and this is a limitation of our approach [3] .

**XS-Leak.** Deployment of Context-Auditor in blocking mode might raise the XS-leak issue: In some instances, an attacker might infer the state of a victim at a target site by sending requests via specially-crafted inputs [128]. Here, Context-Auditor

[3]In fact, this is a fundamental limitation that man-in-the-middle components cannot always correctly infer all behaviors of client or server side components [127].

blocks the response since it can not distinguish between offsets that originate from an attacker and the ones that are already part of the HTML content. We can limit the XS-leak issue via the integration of taint-tracking approaches and only flagging context switching cases that are triggered by untrusted user input.

## 3.8 Related Work

There are many research projects on detecting XSS exploits, such as sanitization [25; 27], policy enforcement [108], code-data separation [28], moving target defense [129], and other server-side [130; 26; 131; 95] and client-side [41; 36] mitigations. However, none of them tackles the root cause of content injection vulnerabilities. As Context-Auditor is based on context-sensitive parsing, I will only discuss papers with relevant approaches in the rest of this section. Stock et al.'s approach [132] in detection of DOM-based XSS vulnerabilities is the closest to our idea. They focus on the tokenization process with an observation that benign user input should only be tokenized into literal tokens, and any non-literal token coming from user input indicates an exploit. Context-Auditor generalizes their insight into the root cause of all content injection vulnerabilities and build a generic defense against such exploits. Prokhorenko et al. propose the context-centric injection detection model to identify exploits including XSS and SQL injection [133]. Compared to our automaton, their view of context is more coarse-grained. Their model is also tied to server-side PHP code.

ScriptGard focuses on correct placement of sanitizer functions in server-side code considering the context that a sanitizer function is used in [43]. XSS-GUARD [26] uses parsing analysis to determine authorized scripts in an HTTP response that are intended by its developers. It introduces the concept of "Shadow Web Pages" and forces the application to follow the same execution path for both untrusted and benign

input (by building parse trees and equivalency checks). Although their idea seems similar to Context-Auditor, the parsing state in our work exhibits a broader view by looking deeper into the syntactical structure of all supported languages. Moreover, our solution supports a wide range of content injection exploits instead of merely XSS exploits. Mitigation of content injection vulnerabilities is also beneficial to limit compromises caused by recent JavaScript-based vulnerabilities: Such as vulnerabilities in `postMessage` event handlers [134], client-side CSRF [135], prototype pollution [136], XS-leak [137; 128], and browser-based side-channels [138].

Taint-tracking had also been used to help detect and prevent limited content injection exploitations (SQL injection, XSS, etc.) [139; 97]. Positive [140; 43] and negative [141; 108] taint-tracking trace safe data or untrusted inputs into sensitive sinks, respectively, and examine whether they satisfy particular security policies. Positive taint-tracking requires domain knowledge and rewriting of the source code to track sources of safe data throughout the program, it is the opposite of our core idea in Context-Auditor (which relies on the context switching concept and is not limited to the source code or environment settings). Negative taint-tracking, however, could assist Context-Auditor and provide offsets of suspicious inputs to our content injection detection module; Context-Auditor can further analyze those offsets and report any context switching that occurred while parsing the offsets.

## 3.9 Conclusion

For too long, the research community has focused on mitigating XSS exploitation by blocking or detecting JavaScript execution. I believe that a shift in thinking is necessary: by broadening our scope to content injection, in terms of both vulnerabilities and exploits, we can finally address the root cause of content injection exploits: untrusted input that causes context switch in a parser. By modeling the parsing pro-

cess, we can detect these exploits. This chapter demonstrated Context-Auditor, a generalized detection mechanism for content injection exploits. While Context-Auditor is capable of detecting a wide range of injection vectors, this prototype supports injections in HTML, CSS, JavaScript, and shell commands. The evaluation showed that Context-Auditor is effective, performant, and unintrusive.

The idea behind Context-Auditor began with an exploration of JavaScript injection vulnerabilities in web applications. This investigation revealed a broader class of content injection vulnerabilities, motivating the need for a more comprehensive, context-aware mitigation strategy. Context-Auditor offers a significant step forward in addressing these vulnerabilities by focusing on unintended context switches during parsing, which are at the root of many injection-based attacks. While Context-Auditor plays a crucial role in safeguarding web applications and protecting users in the current Web ecosystem, a significant challenge remains in the execution of privacy-invasive JavaScript code in a user's browser. This code can perform browser fingerprinting, posing serious privacy risks to users and potentially leading to compromises in their accounts. In the following chapters, I will explore two additional aspects of safeguarding users' Web interactions. Chapter 4 discusses the effectiveness of Simulacrum extension (introduced in Section 2.4) in protecting against DOM-based extension fingerprinting. Chapter 5 then introduces Browser Polygraph, a tool designed to protect user accounts from fingerprint spoofing and ATO attempts, offering a robust defense against the exploitation of browser fingerprints in fraudulent activities.

# Chapter 4

# EVALUATION RESULTS OF SIMULACRUM

This chapter centers on protecting users within the modern Web ecosystem, particularly against the privacy-invasive threat DOM-based extension fingerprinting. In Section 2.4, the DOM Reality Shifting concept from [20], which provides a solution for DOM-based extension fingerprinting, as well as the design and implementation of the Simulacrum extension, were quoted from [20].

In this chapter, I present my comprehensive evaluation results of Simulacrum (Section 4.1), demonstrating its functionality and effectiveness in preventing against DOM-based extension fingerprinting. Following the evaluation, I will discuss the broader implications of these results, highlight potential limitations, and provide practical guidelines for extension developers to enhance privacy protection without compromising functionality Section 4.2.

## 4.1 Experimental Evaluation

Here I will present a comprehensive evaluation that explores multiple dimensions of Simulacrum's performance.

**Simulacrum's effectiveness.** As our main goal is to mitigate browser extension fingerprinting that leverages DOM-based modifications, a robust and accurate fingerprinting system is needed to evaluate Simulacrum's approach. To that end, the code and dataset of Carnus [16] is leveraged, as it is the only proposed system that *automatically* generates unique signatures that identify extensions based on their DOM interactions. While Carnus has modules for different types of extension techniques (e.g., detecting Web Accessible Resources) the focus is on the DOM-based module.

To account for the dynamic nature of extension behavior and to improve robustness, Carnus requires at least 90% of a fingerprint to match the DOM modifications to infer that an extension is installed. In their experiments, Karami et al. [16] reported 5,793 extensions being detected by Carnus based on their DOM-based fingerprints. To test the effectiveness of Simulacrum's countermeasure, that experiment is replicated using the exact dataset of extensions. Selenium is then used to open a fresh browser installed with Simulacrum. Then, using each of the 5,793 extensions installed, the browser is directed to the website running Carnus' honeypage that will attempt to detect the extension. This experiment findings show that Simulacrum is highly effective as it successfully prevents Carnus from detecting 5,553 (95.86%) of the extensions. In comparison, CloakX [19] could only protect up to 751 (12.96%) of these extensions [16] by randomizing a subset of node attributes.

**Partial fingerprints.** Next the set of extensions that Carnus is unable to detect was investigated, finding that certain extensions manage to inject a *subset* of their fingerprints in the Parallel DOM. This can occur due to various behaviors which is explained over the following paragraphs, and Carnus does not detect them as it is not able to reach the 90% threshold required for identifying the extensions. Nonetheless the robustness of Simulacrum's defense is further assessed by training Carnus on these partial fingerprints to examine whether it is able to identify any of these extensions.

To find extensions that partially inject fingerprints into the Parallel DOM, modification of Carnus was necassry to calculate the percentage of fingerprints that are injected into the Parallel DOM. This allowed identification of 37 extensions with partial fingerprints. This analysis reveals that the partial fingerprints of 28 extensions are unique enough within the entire collection of extensions fingerprints to identify those extensions. Therefore, a total of 268 extensions remain fingerprintable against Simulacrum, which amounts to only 4.63% of the extensions. Next I further explore

the underlying reasons that lead to extensions not being protected by Simulacrum.

**Document_start.** I conducted an experiment on the 268 fingerprintable extensions to identify those that modify the DOM before the `DOMContentLoaded` event, which results in their fingerprints being included in the Parallel DOM. To that end, I installed each extension separately and visited the honeypage, taking a snapshot of the DOM at the `DOMContentLoaded` event, which I compared to the unmodified DOM. I find that 245 (4.22% overall) extensions make modifications before `DOMContentLoaded` as they are configured to run their content scripts at `document_start`, and alter the DOM before the construction of the Parallel DOM.

Next I evaluated the necessity of running the content script at `document_start` for extensions that modify the DOM before the `DOMContentLoaded` event. To that end, I randomly chose 20 extensions (that are in English and have at least 1k users) out of the 240 extensions, and modified the `run_at` attribute value in their `manifest` file from `document_start` to `document_idle`. I found that 19 extensions maintained their functionality despite my modification. Only one extension did not fully operate after the modification, which I believe was due to improper design patterns. Specifically, the developer of this extension implemented a non-critical logic at `DOMContentLoadedEvent`'s event handler and because I inject the code at `document_idle`, it does not work properly.

**Injected scripts.** While extensions have direct and full access to the DOM and are able to make all necessary DOM modifications through their content scripts, developers may decide to inject a `<script>` element into the page to make DOM modifications. To further explore this, 268 extensions were modified to prevent them from running their content scripts at `document_start`. To achieve this, `document_start` is replaced with `document_idle` in the manifest file and background script. Second, another browser extension was created to identify `<script>` elements injected

**Table 4.1:** Breakdown statistics for Simulacrum's effectiveness.

| **Extensions** | 5,793 (100%) |
|---|---|
| *Protected* | 5,525 (95.37%) |
| *Unprotected* | 268 (4.63%) |
| Document_start | 245 (4.22%) |
| Script injection | 40 (0.69%) |
| Partial fingerprints | 28 (0.48%) |

into the DOM. Third, I installed each modified extension besides Simulacrum and the new extension for collecting injected `<script>` elements and visited the Carnus honeypage; if we observe any injected `<script>` element we can conclude that it was injected by the extension. Finally, I identify if this injected `<script>` makes any DOM modifications. Since the injected `<script>` will not be transferred to the Parallel DOM, any modification in the Parallel DOM are the result of running this `<script>`. To find these modifications, we simply compare the Parallel DOM with its clean version. This way I found 40 (0.69% of the total) extensions that inject a `<script>` element into the DOM, and among these 17 extensions make modifications before the `DOMContentLoaded` event.

**Effectiveness.** Table 4.1 summarizes the Simulacrum's effectiveness. Overall, findings show that it is highly effective at protecting extensions from DOM-based fingerprinting as it is able to protect 95.37% of the extensions that can be fingerprinted by Carnus. For the remaining 268 extensions that cannot be protected, I find that the majority is due to extension modifications being made at `document_start`. While Simulacrum cannot hide these extensions, I propose straightforward strategies for addressing the underlying issues in Section 4.2.

**Simulacrum's impact.** Next, I seek to measure the extent to which Simu-

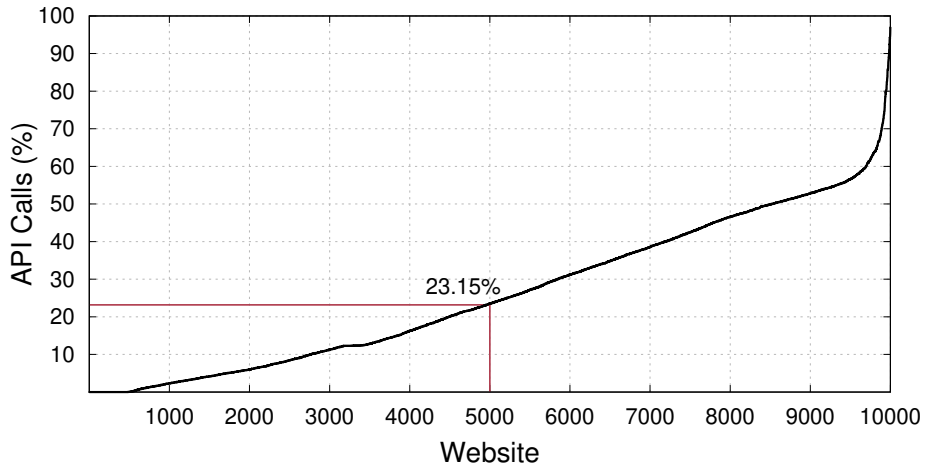


**Figure 4.1:** DOM-interacting APIs calls in the top 10K sites.

lacrum's defense affects the user experience. To that end I visit the top 1k websites and comparatively measure the overhead introduced by the presence of Simulacrum extension, versus a browser without the extension. I focus on two key aspects: the page loading time and impact on functionality.

*Function overriding.* JavaScript has a set of functions for interacting with the DOM; by overriding them, Simulacrum adds overhead to their execution time. Here I investigate the top 10k websites to better understand the performance penalty introduced by our defense. To that end, I first use VisibleJS extension to collect all the websites' API calls. In this experiment, I just collect the API calls that are executed after firing the `DOMContentLoaded` event, since only they incur Simulacrum's overhead. To identify which calls interact with the DOM, the type of the arguments, receiver, and return values of all API calls are assessed. If they are a node or any other DOM-related object, the API call is considered to be interacting with the DOM. Besides, the API calls that are used for managing data structures, such as functions that are implemented by the `Set`, `Array`, or `Map` interfaces were excluded. As an example, while the argument of `array.push(node)` is a node, it is not being counted as a DOM interaction.

Figure 4.1 depicts the percentage of API calls that interact with the DOM in each of the top 10K websites. I find that for 85% of the sites, Simulacrum does not intercept half of the JavaScript API calls, while for half of the sites almost 76.85% of the calls are unaffected. I also plot the 20 most commonly invoked interfaces in Figure 4.2. The most common interface is `String`, used more than one billion times in the top 10K websites. Among these interfaces Simulacrum only override the eight highlighted interfaces as Simulacrum does not need to modify the rest. I find that the invocations of `String` are 2.5x those of `Node` and 31x those of `Document`. Generally, I observe that the majority of JavaScript API calls do not interact with the DOM and are thus not affected by Simulacrum. Next, I focus on more precisely quantifying Simulacrum's performance overhead.

*DOM loading time.* In this experiment I visit the top 1k websites with two browser instances, one that has our extension installed and one without, and compare the time required for loading the websites' DOM. To account for uncontrollable fluctuations in the environment (network latency, OS scheduling, etc.), I average the results for each website over 10 runs. Overall, I was able to obtain both loading times for 925 out of the 1k websites that were visited. I observed that for 19 of the remaining 75 websites the page loaded successfully when visiting with a vanilla browser but did not complete loading when using a Simulacrum browser. The remaining 56 did not load successfully when using a vanilla browser but, interestingly, five of them loaded when visiting with Simulacrum installed.

In Figure 4.3 the difference in DOM loading times is presented for the 925 websites in both absolute and relative measurements. The results show that 69% of those websites incur less than a one second overhead, demonstrating the feasibility and practicality of Simulacrum approach. I also measured the delay imposed by overriding all relevant JavaScript functions, and the overhead of inserting noise; these amount

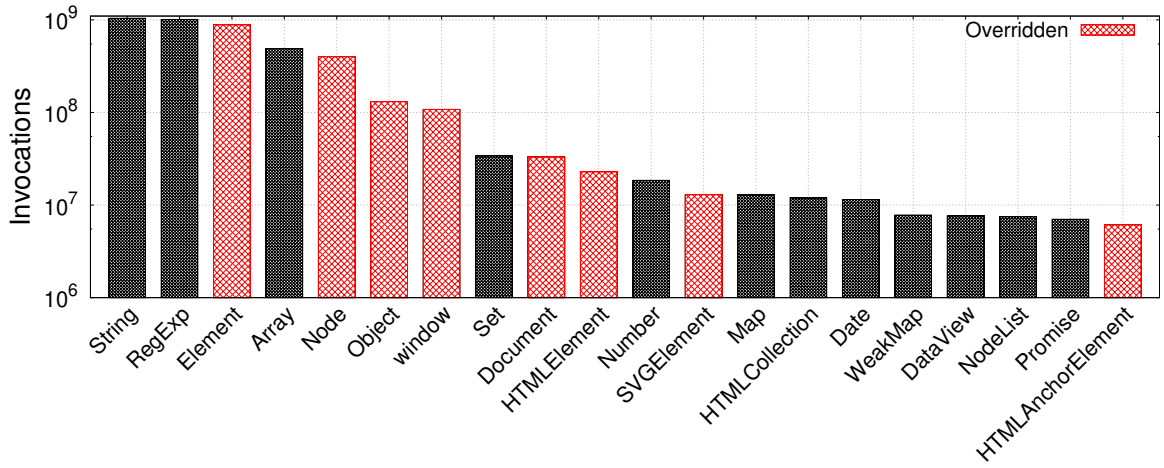


**Figure 4.2:** Interface invocations in the top 10K websites. Interfaces overridden by Simulacrum are explicitly noted.

to an average of 14.8ms and 31ms respectively.

Regarding DOM loading times, I observe a negative difference for 63 websites (6.81%) and a positive difference for 862 websites (93.19%). In the case of negative differences, these websites appear to complete loading faster when they are visited with a browser that has Simulacrum extension installed. The findings show that half of these websites (33 out of 63) have a "speed-up" lower than 200 milliseconds, and that only 6 of the websites appear to surpass one second. While small, such negative differences can be attributed to network delays. In the case of significant differences this could be the result of Simulacrum's defense affecting a website's functionality or blocking the retrieval of content, ads, etc., and thus complete loading faster. For the 862 websites that have a positive difference, where Simulacrum defense introduces a delay overhead, the findings show that this delay is 895 milliseconds (116% relative) on average; 563 of the websites exhibit more than 50% relative overhead, with 254 sites between 50%-100%, 194 between 100%-200%, and 115 sites with more than 200% of overhead, which heavily skews the overall overhead, as can be seen in Figure 4.3.

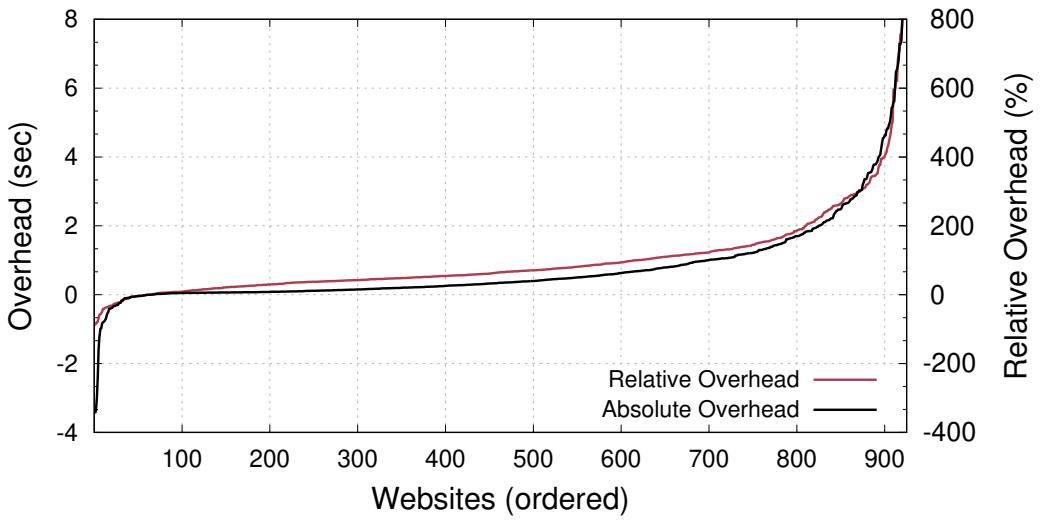


**Figure 4.3:** DOM loading time difference caused by Simulacrum.

More specifically, I observe that the delay for half of the websites (431 out of 862) is below 390 milliseconds (38% relative) and that only 11.72% (101 out of 862) observe delays that exceed two seconds (224% relative). When extensions that remove page elements are present, Simulacrum activates an additional protection for replacing these elements (see position-based attacks, §2.4.3.4), and an additional overhead of 172 milliseconds is incurred per website on average.

*User experience.* As the additional JavaScript code and DOM manipulations might impact the user experience, I experimentally quantify the impact using Google Lighthouse [142]. Lighthouse is an open-source automated tool for obtaining performance metrics and insights [142], which provides an overall performance score based on six metrics; among those, I was specifically interested in the `time-to-interactive (TTI)` metric as it is not affected by network fluctuations. The TTI metric focuses on the amount of time it takes for a web page to become fully interactive. To measure TTI, Lighthouse waits for the page to display useful content and registers event handlers for most visible page elements.

To evaluate Simulacrum on sites with heavy JavaScript usage I refer to BuiltWith [143],

which publishes a list of popular JavaScript technologies across the Web. I select five popular technologies and for each I randomly choose ten out of the list of top 50 websites. I run the Google Lighthouse performance analysis tool against each website, with and without Simulacrum installed, and visit each website ten times in each setup. I calculate the average `TTI` overhead for websites that loaded and had a positive overhead. As can be seen in Table 4.2, in most cases the overhead is less than one second, while the largest overhead of 2.47 seconds is observed in sites built with `Bootstrap`. When taking into consideration the average page loading times recently reported for popular websites in the US [144] (with ∼60% and 18% of pages requiring more than 10 and 30 seconds respectively) I argue that the overhead introduced by Simulacrum prototype is reasonable given the privacy enhancements offered by Simulacrum. I believe that this tradeoff can be significantly improved by leveraging DOM Reality Shifting for deploying defenses against other classes of web attacks, which I consider as part of future work.

Furthermore, I evaluated Simulacrum using the *Celtic Kane* [145] open-source benchmark for JavaScript and DOM speed tests. The results show that Simulacrum does not influence the duration of most test cases, as shown in Figure A.2 in the Appendix. While the DOM benchmark increases from 8ms to 38ms, overall our performance tests indicate that overhead is negligible or within acceptable ranges when compared to the loading times and responsiveness of modern websites.

*Memory overhead.* I crawled the top 1k websites with and without Simulacrum and measured the heap usage of JavaScript (`usedjsHeapSize/jsHeapSizeLimit)` using the `performance.memory` API (this experiment does not measure memory consumption for same-site iframes and workers and also ignores garbage collection). I visited websites five times and found an average increase in consumption of only 0.25%.

**Table 4.2:** Average Time-To-Interactive overhead in seconds.

| Technology | TTI Overhead |
|---|---|
| core-js | 1.44 (17.5%) |
| facebook-SDK | 0.99 (19.35%) |
| react | 0.65 (16.56%) |
| bootstrap | 2.47 (24.76%) |
| lodash | 0.58 (16.53%) |

*Website breakage.* Next I explore how Simulacrum anti-fingerprinting extension impacts the general functionality of web applications. To evaluate websites' functionality, I randomly selected 50 websites among the top 100 Alexa list and interacted with each website by testing common operations with Simulacrum installed (I provided more details on the approach in Appendix A). I opted for popular websites as they tend to offer rich functionality and make heavy use of a wide range of JavaScript features, and their complexity allows us to stress test Simulacrum prototype. I also note that a *continuous testing* approach was followed while developing Simulacrum, which allowed the identification of implementation peculiarities in websites and JavaScript frameworks/libraries leading to refining the Simulacrum's code for handling such cases. For instance, some issues were initially faced in Instagram website that were the result of the shallow JavaScript node cloning API, which in turn led to improve the implementation of the `deepClone` function. Assigning an `id` attribute to all page elements was also challenging for a website such as Spotify; it was handled by keeping a record of elements that do not require an `id` attribute and tuning the corresponding `getter` functions.

Table 4.3 details the breakage caused by Simulacrum, with major breakage in 6 sites (12%). Submitting credentials in the Google and Amazon login pages results in

**Table 4.3:** Breakage in 50 randomly-chosen popular websites.

| **Breakage** | **Statistics** | **Websites** |
|---|---|---|
| *Major* | 6 (12%) | |
| Authentication | 2 (4%) | `google.com`, `amazon.com` |
| Missing Content | 2 (4%) | `linkedin.com`, `ebay.com` |
| Disabled Actions | 2 (4%) | `twitch.tv`, `etsy.com` |
| *Minor* | 5 (10%) | |
| Disabled Elements | 1 (2%) | `csdn.net` |
| Appearance | 2 (4%) | `tmall.com`, `office.com` |
| Other | 2 (4%) | `instagram.com`, `apple.com` |

a crash or an error message. LinkedIn successfully passes authentication and displays contents of the main page, however navigating through the website is not smooth and contents might not show properly. Finally, product details on the main Ebay page are missing, but we can look up items in the website's search bar. I also found 5 websites (10%) with minor issues that do not drastically impact websites' core functionality, such as affecting the page's appearance or certain menu options being disabled. While Simulacrum system does not affect the majority of websites that I tested, privacy-preserving defenses typically introduce a trade-off between privacy and functionality. Finally, Simulacrum supports allowlisting websites.

*Extension breakage.* While the experiments demonstrate that Simulacrum is highly effective in hiding the presence of installed extensions, we also need to verify that this protection does not hinder extensions' functionality. Therefore, I randomly chose 50 English extensions from the list of protected extensions to evaluate their functionality. For each extension, I manually explore the extension to learn its

functionality in a clean Chrome browser, and then extensively re-evaluate its functionality in the presence of Simulacrum. The evaluation shows that all of the extensions work properly and retain their complete functionality, thus demonstrating that Simulacrum can effectively protect the randomly chosen extensions without hindering their functionality. I also experimentally verified that Simulacrum does not interfere with other privacy extensions that wrap APIs, by testing it with Privacy Badger [87], AdBlock [146] and uBlock [147] across five popular news websites.

## 4.2 Discussion, Limitations, and Guidelines

**Browser-based defense.** While Simulacrum effectively addresses the root cause of behavior-based fingerprinting, the experimental evaluation in this dissertation reveals certain corner cases or uncommon extension behaviors that lend themselves to fingerprinting. However, these idiosyncrasies are not fundamental to the extension's actual functionality and may even be the result of developer oversight. As such, providing explicit guidelines for developers seems beneficial so that it can allow them to protect their extensions from being fingerprinted through minimal changes that do not affect the extension's functionality. I hope that this inspires browser vendors to consider incorporating Simulacrum defensive mechanism as native browser functionality, as it would allow the browser to address cases that cannot be handled by this solution due to the inherent limitations of an extension-based defense, while further reducing overhead. I note that this is theoretically feasible but might, nevertheless, require significant effort or changes to a browser's design.

**Extension modification timing.** For changes that occur at `document_start`, modifications can be postponed to the `DOMContentloaded` event without affecting the extensions' functionality or the user experience in the vast majority of cases. Unless this is absolutely necessary for functionality, developers should opt for running at

`document_idle` or wait for the `DOMContentloaded` event to be fired prior to making changes.

**Injected scripts.** Evaluations in this chapter found 40 fingerprintable extensions injecting scripts that modify the DOM instead of directly modifying it from their content script. As the modifications occur from a script being executed in the page's execution environment, Simulacrum does not exclude them from the Parallel DOM. However, this approach does not offer any additional capabilities, and I strongly recommend extension developers to limit DOM-changing behavior to content scripts.

**Breakage.** Debugging popular websites and identifying the root causes of breakage requires significant engineering effort due to the use of advanced JavaScript libraries, obfuscation, and code minification. The most common cause of breakage early on was the *shallow clone* problem (see §2.4.3.1), which introduced inconsistencies between the Parallel and User DOM. Since Simulacrum is a research prototype that highlights the efficacy of DOM Reality Shifting for preventing DOM-based extension fingerprinting, I believe that Simulacrum's effects on JavaScript-heavy websites is acceptable, and should be incorporated by browser vendors into the browser. That engineering process would also likely address all breakage.

**Simulacrum detectability.** In practice, websites can infer the presence of Simulacrum extension (e.g., by observing the `id` attributes that are added to elements). However, this leaks minimal entropy compared to the entropy obtained by fingerprinting multiple installed extensions (which can be uniquely identifying [16; 148]), since the leaked information is a binary attribute representing whether Simulacrum is present (e.g., note the entropy of binary attributes compared to more complex ones like the `User-Agent` or the list of fonts [51]). Since the anonymity offered by privacy-enhancing technologies (PETs) correlates with the number of users [149], wide adoption of our defense would further decrease the entropy leak (similar to any other

PET, e.g., Tor [150]). If Simulacrum defense was incorporated into the browser this leak would be eliminated. Moreover, Simulacrum has the additional privacy benefit of preventing the inference attacks presented in [16].

**Timing attacks.** An interesting class of attacks are side-channel timing attacks. This would involve page JavaScript inferring if an extension added or modified an element due to Simulacrum executing different code paths. Similarly, Goethem and Joosen [151] exploited timing differences in evaluating the `web_accessible_resources` property inside an extension's `manifest` file to infer its existence. In general timing attacks are fairly tricky to implement, detect, and prevent, yet history has shown that they only increase in effectiveness. While such attacks are outside our current threat model, they represent an exciting avenue for future work.

**Manual analysis.** Simulacrum uses different wrappers for functions that are assigned to different categories. Categorizing functions relies on analyzing their behavior, which requires assigning the correct argument(s) and receiver, and observing the resulting behavior (which is not necessarily visible through JavaScript). Therefore, manual analysis of functions was necessary in course of development to ensure the correctness of overriding process. It is also important to note that JavaScript interfaces tend to remain stable, and any potential changes to methods can be easily handled. Moreover, any new DOM APIs that appear can be readily integrated following the existing implementation templates for Simulacrum.

## 4.3 Related Work

Numerous papers have demonstrated or leveraged browser extension fingerprinting techniques [17; 52; 152; 148; 16; 151]. Several studies over the past few years have either implemented or proposed a wide range of anti-fingerprinting techniques; however, all of the methods differ in scope or capability from the complete DOM-based-

fingerprinting prevention implemented by Simulacrum. As prior defenses mostly focus on other fingerprinting techniques, those studies are complementary to ours and, in practice, could be combined into a more holistic and comprehensive anti-fingerprinting defense.

Prior work has enabled users to manually disable extensions on specific websites [153; 154]. Simulacrum differs as it works automatically without requiring user interventions or decision-making. Other work [155] hides user interests by randomly visiting websites, defending against a complementary fingerprinting technique. Sanchez-Rola et al. [152] devised a timing-based method for fingerprinting extensions and proposed modifications to the web browser's extension invocation to prevent the timing-based attack.

Most related to Simulacrum is CloakX [19], a defense that modifies publicly accessible extension identifiers to prevent fingerprinting. The modifications randomize web-accessible resources [148; 17] as well as other identifiers. However, CloakX does not address behavioral fingerprinting and does not prevent the majority of fingerprints generated by Carnus [16]. Conversely, Simulacrum effectively targets the root cause of the attack, while employing a less invasive approach that does not require modification of the user's extensions.

Recent work proposed a countermeasure against CSS-based extension fingerprinting [18]. Their defense creates a mirror copy of the DOM tree using Shadow DOM, which automatically excludes content styles from extensions. The defense overrides the `getComputedStyle` method of each element to return the style of the Shadow DOM elements so as not to include the content styles injected by extensions. While this work protects against a different class of fingerprinting than Simulacrum, its proposed CSS defense would benefit from using our techniques. It is important to note that the `Shadow DOM` API [156] is not sufficient for implementing our defense, as it

was not designed to secure or hide page elements; instead, it encapsulates elements to prevent problems such as variable name collisions. Using it does not prevent a malicious script from accessing the `Shadow DOM`'s content (even when attached in `closed` mode). Therefore, the `Shadow DOM` does not satisfy our security requirements. Unlike any prior work, Simulacrum robustly overrides 1,532 functions that isolate DOM modifications and prevents DOM-based fingerprinting.

Simulacrum extends the concept of `virtual machine layering`, which creates an abstraction layer on top of the JavaScript VM that controls sites' access to APIs. This was used for instrumenting JavaScript [157], enabling replay [158], and controlling access to APIs [88; 159]. Photon [157] transforms JavaScript (outside the browser) to create a VM layer that developers can use to instrument and evaluate their code. Mugshot [158] captures and replays JS events using a server-side web proxy. This only requires a single overriding strategy, removing the need for categorizing functions (as it is done for Simulacrum) and allowing for straightforward automated overriding; this, however, would not be sufficient for the challenging process necessitated by Simulacrum's defense. Moreover, unlike Photon and Mugshot, anyone can use Simulacrum to protect their privacy *without* requiring any external operations (e.g., translation or a server-side web proxy) or user intervention. Even though Chrome Zero [88] and the tool from [159] enhance privacy by allowing users to limit sites' access to JavaScript APIs, their approach is inherently designed to *prevent website functionality* while our system focuses on *maintaining* it.

## 4.4 Conclusions

The evaluations in this chapter demonstrate Simulacrum's effectiveness against the state-of-the-art DOM-based fingerprinting system, while introducing minimal performance overhead, low breakage for websites and negligible breakage for other exten-

sions' functionality. Overall, I envision DOM Reality Shifting as a building-block for various advanced privacy-enhancing browser countermeasures.

In this chapter, the effectiveness of the Simulacrum extension was examined in protecting users and concealing the presence of extensions from certain extension fingerprinting attempts. In the next chapter, I will explore how we can safeguard user accounts against fingerprint spoofing and account takeover (ATO) attempts through the use of minimal, non-invasive browser fingerprints.

Chapter 5

# EFFICIENT DEPLOYMENT OF COARSE-GRAINED BROWSER FINGERPRINTS FOR WEB-SCALE DETECTION OF FRAUD BROWSERS

## 5.1 Introduction

The war between fraudsters and website operators has been ever-present since the first login form was conceived. Fraudsters use any means necessary to steal cookies or login information from victims, and then use that stolen information to authenticate to the website as the victim [160; 161; 162]. Such attacks are called *account takeover fraud* or ATO [163]. ATOs cause significant financial damage—the FBI reports that victims reported 2.7 billion USD in losses due to Business Email Compromise (which is a fraud that is just a subset of ATO) in the year 2022 alone [21].

One way that website operators have attempted to protect users is through *risk-based authentication*—analyzing available information about the login attempt to assess the risk of it being an in-progress ATO attempt. One such technique is the matching of the user's current browser against prior browsers used by that account. Freeman et al. [164] used the `user-agent` to identify the browser, and follow-up work has leveraged the burgeoning field of browser fingerprinting (originally identified as a privacy threat to users, but, in this case, utilized "for good") [165; 166; 167; 72].

However, recent work shows that attackers can easily steal and replay browser fingerprints [168; 169]. In fact, criminals have turned this into a type of cybercrime-as-a-service [170]: When account information is stolen, the attacker also collects a browser profile. Underground marketplaces such as the Genesis Market (through which over 80 million stolen profiles for various services of over 2 million potentials

were traded before the site was shut down by the FBI in 2023 [22]) facilitate the sale of these profiles on a massive scale. After acquiring stolen profiles, fraudsters load them into specially-customizable web browser variants, so-called "anti-detect browsers" ostensibly created for privacy purposes, to bypass risk-based authentication and commit ATO fraud. In this dissertation, I refer to these browsers as *fraud browsers*. Notably, financial institutions are one of the most impacted sectors, facing numerous cases of fraud perpetrated through *fraud browsers* [171].

Fine-grained browser fingerprinting offers a solution to detecting fraud browsers through its user tracking techniques. These techniques generate detailed fingerprints using advanced browser fingerprinting techniques such as WebGL [67; 46], canvas [45; 10] and even CPU timing [50], which can then be compared with previously established user fingerprints to identify potential fraud. Nevertheless, through consultation with a leading financial institution, herein referred to as *FinOrg* [1] , I found that existing fine-grained fingerprinting methods face deployment challenges at *FinOrg* due to stringent constraints on fingerprint extraction time and data size. Current practices for thwarting fraud browsers predominantly rely on manual effort (studied by Azad et al. [172]) and may become obsolete with the release of new browser software.

Addressing these issues, I present a prototype for the analysis of *efficient coarse-grained fingerprints* within large-scale Web traffic, aimed at fraud browsers detection. By coarse-grained browser fingerprints, I refer to a simplified and less detailed set of browser fingerprinting features designed specifically for fraud detection, optimized to meet resource and performance constraints (as defined by FinOrg). These fingerprints, target discrepancies in the implementation of JavaScript APIs across different browsers. This novel, data-driven, and privacy-preserving strategy focuses on

[1] FinOrg is a major financial company that provides financial services through internet platforms.

training a machine learning model via minimal coarse-grained features from *FinOrg* users. My method aims to rigorously validate the authenticity of a browser session's reported `user-agent` string, which fraudsters typically manipulate to match their target's `user-agent`. I have extended existing research in coarse-grained fingerprinting [69; 173], which are typically used to infringe user privacy, and instead apply the proposed extended techniques to large-scale Web data, with a particular focus on privacy, coarse granularity, efficacy, and continuous detection adaptability.

In my prototype, I create a clustering model, called Browser Polygraph, that operates on privacy-preserving and coarse-grained fingerprints and determines a *risk factor* score that represents divergences between the actual identity of the observed browser and its identity as claimed by its `user-agent`. Browser Polygraph is tailored to meet the performance demands of large, real-world, financial companies, focusing on efficient and effective browser fingerprinting without delving into expensive, fine-grained analysis.

I deployed Browser Polygraph in production at FinOrg for 4.5 months, observing over 205,000 user sessions. Of these, Browser Polygraph identified 897 suspicious sessions, 2% of which were used in an ATO attack within 72 hours (compared an ATO prevalence of 0.43% across all sessions). Of the sessions that Browser Polygraph identified as *highly* risky, 5.83% were used in an ATO within 72 hours.

I designed Browser Polygraph with concept drift in mind, and built in mechanisms to detect when retraining is needed (e.g., due to new browser versions). I experimentally show that retraining was needed approximately after three months. This duration may vary depending on the specific features of new browser releases, but it can be automatically triggered by Browser Polygraph's drift detection module, increasing the resiliency of the approach in the cat-and-mouse game between attackers and defenders. Furthermore, in a separate evaluation, I assessed Browser

Polygraph's efficacy in detecting a subset of fraudulent browser fingerprints generated by fraud browsers, demonstrating its potential in identifying these threats. In essence, Browser Polygraph is a preliminary prototype that represents a significant step forward in the automatic generation of coarse-grained browser fingerprints and their deployment in risk-based authentication systems. This approach can be further enhanced by future advances in fine-grained browser fingerprinting, ensuring continued efficacy in the face of evasion by cyber criminals.

In summary, the contributions of this paper are as follows:

- I propose a new privacy-preserving idea of detecting lying browsers while maintaining the performance characteristics necessary for real-world deployment at FinOrg using efficient coarse-grained fingerprints.
- I leverage this idea into a clustering model, called Browser Polygraph, that outputs `risk factor` of suspicious user sessions, which can be used as part of risk-based authentication.
- I deployed Browser Polygraph at a major financial company FinOrg, where I demonstrated its effectiveness at detecting real-world fraud.
- I demonstrated Browser Polygraph's effectiveness (in an independent evaluation) in detecting a subset of fraudulent browser fingerprints created by fraud browsers.

## 5.2 Background

This chapter builds on past research in browser fingerprinting, repurposing it to detect fraud browsers. Browser fingerprinting has already been explored in Section 2.2. Having that background, I will introduce fraud browsers and present the results of my investigations on the behaviors of fraud browsers.

### 5.2.1 Fraud Browsers

Privacy concerns have given rise to specialized browsers, commonly referred to as *anti-detect browsers*, designed to obscure user identity. These browsers allow a user to distort or hide a browser's identifying information, such as IP addresses, user-agents, JavaScript attributes, etc. However, these browsers are being actively misused by criminals—not for privacy reasons, but for conducting fraudulent activities. Therefore, I use the term *fraud browsers* in this dissertation to describe these browsers. Fraud browsers allow criminals to load profiles of the victim, typically stolen by phishing kits [161], and then later sold on online marketplaces [170]. Some marketplaces even have their own fraud browsers, such as the now-defunct Genesis Market.

Since addressing the challenges presented by fraud browsers is a vital element of this research, I examined several notable fraud browsers employed today. This examination revealed a commonality in their adeptness at customizing, randomizing, or concealing typical fingerprinting attributes, such as Timezone, Language, Screen Resolution, WebRTC, WebGL, Canvas, Audio Context, Geo Location, etc. Specifically, I analyzed browsers like, ClonBrowser [174], AntBrowser [175], Incogniton [176], GoLogin [177], Octo Browser [178] Linken Sphere [179], Sphere [180], VMLogin [181] CheBrowser [182], and AdsPower [183], noting usage of some of those in real-world fraud.

The uniqueness of each fraud browser lies in its functionality. For instance, Linken Sphere offers extensive customization, including browser or OS choice, fingerprint profiles, and cookie import capabilities. While offering fewer browser and OS customization options, Che Browser provides a unique service that allows users to purchase profiles for various Chrome versions. Incogniton and Octo Browser, both Chrome-

based, support user profiles and cookie imports. GoLogin provides a wide range of operating system options and the capability to manage from 100 to thousands of user profiles. My focus diverged from these software's extensive capabilities towards a more targeted approach. I concentrated on uncovering inconsistencies within the simplest and most direct JavaScript attributes rather than comparing their sophisticated spoofing or randomization techniques (details in Section 5.6).

### *5.2.2 Fraud Browsers' Behavior*

I further delved into the behavior of fraud browsers to understand how they operate so that I could create strategies to detect them. I installed several fraud browsers (noted in Table 5.1) on a Windows machine and analyzed how their browser fingerprints change with altering the `user-agent`. This led to the categorization of these browsers into three primary strategies, with a fourth category identified for fraudulent use of legitimate browsers in spoofed environments:

**Category 1:** Browsers such as Linken Sphere and ClonBrowser which exhibit browser fingerprints that do not match any legitimate browser's fingerprint.

**Category 2:** Browsers in this category maintain a browser fingerprint that matches a legitimate browser fingerprint, but the fingerprint does not change when modifying the `user-agent` string.

**Category 3:** Browsers that adopt the browser's JavaScript engine and its browser fingerprint with each `user-agent` selection.

**Category 4:** Legitimate browsers (with same `user-agent` and fingerprint) used in spoofed environments to load stolen information.

Table 5.1 details the investigated fraud browsers and their categorization. Cate-

**Table 5.1:** Fraud browsers categorized based on definitions in Section 5.2.2. Release dates with a preceding ˜ are approximations based on related version release dates or the date of experimentation (Rel. = Release).

| Browser | Rel. Date | Category | New Rel.? |
|---|---|---|---|
| Linken Sphere-8.93 | April 2022 | 1 | ✗ |
| ClonBrowser-4.6.6 | ˜May 2023 | 1 | ✓ |
| Incogniton-3.2.7.7 | ˜May 2023 | 2 | ✓ |
| Gologin-3.2.19 | May 2023 | 2 | ✓ |
| CheBrowser-0.3.38 | ˜May 2023 | 2 | ✓ |
| VMLogin-1.3.8.5 | April 2023 | 2 | ✓ |
| Octo Browser-1.10 | September 2023 | 2 | ✓ |
| Sphere-1.3 | November 2023 | 2 | ✗ |
| AntBrowser | May 2023 | 2 | ✗ |
| AdsPower-4.12.27 | December 2022 | 3 | ✓ |
| AdsPower-5.4.20 | April 2023 | 3 | ✓ |

gory **(1)** and **(2)** browsers are prevalent and exhibit detectable inconsistencies, making them susceptible to identification through coarse-grained browser fingerprints. Category **(3)** and **(4)** browsers (likely employed by more resourceful attackers), however, aim to approximate recreating a user's entire browser environment (and a genuine fingerprint), requiring sophisticated fraud detection techniques. As noted by Azad et al. [172], these precise spoofs often result in configurations that are challenging to distinguish from legitimate usage. Therefore, Browser Polygraph focuses on detecting Categories **(1)** and **(2)** browsers using coarse-grained fingerprinting, given its resource efficiency and practicality for my Web-scale browser authentication goals that I will explain in the next section.

## 5.3 Web-Scale Fingerprinting Requirements

I collaborated with FinOrg to understand their performance requirements based on the technical requirements of a real-world risk system deployed in a significant-traffic web application. This led to two core requirements: (1) fast response time, operating within 100 milliseconds, and (2) data extracted per-user should be minimal, under the threshold of one kilobyte.

Therefore, with these thresholds in mind, I evaluated the performance overhead of the state-of-the-art fingerprinting techniques mentioned in Section 2.2. Specifically, I evaluated the performance overhead of three prominent fine-grained fingerprining tools: FingerprintJS [184], AmIUnique [185], and ClientJS [186]. FingerprintJS and ClientJS were chosen due to their significant market share (as reported by Wappalyzer [187]) in browser fingerprinting. AmIUnique is an academic contribution from Laperdrix et al. [51].

My analysis specifically focused on evaluating these tools' response times and data processing overhead. I calculated the average service time over five visits for each tool to assess response times accurately. In refining the analysis of storage needs for fraud detection, I shifted focus from the size of hashed data to the underlying data structure's size, which is crucial for hashing. This distinction is vital as my interest lies not in the hash values but in verifying whether a browser's specifics align with its claimed `user-agent`.

For FingerprintJS, I utilized their online tool for direct performance metrics. AmIUnique's data was gathered via its browser extension. ClientJS required a different approach; as an npm library, I embedded it into a basic Node.js application to produce an HTML page invoking the fingerprinting library. Before hashing, I logged the necessary data to the console to evaluate its storage requirements.

Table 5.2 demonstrates the evaluation results. While AmIUnique offers extensive fingerprint data collection through its browser extension, its service time and storage needs do not satisfy the requirements. FingerprintJS and ClientJS, although fast, fail to meet the data storage requirement. These results are in line with expectations, as fine-grained fingerprinting, which is required for uniquely identifying a specific user's browser, inherently requires extracting a significant amount of data.

Given the performance overhead of current fingerprinting techniques, particularly given the FinOrg requirements for high-traffic scenarios and real-time security requirements, I propose Browser Polygraph. Browser Polygraph is guided by two principal criteria:

1. **Minimizing latency:** Targeting efficiency, Browser Polygraph is optimized to operate within real-time security constraints.

2. **Minimizing data size:** It is designed to function effectively under stringent data size limitations.

The comparison in Table 5.2 underscores my approach. I aim for rapid response times akin to FingerprintJS and ClientJS while significantly reducing the fingerprint data size. This balance makes Browser Polygraph an optimal solution for Web-scale fraud detection, tailored for the stringent requirements of modern digital security landscapes, addressing both efficiency and scalability (I will explain the performance metrics of Browser Polygraph in Section 5.7.5). It is important to note that the comparison in Table 5.2 highlights timing and memory overhead as the key factors in the initial stage of fingerprint-based fraud detection, where browser data is collected. With respect to performance, fine-grained techniques are primarily designed for user tracking. However, not all of the detailed information collected by these techniques is necessarily efficient for identifying lying browsers, which is the main focus of the

**Table 5.2:** Comparison of time and storage requirements for fine-grained fingerprinting tools and my proposed Web-scale data-driven fraud detection solution.

| Tool | Avg. service-time | Storage req. |
|---|---|---|
| **AmIUnique** | ~1.5s | ~60KB |
| **FingerprintJS** | 51ms | ~23KB |
| **ClientJS** | 37ms | ~10KB |
| **Browser Polygraph** | 6ms | 1KB |

coarse-grained fingerprinting approach (Browser Polygraph) proposed in this dissertation.

## 5.4 Threat Model of Browser Polygraph

In this chapter, I consider an adversary equipped with a victim's credentials (such as username, password, or cookies) and their browser fingerprint, which includes the `user-agent` and any other stolen browser information (which could be bought on the dark web [188]). The attacker's objective is to execute an Account Takeover (ATO) attack using a fraud browser. This adversary has the capability to customize and alter the settings of a chosen fraudulent browser to mimic the victim's fingerprint (the attributes of which have been discussed in Section 2.2). I assume that the attacker will, at a minimum, accurately set the most straightforward and critical attribute for browser fingerprint spoofing—the victim's `user-agent`.

On the defense front, the primary aim in this chapter is to introduce a continuous, low-overhead, fingerprinting-based fraud detection mechanism that detects such ATO attacks. This defense mechanism is distinct from risk-based authentication schemes that validate a user's fingerprint upon logins [169; 173] or in response to access requests to sensitive actions. The proposed approach focuses on persistently monitor-

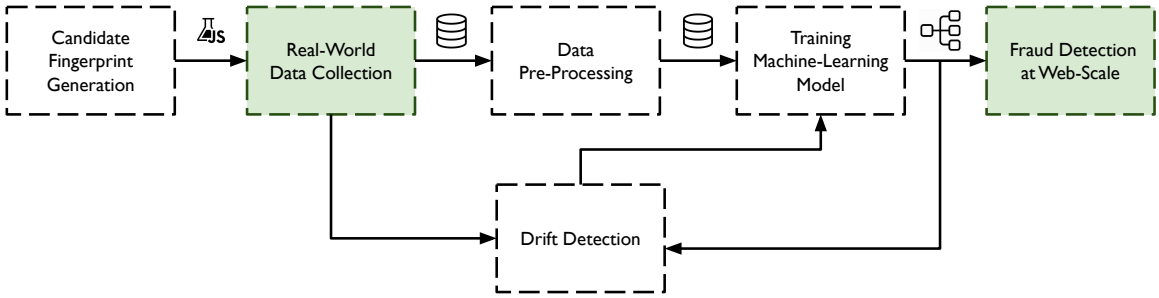


**Figure 5.1:** Overview of Browser Polygraph offering fraud detection at Web-scale. Rectangles in green and white colors suggest online vs. offline tasks.

ing and restricting access of fraud browsing sessions to the FinOrg's infrastructure, thereby minimizing the attack surface available to malicious actors.

## 5.5 Overview of Browser Polygraph

In this research, I introduce a prototype that combines efficient coarse-grained browser fingerprinting with machine learning to identify suspicious browsing activity by detecting if a browser is lying about its `user-agent`. Notably, I am specifically *not* exploring new browser fingerprinting techniques. Instead, I focus on using established, yet effective, methods to detect user sessions showing inconsistent behaviors between the claimed browser and the actual browser. The proposed system is designed to seamlessly adjust to new browser updates, ensuring it remains effective against emerging suspicious browser fingerprints.

The fingerprinting technique in this chapter targets discrepancies in browsers' implementation of the JavaScript APIs, taking inspiration from the work of Akhavani et al. [189]. The evolving landscape of JavaScript APIs makes them a fertile ground for such fingerprinting efforts.

First, I generated JavaScript-based fingerprints by collecting fingerprints on many legitimate browser vendors and versions to identify those *candidate JavaScript attributes* that can be used for fingerprinting. During this selection process, my focus

was on attributes that could be extracted within the performance constraints outlined in Section 5.3 and that uphold privacy standards—namely, these attributes should not enable user tracking but should be viable for fraud detection purposes.

However, I needed to identify the discriminatory fingerprintable features on real-world browsers, which I found are varied in number and different configurations than I could manually test. I deployed the candidate fingerprint collection script through my collaboration with FinOrg. After gathering data for several months, I analyzed it, to identify the JavaScript-based fingerprints that were significant for detecting browser versions. The resulting set of coarse-grained browser fingerprints then served as the foundation for my machine learning model. The model's objective is straightforward: to determine if the `user-agent` of a browser matches the expected features of a similar but legitimate browser.

Figure 5.1 shows a high-level design of the proposed prototype, which includes the following core components:

**Candidate Fingerprint Generation.** I ran multiple JavaScript fingerprint extractions on legitimate browser versions/vendors and identified *candidate JavaScript attributes* with considerable output variances between browsers.

**Real-World Data Collection.** I passed the candidates to FinOrg and they deployed a collection of the candidate fingerprintable JavaScript attributes on FinOrg, and collected this data.

**Data Pre-Processing.** After processing the collected candidate fingerprintable data, I analyzed it to identify key discriminatory fingerprintable features, to reduce the ultimate size of the collected data.

**Training of Machine-Learning Model.** After identifying the features, I trained a clustering algorithm. Its primary function is to group browsers together based on how they implement the JavaScript APIs.

**Fraud Detection.** With the clustering algorithm, Browser Polygraph identifies if the features of a browser (extracted using the coarse-grained fingerprints) map to the same cluster as the browser's `user-agent`. Non-matching is indicative of fraud, and I further calculate semantic similarity between the claimed browser and the fingerprint-matching browser to assign a `risk factor`.

**Drift Detection.** The clustering algorithm's ability to correctly assign clusters to new browser versions can regularly be assessed, initiating retraining as needed to maintain accuracy.

Emphasizing *user experience*, the proposed design minimizes operational disruptions, conducting most processes offline while handling real-time Data Collection and Fraud Detection.

## 5.6 Design

Now, I delve deeper into the structural and functional aspects of my fraud detection system Browser Polygraph. Each of the five components, previously highlighted in Section 5.5, will be examined in detail to offer a comprehensive understanding of their design and operation.

### *5.6.1 Candidate Fingerprint Generation*

The central objective of this component is the identification of candidate JavaScript fingerprints that are suitable to be evaluated for their usefulness in the subsequent step of Real-World Data Collection. These candidate fingerprints aim to discern variations in the implementation of JavaScript APIs between legitimate browsers. The emphasis, unlike prior work, is not on pinpointing specific browser versions/vendors but to achieve coarse-grained browser fingerprinting. As my evaluation in Section 5.7 will show, this granularity is sufficient to detect discrepancies that may emerge during

fraud detection.

While the quantity of candidate fingerprints advanced to the subsequent phase is flexible, to collect data from FinOrg, they must align with the timing and data storage requirements outlined in Section 5.3. Additionally, it was essential that the candidate fingerprints be privacy-preserving and not enable user tracking. Therefore, rather than using all possible candidate fingerprints, I ran automated experiments on browser instances to identify distinguishing candidate fingerprints.

I tested browsers from Chrome-based and Firefox releases from mid-2017 to mid-2022. Notably, the testing environments varied: while I often used BrowserStack, sometimes local installations of browsers were used. Throughout this project, I only needed to perform the candidate fingerprint generation stage once, in mid-2022. Following this, I collected fingerprints for new browser releases as needed using BrowserStack. Overall, I gathered fingerprints from browser instances ranging from Chrome 59–119, Firefox 46–119 to Edge 17,18, and 80–119. In the following, I will explain the process of generating the list of candidate JavaScript fingerprints.

To select the most suitable JavaScript candidate fingerprints, I based the initial list on Akhavani et al.'s BrowserPrint [189], as it discussed browser fingerprinting via JavaScript attributes. BrowserPrint used the existence of a JavaScript attribute as a fingerprintable technique: the idea being that browsers introduce different features at different times, and these can uniquely identify the browser vendor and version. However, I anticipate these features to become less effective over time. My insight is that, rather than storing the existence of every fingerprintable attribute, which requires not only storage but also continuous updating, I abstract the *shape* of a specific fingerprintable JavaScript prototype by capturing *only the number of properties of the JavaScript prototype*. A set of these build my *coarse-grained fingerprints*.

To determine the JavaScript objects to include in the list of candidate fingerprints,

I prioritized the progression of JavaScript interfaces, assembling a comprehensive list from MDN [190] documentation, which comprised 1006 prototype names, and assessed them using `Object.getOwnPropertyNames` method to count the properties of each prototype. With the properties count serving as coarse-grained fingerprints, I extracted all possible fingerprints from all candidate browsers, sorted these fingerprints based on their standard deviation across all candidate browsers, and identified the top 200 features from the sorted list (*deviation-based features*). The normalized standard deviation of the selected features ranges from 0.0012 to 1.3853.

As discussed earlier, BrowserPrint [189] utilizes the presence or absence of specific JavaScript properties as fingerprinting APIs, identifying 313 such JavaScript features, which they refer to as suspicious APIs. They analyzed these features for their unique fingerprinting behavior across various browser versions. Because these properties tend to appear or disappear over time, I categorize them as *time-based* features. I chose to integrate these features into the subsequent real-world data collection phase. However, since BrowserPrint's analysis covered browser versions up to 2020, and my study focuses on newer browsers released before mid-2022, I anticipated that this list might be outdated for the current use-case. As I will demonstrate, only 6 remained relevant and were included in my data pre-processing stage (explained in Section 5.6.3).

Another lens through which browsers' evolution can be viewed is their varied implementations of certain functions, thus acting as potential fingerprinting indicators. Targeting no-input functions, I analyzed their behavior by navigating the top 100 websites on different browser instances (limited to Firefox and Chrome) using the VisibleJS [20] extension. However, the limited volatility observed in no-input functions led me to ignore those and narrow down my features to 513 JavaScript candidate fingerprints (*deviation-based* and *time-based*) for the subsequent phase (full

list in Appendix D).

### 5.6.2 Real-World Data Collection

After developing 513 JavaScript candidate fingerprints, I collaborated with FinOrg to collect coarse-grained candidate fingerprints of real-world browsers. My purpose was to record coarse-grained candidate fingerprints alongside the `navigator.userAgent` attribute. FinOrg integrated my script into one flow of an online shopping platform, with continuous data collection over eight months. During this period, they provided me with periodic datasets, laying the groundwork for my analysis. The data comprised integer outputs from the 513 JavaScript candidate fingerprints, the `navigator.userAgent` string, and a fully anonymized `SessionID`, ensuring users' privacy (as described in Section 5.10). I used four and a half months of this data (comprising 205k rows) for training of my machine learning model. In the following, I detail the process of refining the collected data and the methodology behind the training of my machine learning model.

### 5.6.3 Data Pre-Processing

As I delved into the collected data, unexpected patterns surfaced, challenging my initial hypotheses established during the Candidate Fingerprint Generation phase. A particular data sample from the first day of March revealed that 186 features exhibited a singular value across all samples. This was particularly notable among *time-based* features, where 40% displayed unique values, suggesting their diminishing relevance in newer browser versions. Consequently, after thoroughly reviewing the documentation and identifying vendor-specific distinctions, I refined the feature selection, focusing on 6 of *time-based* features that were more indicative of genuine browser behavior.

From analyzing the *deviation-based* features, I noted a significant portion exhib-

ited unique values (30%), leading to their exclusion from further analysis. However, the remaining data revealed inconsistencies in some feature values among identical browser instances. Finding the root cause of these discrepancies is a difficult task: our only data is the coarse-grained fingerprint and a `user-agent`, where the `user-agent` does not match. This could be a fraud instance, or a benign instance, and therefore I conducted an in-depth manual analysis of the wider breadth of legitimate browsers, particularly of Firefox and Chrome browsers and their derivatives. This process uncovered specific facts that influenced those feature values (and I did not initially consider). Some of my notable revelations from real-world data are described in the following:

• **Firefox configuration influence**: Firefox has an array of security and performance configurations via the `about:config` page. Customizing these settings can inadvertently alter feature values. For instance, disabling Service Workers through `dom.serviceWorkers.enabled` can result in values related to `ServiceWorker` interfaces being zeroed. Additionally, manipulating settings such as `dom.element.transform-getters.enabled` can influence properties manifested through the `Element` interface.

• **Chrome extensions & flags**: Chrome's equivalent to Firefox's configuration page is `chrome:flags`. While it mostly pertains to experimental properties—and is arguably less frequently modified by the average user—it was deemed negligible for this study. However, Chrome extensions, which are widespread among users, were observed to impact fingerprinting values. For instance, the DuckDuckGo extension introduced two custom properties to the `Element` interface; this modification increments the integer value of one of the features by two. To assess the influence of common extensions, I revisited the set of default Chrome extensions previously identified by Picazo et al. [191] in 2020. At the time of my review in April 2024, I found

that, of the seven removable extensions reported, Google had integrated the functionality of six into Chrome as built-in apps, leaving only one as a default extension. Notably, the removal of these integrated apps and the remaining extension had no impact on the values of the selected feature set.

• **Brave**: a privacy-centric browser based on Chromium that could be misidentified as Chrome. My investigation revealed while the examined version of Brave generated a `user-agent` identical to Chrome 111, there were discernible discrepancies in attribute values across certain interfaces, such as `Element`, compared to the genuine Chrome 111.

• **The Tor Browser**: Mirroring Brave's behavior, the examined version of Tor Browser presented a `user-agent` string aligning with Firefox version 102, yet the attribute values significantly deviated from those of the original Firefox 102. Given that Tor Browser's updates had lagged—being nearly a year behind the contemporary Firefox iteration—I deemed it prudent to exclude this browser instance from my analysis.

Based on this analysis, I adjusted the feature set, particularly for *deviation-based* attributes, some of which could be modified by user/browser privacy settings. I identified and excluded the most affected features to enhance the reliability of the coarse-grained fingerprints. Moreover, a detailed examination of real coarse-grained fingerprints highlighted minimal deviations in certain features, and I excluded those as well. From this process, I selected 22 *deviation-based* features, complemented by 6 *time-based* features. This refined selection forms my comprehensive set of 28 features, optimizing the fraud detection capabilities. The details of these features are cataloged in Table C.1 of Appendix C.

### *5.6.4 Training of Machine-Learning Model*

In this section, I delve into my data-driven fraud detection system, Browser Polygraph. The core objective of Browser Polygraph is to verify the veracity of the browser's identity, specifically its `user-agent` string, and to identify any significant deviations that might indicate a falsified identity. My dataset comprises 28 feature values for each user session, each linked to its respective `user-agent` string. Given that feature values tend to be consistent across similar browser releases, I opted for a semi-supervised learning approach for the model.

The initial phase of my methodology involves disregarding the `user-agent` strings and focusing solely on the feature values, which represent real-world browser fingerprints. This data is then employed to train the clustering model, designed to group browsers into distinct clusters. These clusters are formed based on the similarities in the implementation of JavaScript APIs by different browsers. For the clustering process, I selected the `kmeans` clustering algorithm due to its efficacy in handling such categorization tasks.

Post-training, the model is further refined by leveraging the ground truth of the `user-agent` strings. This step involves analyzing how these `user-agents` are distributed across the formed clusters, thereby enabling us to assess the model's accuracy in correlating the `user-agent` strings with the expected browser features. The following will provide an overview of the fitting and training processes.

#### 5.6.4.1 Pre-Processing

Some of my features had large values which could skew the results of the model towards them. Therefore, I used Standard Scaler to scale some of the *deviation-based* attributes. The *time-based* attributes were already in the binary format which was

suitable. Another crucial aspect of preprocessing was outlier detection, for which I employed the Isolation Forest method [192]. This method was selected due to its effectiveness in identifying anomalies in high-dimensional datasets. By setting a threshold of 0.002%, I successfully filtered out data presumed to be outliers, ensuring a more refined and accurate dataset for subsequent analysis. Upon examining the data excluded by the 0.002% threshold, it is evident that none of the eliminated records (172 rows) corresponded to feature values indicative of a legitimate browser instance, as delineated in Section 5.6.1. Hence, this threshold selection is demonstrably appropriate.

#### 5.6.4.2 Feature Selection

In the feature selection phase, I used Principal Component Analysis [193] (PCA) to streamline the dataset. PCA is a technique in machine learning that reduces dimensionality while retaining the most informative aspects of the data. It works by transforming the original features into a new set of variables, the principal components, which are orthogonal and maximize variance. The decision to use PCA was driven by its efficiency in simplifying complex data without significant loss of information. Analysis of the PCA results, particularly through the visualization shown in Figure 5.2, revealed that using seven components could capture over 98.5% of the cumulative variance in the dataset. This level of variance was deemed optimal for my analysis, striking a balance between simplicity and information retention.

#### 5.6.4.3 Clustering

In this part of the analysis, having established seven as the optimal number of components through PCA, I employed the `kmeans` clustering algorithm on the dataset (kmeans was chosen due to its efficiency and straightforward implementation). The

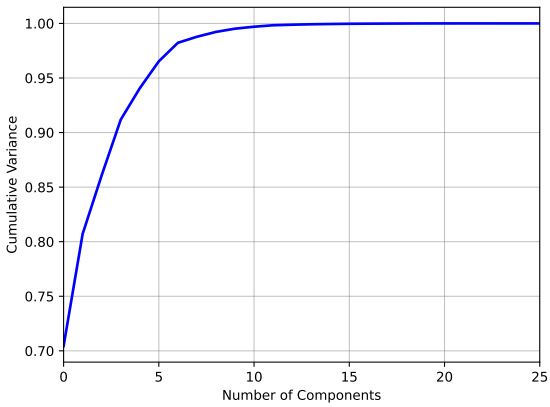


**Figure 5.2:** Cumulative variance based on the number of components in PCA.

optimal number of clusters ($k$) was determined using the elbow method, as illustrated in Figure 5.3. This method plots the Within-Cluster Sum of Squares [194] (WCSS) against the number of clusters to evaluate the variance within each cluster. A decrease in WCSS signifies a closer alignment of data points to their cluster centroids. In my analysis, the elbow graph demonstrated potential optimal cluster counts at $k = 3$, $k = 6$, and $k = 11$. At these points, the graph shows a noticeable "elbow", implying that additional clusters would not significantly improve the fit to the data. To further refine my decision, I analyzed the relationship between WCSS values and the number of clusters in Figure 5.4, where a pronounced increase at $k = 11$ suggested it as the ideal choice.

After selecting k=11 as the optimal number of clusters, I evaluated the model's accuracy. Since I am using a semi-supervised learning approach, the accuracy metric measures how well the clusters represent the `user-agent` instances. I define a cluster assignment as accurate if all instances of the same `user-agent` are assigned to the same cluster, specifically the cluster containing the majority of data points for that `user-agent` string. For instance, if most data points with the `user-agent` of Chrome 112 are assigned to cluster 0, I consider cluster 0 as the corresponding clus-

ter for Chrome 112. Any data with `user-agent` of Chrome 112 that is assigned to a different cluster would be considered misclustered. Using this definition, my model achieved a clustering accuracy of 99.6%.

Beyond model accuracy, I further validated the `user-agents` and their corresponding clusters to ensure they matched my expectations. This process involved cross-referencing the results with legitimate browser instances from the Candidate Fingerprint Generation stage (Section 5.6.1). I confirmed that, in over 98% of the data, the corresponding clusters matched the expected behavior. However, I noted some discrepancies with older browser versions, such as Chrome 81 and Edge 17. On closer inspection, I found that these versions had few data points, in some cases less than 100 instances. This limited dataset occasionally led to misleading results, such as Chrome 81 fingerprints showing Firefox-like feature sets. For these specific `user-agents`, making up less than 2% of the traffic, I decided to align the cluster number with the feature values of legitimate browser instances, extracted in the Candidate Fingerprint Generation stage (Section 5.6.1). This adjustment was essential for maintaining the accuracy of clustering.

The discrepancies observed with older browser versions can be attributed to the lack of sufficient descriptive data points, a challenge that is not uncommon in machine learning applications dealing with evolving technologies. To mitigate this issue in the future, a more extensive data collection process would be beneficial, akin to the detailed approach in Google Picasso's work [167]. Such a method provides a richer dataset, enabling more accurate clustering of even the lesser-known or outdated browser versions.

With prior adjustments, the distribution of `user-agents` across the selected clusters (with $k = 11$) is detailed in Table 5.3. To illustrate why the selected cluster size offers a more fitting analysis than other potential choices (3, 6, or 11), I examined

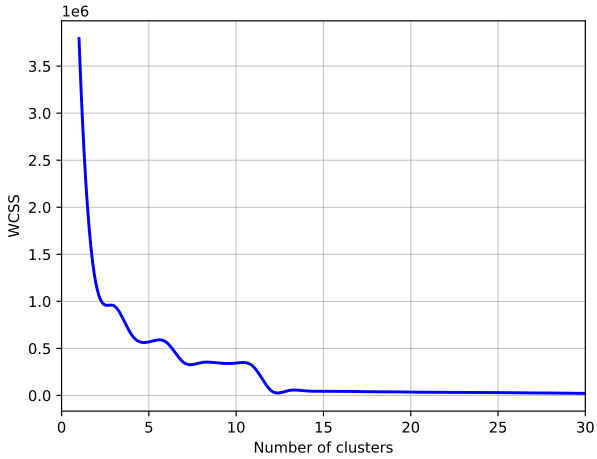


**Figure 5.3:** Elbow method to select the optimal number of clusters (WCSS vs. number of clusters).

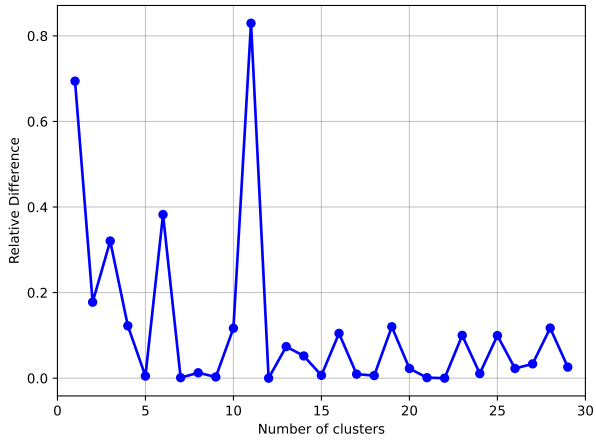


**Figure 5.4:** Relative WCSS vs. clusters: showing k=11 as an optimal number of clusters for the `kmeans` model.

the distribution of `user-agents` across a $k = 6$ cluster scenario. The outcomes highlighted in Table E.1 (Appendix-E) versus the optimal selection in Table 5.3, affirm that the prior choice more effectively differentiates between browser releases, underscoring its suitability for the analysis. I have also explored the impact of changing model parameters on the accuracy of the model in Appendix-F, which further reinforces that my choice of 28 features, 7 PCA components, and k=11 clusters lead to optimal accuracy for Browser Polygraph.

**Table 5.3:** `user-agents` assigned to clusters with k=11.

| Cluster | user-agents |
|---|---|
| 0 | Chrome 110-113, Edge 110-113 |
| 1 | Firefox 101-114 |
| 2 | Chrome 59-68, Firefox 51-91 |
| 3 | Chrome 114, Edge 114 |
| 4 | Chrome 69-89, Edge 79-89 |
| 5 | Chrome 102-109, Edge 102-109 |
| 6 | Edge 17-19, Firefox 46-50 |
| 9 | Firefox 93-100 |
| 10 | Chrome 90-101, Edge 90-101 |

### *5.6.5 Fraud Detection*

After successful training, the Browser Polygraph model can perform real-time monitoring and detection of potentially fraudulent activities in live Web traffic. The operational process of my Fraud Detection system is as follows:

When a user accesses FinOrg's website, JavaScript code collects the required feature values from their browser. These values are then analyzed by Browser Polygraph to identify the user's cluster affiliation. A key aspect of fraud detection involves comparing this predicted cluster with the cluster corresponding to the browser's declared `user-agent` (refer to Table 5.3). Any mismatch triggers the specialized risk analysis function.

The risk analysis function initially calculates the distance between a session's `user-agent` and every other `user-agent` in the predicted cluster (see Algorithm 1).

The smallest of these distances is then employed as the session's `risk factor`. I define the distance between two `user-agent` strings based on the differences in their vendors and version numbers. For two `user-agents` with different vendors, the distance is at its maximum value. However, for `user-agents` from the same vendor, I first calculate the absolute numerical difference between their version numbers and then divide this difference by 4— which I empirically selected referring to Table 5.3. This method effectively reduces the likelihood of false positives, especially when a browser instance is incorrectly assigned into a cluster of a similar vendor but a different release.

Browser Polygraph's strength lies in its efficiency and adaptability. The computationally intensive task of training the model is performed offline, ensuring that the real-time fraud detection process is possible. This not only minimizes delays during user interaction but also allows for ongoing system enhancements. In Section 5.7, I will explore the correlation between these `risk factors` and other risk metrics within FinOrg.

### *5.6.6 Drift Detection*

An essential feature of Browser Polygraph is its ongoing evaluation mechanism to monitor the performance of the clustering algorithm. This includes regularly assessing the accuracy in assigning clusters to new browser releases. I implemented a robust drift detection strategy that actively identifies shifts in data patterns or browser behavior. When such shifts are detected, indicating a potential divergence from the established clustering norms, Browser Polygraph can automatically initiate a retraining process. This retraining ensures that Browser Polygraph remains up-to-date and accurately aligned with the latest browser versions' features.

More specifically, on designated dates, the automatic drift detection module evaluates the cluster number and accuracy of Browser Polygraph in clustering new

**Algorithm 1** Algorithm to calculate the `risk factor` based on a session's `user-agent` and its predicted cluster.

```
Input: sessUserAgent, predictedCluster, userAgentTable
Output: riskFactor
riskFactor ← ∞
for each uA in userAgentTable[predictedCluster] do
    if sessionUserAgent.vendor ≠ uA.vendor then
        distance ← 20
    else
        diff ← abs(sessUserAgent.version - uA.version)
        distance ← floor(diff / 4)
    end if
    if distance < riskFactor then
        riskFactor ← distance
    end if
end for
return riskFactor
```

browser releases. These dates are strategically chosen a few days after the latest releases of Firefox, Chrome, and Edge browsers. When the cluster number of a new browser release aligns with its closest release according to Table 5.3 with an accuracy rate above 98%, it indicates no significant shift in browser behavior, and no retraining is necessary. However, a change in the cluster number (compared to its closest release from Table 5.3) or a drop in the accuracy rate below 98% signals a shift in browser behavior, prompting Browser Polygraph to automatically initiate retraining. Section 5.7.3 discusses the process using Browser Polygraph's trained model (from March to mid-July 2023) to perform drift analysis on newly collected data (from late-July to October 2023).

**Table 5.4:** Percentages of the traffic having Untrusted_IP, Untrusted_Cookie or ATO flags set in the entire traffic comparing to the batch flagged by Browser Polygraph and a randomly chosen batch.

| Category | Untrusted_IP | Untrusted_Cookie | ATO |
|---|---|---|---|
| All users | 51% | 49% | 0.43% |
| Flagged by Browser Polygraph (all) | 78% | 75% | 2% |
| Flagged by Browser Polygraph (risk factor > 1) | 93% | 89% | 3.89% |
| Flagged by Browser Polygraph (risk factor > 4) | 94% | 85% | 5.83% |
| Randomly-chosen | 48% | 53% | 0.22% |

## 5.7 Evaluation

### *5.7.1 Real-World Experiment*

To demonstrate the practical application of Browser Polygraph in detecting fraud, I collaborated with FinOrg for data collection on one of their secondary purchase portals, as outlined in Section 5.6.2. I used 4.5 months of this data to train my `kmeans` model (Section 5.6.4). The resulting clusters formed the baseline for my fraud detection method (Table 5.3).

For training purposes, I analyzed 205k logged-in user sessions from March to mid-July 2023, which included `user-agents` from 113 different browser releases. FinOrg also provided session tags from their internal systems, specifically `Untrusted_IP`, `Untrusted_Cookie`, and `ATO` (Account Take-Over), used solely for evaluation purposes. These tags identified sessions from unfamiliar IPs (note that I did not receive the old or new IP, just that FinOrg was unfamiliar with the IPs), newly-established cookies, or those with suspicious activities linked to the browser fingerprint within a 72-hours period (meaning that FinOrg believed the account was involved in an ATO).

This dataset of only logged-in users minimized interference from web crawlers and bots, offering a solid foundation for my model. Post-training, Browser Poly-

GRAPH flagged 897 sessions as suspicious during four-and-a-half-month period, considering various `risk factor` levels. This number of flagged sessions aligns with fraud data from similar studies, such as Varmedja et al. [195]'s work that used a popular credit card fraud database from Kaggle with 0.173% of reported credit card frauds in 2019 [196].

BROWSER POLYGRAPH showed heightened accuracy in identifying potential fraud, with flagged sessions having a 27% and 26% higher likelihood of being tagged as `Untrusted_IP` and `Untrusted_Cookie` flags, respectively. Sessions flagged by BROWSER POLYGRAPH were also about five times more likely to be associated with `ATO` activities (Table 5.4). The correlation between BROWSER POLYGRAPH's `risk factor` and the security tags was notable. Higher `risk factors` correlated with increased cases of `Untrusted_IP`, `Untrusted_Cookie` and `ATO` flags, underscoring BROWSER POLYGRAPH's effectiveness. Analysis indicated that lower `risk factors` were often assigned to sessions where the claimed `user-agent` shares the same vendor but only a slight variation in version number compared to other `user-agents` in the assigned cluster. Such cases might not always indicate fraud but could result from update inconsistencies or the use of certain extensions or browser configurations impacting attribute values (refer to Section 5.6.3).

In a supplementary experiment, I compare against a randomly selected 897 sessions from our dataset of 205k user sessions for further analysis. The results confirmed BROWSER POLYGRAPH 's superior performance over the random selection method, demonstrating its utility in identifying potentially fraudulent sessions with increased precision (Table 5.4).

Despite BROWSER POLYGRAPH's success in identifying sessions with `Untrusted_IP` and `Untrusted_Cookie` flags, only 2% of these 897 sessions have `ATO` flags (though this rate nearly doubled and tripled in batches with higher `risk factor`).

This lower-than-expected detection rate may be due to various factors, such as the specific nature of the data (the tagging of ATO) or the context of system integration. To gain a better visibility, the next subsection will evaluate Browser Polygraph's effectiveness in detecting sessions originating from fraud browsers.

### *5.7.2 Fraud Browsers' Detection*

I next evaluate the capability of Browser Polygraph to identify fraudulent attempts at spoofing legitimate browser fingerprints. Specifically, Browser Polygraph focuses on uncovering fraud within browsers classified under Categories **(1)** and **(2)** (in Table 5.1). It achieves this by analyzing discrepancies between the `user-agent` and the extracted fingerprints. To facilitate the evaluation, I established a private website equipped with the fingerprinting script developed during the *Real-World Data Collection* phase (Section 5.6.2). I was the only ones who accessed the website.

The primary objective is to assess Browser Polygraph's effectiveness in identifying fraudulent sessions. To this end, I selected several fraud browsers from Categories **(1)** and **(2)** (Table 5.1). These browsers were installed on a Windows machine, and multiple browser profiles were created for each, employing various `user-agents` representative of all clusters in Table 5.3. I then visited my private test website using each profile. In creating browser profiles, I tailored each one based on the browser's capacity for customization. This involved selecting a `user-agent` and their corresponding browser engines (if possible), aiming for diverse representation. Where feasible, for each cluster in Table 5.3, I generated two profiles using candidate `user-agents` from the same cluster. In cases where a fraud browser limited this capability, I opted for either randomized `user-agents` or those uniquely provided by the browser itself.

**Table 5.5:** Fraud browsers' detection capability via Browser Polygraph.

| Browser | Flagged Num | Not-Flagged Num | Avg. `risk factor` | Recall |
|---|---|---|---|---|
| GoLogin-3.3.23 | 12 | 4 | 11.66 | 75% |
| Incogniton-3.2.7.7 | 7 | 2 | 8.85 | 78% |
| Octo Browser-1.10 | 16 | 3 | 10.18 | 84% |
| Sphere-1.3 | 6 | 3 | 10.5 | 67% |

The approach in this experimental setup was designed to assess the capability of Browser Polygraph to detect sessions initiated by fraud browsers. I passed the data collected in this experiment to Browser Polygraph's Fraud Detection module (trained in Section 5.6.4) to evaluate its effectiveness in identifying fraudulent browser fingerprints. The data presented in Table 5.5 highlights Browser Polygraph's effectiveness in detecting fraudulent sessions, evidenced by a high average `risk factor` for these sessions.

While Browser Polygraph demonstrated an impressive recall rate of 79% for the first three fraud browsers in Table 5.5, the recall rate for Sphere browser was observed at 67%, this outcome is primarily influenced by two factors. The first concerns the freely available version of the Sphere software, which is significantly outdated. This limits users' ability to customize their preferred user profiles. The second factor is the predominance of user profiles in this version that adhere to older versions of Chrome (e.g., Chrome 63, 64, 65), with Sphere 1.3 emulating a fingerprint similar to Chrome 61, all falling within Cluster 2. This latter reason also accounted for the non-flagged attempts by the first three browsers.

### *5.7.3 Drift Analysis*

With the continual release of new browser versions and fraud browsers, a pertinent question arises: How long does the trained model in the Browser Polygraph

remain effective? This subsection explains the behavior of Browser Polygraph's drift detection strategy (refer to Section 5.6.6) that focuses on determining the necessity and timing for retraining Browser Polygraph via an extensive drift analysis.

Referring to the trained model of Browser Polygraph in Table 5.3, for a new browser release, such as Chrome 115, if the feature set values do not significantly differ from the closest prior release (e.g., Chrome 114), I do not expect the drift detection algorithm from Section 5.6.6 to trigger a retraining. However, if there is a drastic change in the values of the feature set, resulting in a different cluster number or a noticeable drop in accuracy, the algorithm will initiate a retraining process. To evaluate how the drift detection module accurately identifies retraining time, I used the clustering model from Section 5.6.4 that was trained from March to mid-July 2023. Subsequently, I obtained new data spanning from late-July to October 2023 from FinOrg, encompassing several releases of major browsers. With this updated dataset, I selected specific dates to evaluate the accuracy of Browser Polygraph in clustering newly released browsers. These dates were strategically chosen a few days following the latest Firefox release, with the newest Chrome and Edge versions released approximately one to two weeks earlier. My focus was on two key metrics: determining the predominant cluster number for each new browser instance and calculating the percentage of browsers assigned to this cluster for each candidate browser, as detailed in Table 5.6.

My findings indicate that new releases of Chrome, Firefox, and Edge were correctly clustered with an accuracy exceeding 99% for approximately three and half months. However, starting from late October, a shift in the assigned cluster number for Firefox 119 was observed, alongside a decline in the clustering accuracy for Chrome 119. This served as an indicator for retraining the clustering model. Further examination of the feature values for Firefox 119 confirmed substantial changes

**Table 5.6:** Drift Analysis of Browser Polygraph for data collected from late-July to October 2023.

| Browser | Date | Cluster | Accuracy |
|---|---|---|---|
| Chrome 115 | | 3 | 99.45 |
| Firefox 115 | 07/25 | 1 | 99.3 |
| Edge 115 | | 3 | 100 |
| Chrome 116 | | 3 | 99.6 |
| Firefox 116 | 08/25 | 1 | 99.99 |
| Edge 116 | | 3 | 99.88 |
| Chrome 117 | | 3 | 99.25 |
| Firefox 117 | 09/25 | 1 | 99.81 |
| Edge 117 | | 3 | 99.94 |
| Chrome 118 | | 3 | 99.65 |
| Firefox 118 | 10/23 | 1 | 99.46 |
| Edge 118 | | 3 | 99.91 |
| Chrome 119 | | 3 | 97.22 |
| Firefox 119 | 10/31 | 10 | 98.57 |
| Edge 119 | | 3 | 99.84 |

in the `Element` prototype's implementation compared to its predecessor, providing additional evidence supporting the hypothesis for the retraining time.

It is worth mentioning that the observations in this analysis align with the retraining signal from the drift detection module, which was triggered in October. Although in this analysis, the trained model in Browser Polygraph remained accurate for three and a half months, this duration may vary depending on the parameters of the model and the extent of changes in new browser releases.

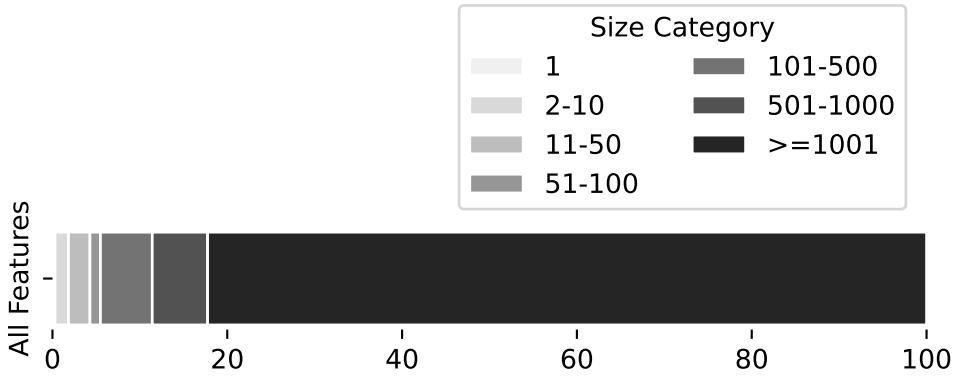


**Figure 5.5:** Percentage of fingerprints in anonymity sets.

### *5.7.4 Privacy Analysis*

To evaluate the privacy-preserving nature of Browser Polygraph's feature set, I analyzed the anonymity sets and entropy of features within the dataset, which comprises 205k fingerprints. The former assesses how diverse the fingerprints are, while the latter provides insight into the diversity of individual features. Figure 5.5 displays the percentage of user fingerprints across anonymity sets of varying sizes. Notably, only 0.3% of our fingerprints are unique, a negligible rate compared to a browser fingerprinting study that reported 33.6% unique fingerprints [68]. Furthermore, while the same study found 8% of fingerprints in anonymity sets larger than 50, my study shows a significantly higher rate of 95.6%, making it nearly impossible to track individual users.

I also investigated Shannon entropy and normalized entropy for the collected attributes (`user-agent` and the integer outputs) as reported in Table 5.7 (sorted by normalized entropy). The `user-agent` was the most diverse feature in our dataset, yielding a normalized entropy value of 0.58. This value is the same as the normalized entropy of 0.58 for the `user-agent` string for AmIUnique study [68]. This similarity emphasizes that my data collection method does not increase user identifiability beyond what is already possible with the `user-agent` string alone, thus maintaining

**Table 5.7:** Entropy information of selected features used in Browser Polygraph, sorted by normalized entropy.

| Features | Entropy | Normalized Entropy |
|---|---|---|
| `user-agent` | 5.97 | 0.58 |
| `Object.getOwnPropertyNames(Element.prototype).length` | 2.51 | 0.47 |
| `Object.getOwnPropertyNames(SVGElement.prototype).length` | 2.33 | 0.43 |
| `Object.getOwnPropertyNames(Document.prototype).length` | 2.17 | 0.42 |
| `Object.getOwnPropertyNames(IntersectionObserver.prototype).length` | 1.33 | 0.37 |
| `HTMLVideoElement.prototype.hasOwnProperty('webkitDisplayingFullscreen')` | 0.58 | 0.37 |
| `Object.getOwnPropertyNames(CSSRule.prototype).length` | 0.56 | 0.35 |
| `Object.getOwnPropertyNames(StaticRange.prototype).length` | 0.58 | 0.29 |

user anonymity.

### *5.7.5 Performance Analysis*

To validate the study's adherence to the performance benchmarks set in Section 5.3, I measured Browser Polygraph's response time within the experimental setup described in Section 5.7.2. Through a series of tests mirroring those applied to fine-grained fingerprinting solutions, Browser Polygraph demonstrated an average response time of 6ms. Additionally, the storage requirement for our fingerprints was only 1KB, which significantly surpasses the efficiency of the solutions in Table 5.2.

In Appendix G, I compare the performance of Browser Polygraph's coarse-grained fingerprints with two fine-grained techniques for browser clustering (FingerprintJS and ClientJS) using synthetic data. The results demonstrate that Browser Polygraph performs better in clustering for this specific experiment. This is because Browser Polygraph's coarse-grained fingerprinting is fundamentally designed for detecting lying browsers, making it more efficient for browser clustering purposes.

## 5.8 Discussion

The most trivial step for configuring a fraud browser is setting the `user-agent` value—alignment of this value to the victim's actual `user-agent` however does not

suffice for the adversary to fully spoof a victim's browser. A fraud browser could still show inconsistencies in its other configurations, and the primary goal of Browser Polygraph is to identify such inconsistencies. The proposed solution, however, has some limitations:

**Bypassing Browser Polygraph**. Lin et al. [169] showed how stealing and spoofing actual user fingerprints could bypass the authentication process of critical websites. Their work can bypass Browser Polygraph in theory and it is essentially the problem of many JavaScript-based detection techniques; however, their current solution was not spoofing the output of `Object.getOwnPropertyNames` function for our feature set values, and it needs an additional update to cover it. While these cat-and-mouse games can be played indefinitely, the goal of Browser Polygraph is *not* to identify new fingerprinting techniques but rather to introduce the concept of coarse-grained fingerprints that can be used in a production website with strict limitations on fingerprint extraction time and data size.

**user-agent randomization.** `user-agent` randomization is a common anti-finger printing strategy, potentially increasing false positives in Browser Polygraph. However, I do not recommend its application due to potential complications, such as triggering bot detection mechanisms [197].

**Verification of new browsers.** Browser Polygraph's current focus is on popular desktop browsers and those with analogous `user-agent` strings, due to the prevalence of fraud browsers offering more tools to spoof desktop rather than mobile browsers. Currently, it does not encompass mobile browsers or those with unique engines and `user-agent` strings. However, extending its verification capabilities to these areas is possible. This expansion would involve gathering baseline data for these new browsers, as detailed in Section 5.6.1. With baseline values and a thorough real-world dataset for any new browser type, adapting Browser Polygraph for

detection becomes straightforward.

**Deployment scope.** The current deployment of Browser Polygraph leverages a coarse-grained browser fingerprinting technique, primarily designed as a lightweight fraud detection mechanism in high-volume traffic scenarios. It is crucial to acknowledge that Browser Polygraph is not configured to detect fraudulent sessions conducted by sophisticated attackers, who may completely spoof the fingerprinting environment or use browsers classified under category 3 in Table 5.1. To counter these advanced spoofing efforts, more complex detection methods are necessary, perhaps binary analysis of each specific software, which can unearth unique software-specific fingerprintable attributes.

In my observations, for instance, AntBrowser [175] (similar to many other browsers) includes an `ANTBROWSER` object in its namespace and `antBrowser`-prefixed attributes on the `window` object, significantly increasing its fingerprintability. This is similar to the observation that Nikiforakis et al. [49] made where browser extensions that allow browser spoofing ironically make browsers *more* fingerprintable. While these attributes were initially identified through manual analysis, future work could automate this process, employing techniques such as fuzzing or binary analysis. Such advancements can enable identifying software-specific attributes and directly target the fingerprinting of fraud browsers.

**Scale of the database.** The current implementation of Browser Polygraph employs a dataset whose size does not yet pose significant storage challenges. However, should the dataset grow to an unmanageably large scale in the future, thereby impacting training efficiency, a viable solution would be the adoption of Stratified Sampling [198]. This approach is particularly effective for managing large datasets while ensuring the representativeness of diverse data segments. Using Stratified Sampling, we can efficiently maintain a representative sample of browser features, even

from less popular browser instances, ensuring that our dataset remains manageable and comprehensive.

**Manual efforts.** While Browser Polygraph automates the majority of operations, such as candidate fingerprint generation, fraud detection, and drift detection, manual inspections were necessary during the data pre-processing stage to address inconsistencies in *deviation-based* features. As discussed in Section 5.6.3, I manually assessed the impact of browser configurations on these features and determined whether to retain or discard specific features. Similarly, I manually installed popular extensions on Chrome to observe their influence on feature values. In case the selected features become ineffective over time—meaning that retraining Browser Polygraph does not achieve the required accuracy—or if there are drastic updates to browser implementations necessitating another round of pre-processing, we need to automate the manual inspections involved in this phase.

## 5.9 Related Work

This section discusses prior research relevant to the current study on browser fingerprinting and fraud detection:

**Risks of fingerprint spoofing.** Liu et al. [168] explore how spoofing browser fingerprints can affect advertising content, while Lin et al. [169] demonstrate the potential for spoofed fingerprints to bypass websites authentication.

**Nature of misuse.** In the context of misuse, Kawase et al. [199] explore ATO within an online vehicle marketplace, presenting detection and prevention methods. Campobasso et al. [170] evaluate illegal sales of user fingerprints and tools for impersonating browser identities.

**Advancements in browser fingerprinting.** Nikiforakis et al. [49] focus on extracting browser family and version information using a set of techniques, such as ordering

of methods and properties and detection of vendor-specific methods. Akhavani et al. [189] provide a complete list of JavaScript features for browser differentiation, but it might need revision via new browser releases. Schwarz et al. [13] propose automated techniques for inferring browser environments.

**Browser fingerprinting in the wild.** Durey et al. [165] and Senol et al. [166] explore the adoption of browser fingerprinting in real-world scenarios, with the latter extending the analysis to its effectiveness in fraud detection, particularly in safeguarding against unauthorized access and enhancing account security. Wu et al. [200] conduct a large-scale study of adversarial fingerprints—intentionally crafted by attackers to evade detection—and benign browser fingerprints, highlighting the role of browser fingerprinting in bot and fraud detection

**Browser authentication techniques.** Bursztein et al. [167] use graphical fingerprints for browser verification, and Laperdrix et al. [201] employ canvas challenges for authentication. Google offers Private State Tokens [202] as part of the Privacy Sandbox [202] project, providing a privacy-preserving method to validate users and potentially replacing third-party cookies. There are also proposals such as Web Environment Integrity API [203] which advocate for browser verification through attestation tokens. Additionally, Cloudflare's Privacy Pass [204] uses cryptographic tokens to authenticate browsers, allowing websites to verify legitimate traffic without tracking individual users.

**Fraud browsers' detection.** Traditionally, security researchers and industry professionals have relied on manual analysis to identify suspicious parameters in requests generated by fraud browsers, as highlighted by Azad et al. [172]. These mitigation strategies, often based on regular expressions, face obsolescence with each new release of fraud browsers. Compounding this issue, fraud browser vendors frequently update their software to align with new browser versions, thereby diminishing the effective-

ness of existing fraud detection patterns. An automated prototype that adapts to ongoing browser updates is, therefore, a critical need in the industry.

## 5.10 Ethics

This work does not raise any ethical issues since we ensure that all data collected is in accordance with FinOrg's data collection policy, which considers fingerprinting browsers for fraud detection purposes as a legitimate and legal use case. It's important to note, as discussed in Section 5.6.2, that the fingerprints collected are only integer outputs, used exclusively for fraud detection purposes.

Furthermore, the collected coarse-grained fingerprints lack the granularity to identify specific user browsers, thereby preserving privacy. For instance, a sample from our dataset in March revealed that approximately 11% of sessions in a day shared a `user-agent` associated with a specific Chrome version on a Windows 10 machine, with 96% displaying identical coarse-grained fingerprints. This precludes individual user identification. To provide additional proof, I analyzed the anonymity sets and feature entropy in Section 5.7.4. The analysis found that only 0.3% of the fingerprints in our dataset were unique, which is negligible compared to the 30% unique fingerprints reported in previous studies [68]. This reinforces that the proposed approach does not enable tracking and is solely for fraud detection.

Additionally, examining the entropy of collected features (Table 5.7), the most diverse collected feature was the `user-agent`, indicating that the collected features do not provide additional information beyond what is already exposed by the `user-agent` string.

Moreover, the session identifiers shared with us by FinOrg were completely opaque and randomized, ensuring no possibility of tracing them back to individual users, thereby maintaining user privacy and data confidentiality. Importantly, no Personally

Identifiable Information (PII) was captured; at no time did the research involve access to the users' identity or their IP addresses.

## 5.11 Conclusion

In conclusion, I presented Browser Polygraph, a novel approach to addressing the challenges of detecting fraud browsers in high-traffic environments. Browser Polygraph successfully merges the need for detailed browser data with the scalability required for real-time fraud detection. By introducing coarse-grained fingerprints, I offer a new direction for browser fingerprinting that meets the performance constraints of financial institutions like FinOrg.

This research marks a significant step forward in safeguarding user accounts by combating browser fingerprint spoofing, particularly those driven by fraud browsers. The insights and methodologies presented in this dissertation lay the groundwork for future advancements in the field, particularly in refining fingerprinting techniques to balance performance and privacy. By focusing on safeguarding user accounts and real-world fraud detection, this dissertation provides both theoretical insights and practical solutions, contributing to the evolution of fingerprinting-based fraud detection and enhancing the overall security of users in the modern Web environments.

# Chapter 6

# CONCLUSION

This dissertation presents a comprehensive approach to addressing critical Web security and privacy challenges that users face in the modern Web ecosystem. By focusing on content injection vulnerabilities, DOM-based extension fingerprinting, and fraud browser detection, the solutions proposed in this work significantly contribute to safeguarding users' Web interactions. The dissertation showcases innovative methods that not only strengthen protection but also offer practical tools suitable for real-world deployment.

First, Context-Auditor introduced a context-aware approach to mitigating content injection vulnerabilities. By addressing unintended context switches, it tackled the root cause of these vulnerabilities, providing a robust and adaptable defense for web applications. This foundation not only improves the understanding of these vulnerabilities but also enables a flexible defense mechanism capable of evolving with future threats.

Next, I demonstrated the effectiveness of DOM Reality Shifting in Simulacrum for preventing state-of-the-art DOM-based extension fingerprinting. The results showed strong protection against an advanced browser extension fingerprinting method. Simulacrum proved to be a promising solution, and I recommend integrating this mechanism natively into web browsers to enhance privacy protection and usability.

Finally, Browser Polygraph introduced a lightweight, privacy-preserving approach to detecting fraud browsers, offering organizations an efficient method to combat ATO fraud. Its innovative use of coarse-grained fingerprints provided a practical and effective way for organizations to secure user accounts against fraudulent activi-

ties without compromising users' privacy or employing invasive techniques.

The insights and methodologies behind these contributions will assist security researchers, professionals, and organizations in addressing emerging and ongoing threats, while also paving the way for future advancements in Web security and privacy.

## 6.1 Future Research Directions

The research presented in this dissertation opens up several exciting avenues for future work in Web security. By building on the foundational concepts introduced, there are opportunities to develop more advanced tools and approaches that can better safeguard users' Web interactions. Below are key areas that offer potential for further exploration:

**Building an advanced context-aware scanner.** A promising direction for future research is the development of a sophisticated vulnerability scanner that leverages the intelligence of Context-Auditor's two-way pushdown automata. Having identified injection points, this advanced scanner would then craft more intelligent and adaptive payloads. Based on its knowledge of known exploits, the system's current state and the content of the stack, it could tailor its attacks to be more precise, making it more effective at uncovering complex and previously undetected injection vulnerabilities.

**Handling transformed user input.** The current implementation of Context-Auditor does not address input transformations, however Context-Auditor has the potential to work alongside various user-input detection modules. A promising future research direction would involve designing a more advanced content injection detection tool that incorporates a lightweight yet detailed module for detecting transformed user inputs. This enhancement would significantly improve the tool's accuracy

in identifying a broader range of content injection vulnerabilities.

**XS-leak mitigation.** One of the challenges for CONTEXT-AUDITOR is the emergence of XS-leak vulnerabilities, where attackers can craft user input in such a way that they can infer sensitive information from a web application's response. A compelling future research direction would be to develop detection and mitigation techniques specifically designed to prevent XS-leak vulnerabilities. For instance, leveraging taint-tracking approaches could help distinguish between legitimate content and offsets triggered by attacker-crafted input, thereby minimizing the risk of XS-leak attacks.

**Extending the DOM Reality Shifting concept to other problems.** The DOM Reality Shifting concept implemented in Simulacrum effectively conceals extension modifications from malicious web pages. However, this concept has broader potential beyond extension fingerprinting. A future research direction could explore extending DOM Reality Shifting to address other Web security and privacy challenges, such as preventing other forms of browser-based tracking or mitigating web-based attacks that exploit the DOM. This expansion could significantly enhance the scope of defenses for user privacy and Web security.

**Integrating DOM Reality Shifting countermeasure into native browsers.** The current implementation of Simulacrum works effectively and has minimized breakage to a great extent, but some issues persist, especially with highly dynamic or complex web applications. A future research direction could involve integrating DOM Reality Shifting countermeasure into native browser functionality, while also exploring more adaptive solutions to further minimize breakage. Embedding this defense directly into browsers would enhance both security and usability, offering robust protection against extension fingerprinting without significantly impacting web application performance.

**Usage of coarse-grained browser fingerprints for bot detection.** Although bot detection was not explored in this dissertation, the methodology behind detecting fraudulent browsers using coarse-grained fingerprints shares similarities with bot detection techniques. Verifying whether a browser session is misrepresenting its user-agent closely aligns with methods used in bot detection systems. As bots become more sophisticated, mimicking legitimate browser behavior, applying coarse-grained browser fingerprinting could offer a lightweight and effective approach for detecting bots. Future research could focus on adapting this method to large-scale bot detection systems, providing a privacy-preserving and efficient alternative to traditional detection techniques.

**Enhancing the detection capabilities of fraud browsers.** While Browser Polygraph successfully detects a subset of fraud browsers by identifying discrepancies between the claimed user agent and the actual browser features, there is potential to further extend this research and identify a broader range of fraud browsers categories. As an additional signal for risk-based authentication systems, using lightweight but precise fingerprints for some other popular fraud browsers could specifically aid in detecting and preventing such threats. However, these fingerprints would need to be regularly updated to keep pace with attackers, ensuring that the detection mechanisms remain effective against evolving fraud techniques.

## REFERENCES


[1] W3C, "Same origin policy," https://www.w3.org/Security/wiki/Same_Origin_Policy.

[2] W3C, "Content security policy level 3," https://www.w3.org/TR/CSP3, 2021.

[3] PortSwigger, "Cors enabled," https://www.w3.org/wiki/CORS_Enabled.

[4] PortSwigger, "Cross-site scripting (xss)," https://portswigger.net/web-security/cross-site-scripting, 2021.

[5] Wired, "How hackers slipped by british airways' defenses," https://www.wired.com/story/british-airways-hack-details, 2018.

[6] CBSNews, "Fortnite security flaw exposed millions of users to being hacked," https://www.cbsnews.com/news/fortnite-security-flaw-exposed-millions-of-users-to-being-hacked, 2019.

[7] P. Eckersley, "How unique is your web browser?" in *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2010, pp. 1–18.

[8] K. Boda, Á. M. Földes, G. G. Gulyás, and S. Imre, "User tracking on the web via cross-browser fingerprinting," in *Information Security Technology for Applications: 16th Nordic Conference on Secure IT Systems, NordSec 2011, Tallinn, Estonia, October 26-28, 2011, Revised Selected Papers 16*. Springer, 2012, pp. 31–46.

[9] G. Acar, M. Juarez, N. Nikiforakis, C. Diaz, S. Gürses, F. Piessens, and B. Preneel, "Fpdetective: dusting the web for fingerprinters," in *CCS*, 2013.

[10] S. Englehardt and A. Narayanan, "Online tracking: A 1-million-site measurement and analysis," in *CCS*, 2016.

[11] M. Mulazzani, P. Reschl, M. Huber, M. Leithner, S. Schrittwieser, E. Weippl, and F. Wien, "Fast and reliable browser identification with javascript engine fingerprinting," in *W2SP*. Citeseer, 2013.

[12] K. Mowery, D. Bogenreif, S. Yilek, and H. Shacham, "Fingerprinting information in javascript implementations," *Proceedings of W2SP*, vol. 2, no. 11, 2011.

[13] M. Schwarz, F. Lackner, and D. Gruss, "Javascript template attacks: Automatically inferring host information for targeted exploits." in *NDSS*, 2019.

[14] MDN, "Web apis," https://developer.mozilla.org/en-US/docs/Web/API.

[15] C. Prophitt, "Nefarious linkedin," https://github.com/dandrews/nefarious-linkedin, 2017.

[16] S. Karami, P. Ilia, K. Solomos, and J. Polakis, "Carnus: Exploring the privacy threats of browser extension fingerprinting," in *NDSS*, 2020.

[17] A. Sjösten, S. Van Acker, and A. Sabelfeld, "Discovering browser extensions via web accessible resources," in *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, 2017.

[18] P. Laperdrix, O. Starov, Q. Chen, A. Kapravelos, and N. Nikiforakis, "Fingerprinting in style: Detecting browser extensions via injected style sheets," in *USENIX Security*, 2021.

[19] E. Trickel, O. Starov, A. Kapravelos, N. Nikiforakis, and A. Doupé, "Everyone is different: client-side diversification for defending against extension fingerprinting," in *USENIX Security*, 2019.

[20] S. Karami, F. Kalantari, M. Zaeifi, X. J. Maso, E. Trickel, P. Ilia, Y. Shoshitaishvili, A. Doupé, and J. Polakis, "Unleash the simulacrum: Shifting browser realities for robust extension-fingerprinting prevention," in *USENIX Security*, 2022.

[21] FBI, "2022 internet crime report," https://www.ic3.gov/Media/PDF/AnnualReport/2022_IC3Report.pdf, 2023.

[22] Rohan Goswami, "Cybercrime marketplace genesis market shut by fbi, international law enforcement," https://www.cnbc.com/2023/04/04/genesis-market-shut-by-law-enforcement-in-cybercrime-operation.html, apr 2023.

[23] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, "mxss attacks: Attacking well-secured web-applications by using innerhtml mutations," in *CCS*, 2013.

[24] M. Steffens, C. Rossow, M. Johns, and B. Stock, "Don't trust the locals: Investigating the prevalence of persistent client-side cross-site scripting in the wild." 2019.

[25] D. Balzarotti, M. Cova, V. Felmetsger, N. Jovanovic, E. Kirda, C. Kruegel, and G. Vigna, "Saner: Composing static and dynamic analysis to validate sanitization in web applications," in *IEEE Symposium on Security and Privacy*. IEEE, 2008, pp. 387–401.

[26] P. Bisht and V. Venkatakrishnan, "Xss-guard: precise dynamic prevention of cross-site scripting attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2008, pp. 23–43.

[27] B. Livshits and S. Chong, "Towards fully automatic placement of security sanitizers and declassifiers," *ACM Sigplan Notices*, vol. 48, no. 1, pp. 385–398, 2013.

[28] A. Doupé, W. Cui, M. H. Jakubowski, M. Peinado, C. Kruegel, and G. Vigna, "dedacota: toward preventing server-side xss via automatic code and data separation," in *CCS*, 2013.

[29] M. Musch, M. Steffens, S. Roth, B. Stock, and M. Johns, "Scriptprotect: mitigating unsafe third-party javascript practices," in *CCS*, 2019.

[30] M. Heiderich, C. Späth, and J. Schwenk, "Dompurify: Client-side protection against xss and markup injection," in *European Symposium on Research in Computer Security*. Springer, 2017, pp. 116–134.

[31] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, "Csp is dead, long live csp! on the insecurity of whitelists and the future of content security policy," in *CCS*, 2016.

[32] S. Calzavara, A. Rabitti, and M. Bugliesi, "Content Security Problems?" in *CCS*, 2016, pp. 1365–1375.

[33] W3C, "Trusted types," https://w3c.github.io/webappsec-trusted-types/dist/spec/, 2021.

[34] P. Wang, B. A. Guemundsson, and K. Kotowicz, "Adopting trusted types in productionweb frameworks to prevent dom-based cross-site scripting: A case study," in *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE Computer Society, 2021, pp. 60–73.

[35] A. Doupé, M. Cova, and G. Vigna, "Why johnny can't pentest: An analysis of black-box web vulnerability scanners," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2010, pp. 111–131.

[36] S. Lekies, B. Stock, and M. Johns, "25 million flows later: large-scale detection of dom-based xss," in *CCS*, 2013.

[37] A. Alhuzali, R. Gjomemo, B. Eshete, and V. Venkatakrishnan, "Navex: Precise and scalable exploit generation for dynamic web applications," in *USENIX Security*, 2018.

[38] Firefox, "Noscript," https://noscript.net.

[39] Chrome, "NoScript Chrome Extension," https://chrome.google.com/webstore/detail/noscript/doojmbjmlfjjnbmnoijecmcbfeoakpjm?hl=en, 2021.

[40] D. Bates, A. Barth, and C. Jackson, "Regular expressions considered harmful in client-side xss filters," in *WWW*, 2010.

[41] W. Melicher, A. Das, M. Sharif, L. Bauer, and L. Jia, "Riding out domsday: Towards detecting and preventing dom cross-site scripting," in *NDSS*, 2018.

[42] "Open Source Web Application Firewall," http://www.modsecurity.org.

[43] P. Saxena, D. Molnar, and B. Livshits, "Scriptgard: automatic context-sensitive sanitization for large-scale legacy web applications," in *Proceedings of the 18th ACM conference on Computer and communications security*, 2011, pp. 601–614.

[44] B. Stock, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, "Precise client-side protection against dom-based cross-site scripting," in *USENIX Security*, 2014.

[45] G. Acar, C. Eubank, S. Englehardt, M. Juarez, A. Narayanan, and C. Diaz, "The web never forgets: Persistent tracking mechanisms in the wild," in *CCS*, 2014.

[46] K. Mowery and H. Shacham, "Pixel perfect: Fingerprinting canvas in HTML5," in *Proceedings of W2SP*, 2012.

[47] Y. Cao, S. Li, and E. Wijmans, "(cross-) browser fingerprinting via os and hardware level features," in *Proceedings 2017 Network and Distributed System Security Symposium*. Internet Society, 2017.

[48] N. Takei, T. Saito, K. Takasu, and T. Yamada, "Web browser fingerprinting using only cascading style sheets," in *2015 10th International Conference on Broadband and Wireless Computing, Communication and Applications (BWCCA)*. IEEE, 2015.

[49] N. Nikiforakis, A. Kapravelos, W. Joosen, C. Kruegel, F. Piessens, and G. Vigna, "Cookieless monster: Exploring the ecosystem of web-based device fingerprinting," in *IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 541–555.

[50] I. Sanchez-Rola, I. Santos, and D. Balzarotti, "Clock around the clock: Time-based device fingerprinting," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1502–1514.

[51] P. Laperdrix, W. Rudametkin, and B. Baudry, "Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints," in *IEEE Symposium on Security and Privacy*, 2016.

[52] O. Starov and N. Nikiforakis, "Xhound: Quantifying the fingerprintability of browser extensions," in *IEEE Symposium on Security and Privacy*. IEEE, 2017, pp. 941–956.

[53] X. Lin, P. Ilia, and J. Polakis, "Fill in the blanks: Empirical analysis of the privacy threats of browser form autofill," in *CCS*, 2020.

[54] P. Chinprutthiwong, R. Vardhan, G. Yang, Y. Zhang, and G. Gu, "The service worker hiding in your browser: The next web attack target?" in *RAID*, 2021.

[55] S. Karami, P. Ilia, and J. Polakis, "Awakening the web's sleeper agents: Misusing service workers for privacy leakage," in *NDSS*, 2021.

[56] K. Subramani, X. Yuan, O. Setayeshfar, P. Vadrevu, K. H. Lee, and R. Perdisci, "When push comes to ads: Measuring the rise of (malicious) push advertising," in *IMC*, 2020.

[57] B. Kondracki, A. Aliyeva, M. Egele, J. Polakis, and N. Nikiforakis, "Meddling middlemen: Empirical analysis of the risks of data-saving mobile browsers," in *IEEE Symposium on Security and Privacy*, 2020.

[58] M. Diamantaris, S. Moustakas, L. Sun, S. Ioannidis, and J. Polakis, "This sneaky piggy went to the android ad market: Misusing mobile sensors for stealthy data exfiltration," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1065–1081.

[59] J. Wilander, "Webkit - intelligent tracking prevention (itp)," https://webkit.org/blog/9521/intelligent-tracking-prevention-2-3/, 2019.

[60] S. Englehardt and A. Edelstein, "Firefox 85 cracks down on supercookies," https://blog.mozilla.org/security/2021/01/26/supercookie-protections/, 2021.

[61] D. Temkin, "Google Ads - Charting a course towards a more privacy-first web," https://blog.google/products/ads-commerce/a-more-privacy-first-web/, 2021.

[62] P. Syverson and M. Traudt, "Hsts supports targeted surveillance," in *8th USENIX Workshop on Free and Open Communications on the Internet (FOCI '18)*, 2018.

[63] K. Solomos, J. Kristoff, C. Kanich, and J. Polakis, "Tales of favicons and caches: Persistent tracking in modern browsers," in *NDSS*, 2021.

[64] Y. Dimova, G. Acar, L. Olejnik, W. Joosen, and T. V. Goethem, "The cname of the game: Large-scale analysis of dns-based tracking evasion," *PETS*, 2021.

[65] A. Klein and B. Pinkas, "Dns cache-based user tracking." in *NDSS*, 2019.

[66] A. Aliyeva and M. Egele, "Oversharing is not caring: How cname cloaking can expose your session cookies," in *Asia CCS*. Association for Computing Machinery, 2021.

[67] Y. Cao, S. Li, and E. Wijmans, "(cross-)browser fingerprinting via OS and hardware level features," in *NDSS*, 2017.

[68] A. Gómez-Boix, P. Laperdrix, and B. Baudry, "Hiding in the crowd: an analysis of the effectiveness of browser fingerprinting at large scale," in *WWW*, 2018.

[69] A. Vastel, P. Laperdrix, W. Rudametkin, and R. Rouvoy, "Fp-stalker: Tracking browser fingerprint evolutions," in *IEEE Symposium on Security and Privacy*, 2018.

[70] A. Das, G. Acar, N. Borisov, and A. Pradeep, "The web's sixth sense: A study of scripts accessing smartphone sensors," in *CCS*, 2018.

[71] V. Mishra, P. Laperdrix, A. Vastel, W. Rudametkin, R. Rouvoy, and M. Lopatka, "Don't count me out: On the relevance of ip address in the tracking ecosystem," in *Proceedings of The Web Conference*, 2020.

[72] P. Laperdrix, N. Bielova, B. Baudry, and G. Avoine, "Browser fingerprinting: A survey," *ACM Transactions on the Web (TWEB)*, 2020.

[73] "Simulacrum Code and Data Repository," https://github.com/SimulacrumExtension/Simulacrum.

[74] MDN, "MDN Web API," https://developer.mozilla.org/en-US/docs/Web/API.

[75] Z. Weinberg, E. Y. Chen, P. R. Jayaraman, and C. Jackson, "I still know what you visited last summer: Leaking browsing history via user interaction and side channel attacks," in *IEEE Symposium on Security and Privacy*, 2011.

[76] L. F. DeKoven, S. Savage, G. M. Voelker, and N. Leontiadis, "Malicious browser extensions at scale: Bridging the observability gap between web site and browser," in *10th USENIX Workshop on Cyber Security Experimentation and Test (CSET 17)*, 2017.

[77] D. F. Somé, "Empoweb: Empowering web applications with browser extensions," in *IEEE Symposium on Security and Privacy*, 2019.

[78] R. Perrotta and F. Hao, "Botnet in the browser: Understanding threats caused by malicious browser extensions," *IEEE security & Privacy*, 2018.

[79] MDN, "MDN Object Model," https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Details_of_the_Object_Model.

[80] W. W. W. Consortium, "DOM Living Standard," https://dom.spec.whatwg.org/.

[81] P. Snyder, C. Taylor, and C. Kanich, "Most websites don't need to vibrate: A cost-benefit approach to improving browser security," in *CCS*, 2017.

[82] C. W. G. World Wide Web Consortium, "Resize Observer," https://drafts.csswg.org/resize-observer.

[83] W. W. W. Consortium, "Intersection Observer," https://w3c.github.io/IntersectionObserver/.

[84] "EasyList - Overview," https://easylist.to/.

[85] P. Picazo-Sanchez, J. Tapiador, and G. Schneider, "After you, please: browser extensions order attacks and countermeasures," *International Journal of Information Security*, 2019.

[86] P. Snyder, "Issue 793217: document_start hook on child frames should fire before control is returned to the parent frame," https://bugs.chromium.org/p/chromium/issues/detail?id=793217.

[87] "Privacy badger," https://chrome.google.com/webstore/detail/privacy-badger/pkehgijcmpdhfbdbbnkijodmdjhbjlgp.

[88] M. Schwarz, M. Lipp, and D. Gruss, "Javascript zero: Real javascript and zero side-channel attacks." in *NDSS*, 2018.

[89] Bugcrowd, "2022 Bugcrowd Priority One Report," 2022. [Online]. Available: https://www.bugcrowd.com/resources/reports/priority-one-report/

[90] Hackerone, "Hacker-powered security report: Industry insights '21," https://www.hackerone.com/resources/reporting/hacker-powered-security-report-industry-insights-21, 2021.

[91] US-CERT, "Cert advisory ca-2000-02 malicious html tags embedded in client web requests," https://resources.sei.cmu.edu/asset_files/WhitePaper/2000_019_001_496188.pdf, 2000.

[92] OWASP, "OWASP Top 10," 2021. [Online]. Available: https://owasp.org/Top10/

[93] MITRE, "2021 cwe top 25 most dangerous software weaknesses," http://cwe.mitre.org/top25/archive/2021/2021_cwe_top25.html, 2021.

[94] Y. Xie and A. Aiken, "Static detection of security vulnerabilities in scripting languages." in *USENIX Security*, 2006.

[95] N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A static analysis tool for detecting web application vulnerabilities," in *IEEE Symposium on Security and Privacy*, 2006.

[96] Y.-W. Huang, F. Yu, C. Hang, C.-H. Tsai, D.-T. Lee, and S.-Y. Kuo, "Securing web application code by static analysis and runtime protection," in *WWW*, 2004.

[97] P. Vogt, F. Nentwich, N. Jovanovic, E. Kirda, C. Kruegel, and G. Vigna, "Cross site scripting prevention with dynamic data tainting and static analysis," in *NDSS*, 2007.

[98] M. Heiderich, M. Niemietz, F. Schuster, T. Holz, and J. Schwenk, "Scriptless attacks: stealing the pie without touching the sill," in *CCS*, 2012.

[99] M. Johns and C. Beyerlein, "Smask: preventing injection attacks in web applications by approximating automatic data/code separation," in *Proceedings of the 2007 ACM symposium on Applied computing*. ACM, 2007.

[100] S. Lekies, K. Kotowicz, S. Groß, E. A. Vela Nava, and M. Johns, "Code-reuse attacks for the web: Breaking cross-site scripting mitigations via script gadgets," in *CCS*, 2017.

[101] M. Johns and S. Pfistner, "End-to-end taint tracking for detection and mitigation of injection vulnerabilities in web applications," Nov. 13 2018, uS Patent 10,129,285.

[102] S. Son, K. S. McKinley, and V. Shmatikov, "Diglossia: detecting code injection attacks with precision and efficiency," in *CCS*, 2013.

[103] "Context-auditor's code and data," https://github.com/Context-Auditor/Context-Auditor, 2022.

[104] portswigger, "Hunting nonce-based csp bypasses with dynamic analysis," https://portswigger.net/research/hunting-nonce-based-csp-bypasses-with-dynamic-analysis, 2021.

[105] R. Roemer, E. Buchanan, H. Shacham, and S. Savage, "Return-oriented programming: Systems, languages, and applications," *ACM Transactions on Information and System Security (TISSEC)*, 2012.

[106] "OWASP ModSecurity Core Rule Set ," https://owasp.org/www-project-modsecurity-core-rule-set/, 2022.

[107] A. S. Buyukkayhan, C. Gemicioglu, T. Lauinger, A. Oprea, W. Robertson, and E. Kirda, "What's in an exploit? an empirical analysis of reflected server xss exploitation techniques," in *RAID*, 2020.

[108] W. Xu, S. Bhatkar, and R. Sekar, "Taint-enhanced policy enforcement: A practical approach to defeat a wide range of attacks," in *USENIX Security*, 2006.

[109] M. Luo, P. Laperdrix, N. Honarmand, and N. Nikiforakis, "Time does not heal all wounds: A longitudinal analysis of security-mechanism support in mobile browsers," in *NDSS*, 2019.

[110] "HTML Living Standard," https://html.spec.whatwg.org/, 2022.

[111] "ECMAScript 2019," https://tc39.github.io/ecma262/, 2019.

[112] "CSS Syntax Module Level 3," https://www.w3.org/TR/css-syntax-3/, 2021.

[113] "Bash(1) General Command Manual," http://man7.org/linux/man-pages/man1/bash.1.html, 2021.

[114] "Mitmproxy: An interactive https proxy," https://mitmproxy.org/, 2022.

[115] "Mitmproxy based devtools extension," http://dutzi.github.io/tamper/, 2022.

[116] P. Swigger, "Cross-site scripting (xss) cheat sheet," https://portswigger.net/web-security/cross-site-scripting/cheat-sheet, 2022.

[117] "Xss filter evasion cheat sheet," https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet, 2019.

[118] "Firing Range," https://public-firing-range.appspot.com/, 2022.

[119] K. Fernandez and D. Pagkalos, "Cross site scripting (xss) attacks information and archive," http://xssed.com, 2012.

[120] "Open bug bounty, free bug bounty program and coordinated vulnerability disclosure," https://openbugbounty.org, 2022.

[121] "Open source web application security scanner," http://w3af.org/, 2022.

[122] "DETECTED BY RIPS, MISSED BY OTHERS," https://www.ripstech.com/product/reports/, 2019.

[123] R. Pelizzi and R. Sekar, "Protection, usability and improvements in reflected xss filters," in *ASIACCS*, 2012.

[124] Amazon, "Alexa Top Websites," https://www.alexa.com/topsites, 2020.

[125] "Selenium webdriver," https://www.selenium.dev/documentation/webdriver/, 2022.

[126] N. Galbreath, "libinjection," https://github.com/client9/libinjection, 2018.

[127] T. H. Ptacek and T. N. Newsham, "Insertion, evasion, and denial of service: Eluding network intrusion detection," Secure Networks inc Calgary Alberta, Tech. Rep., 1998.

[128] A. Sudhodanan, S. Khodayari, and J. Caballero, "Cross-origin state inference (cosi) attacks: Leaking web site states through xs-leaks," *arXiv preprint arXiv:1908.02204*, 2019.

[129] A. Niakanlahiji and J. H. Jafarian, "Webmtd: defeating web code injection attacks using web element attribute mutation," in *Proceedings of the 2017 Workshop on Moving Target Defense*, 2017, pp. 17–26.

[130] M. Johns, B. Engelmann, and J. Posegga, "Xssds: Server-side detection of cross-site scripting attacks," in *ACSAC*, 2008.

[131] M. T. Louw and V. N. Venkatakrishnan, "Blueprint: Robust Prevention of Cross-site Scripting Attacks for Existing Browsers," in *IEEE Symposium on Security and Privacy*, 2009.

[132] B. Stock, S. Lekies, T. Mueller, P. Spiegel, M. Johns, F. A. U. Erlangen-nuremberg, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, "Precise Client-side Protection against DOM-based Cross-Site Scripting," in *USENIX Security*, 2014.

[133] V. Prokhorenko, K.-K. R. Choo, and H. Ashman, "Context-oriented web application protection model," *Applied Mathematics and Computation*, vol. 285, 2016.

[134] M. Steffens and B. Stock, "Pmforce: Systematically analyzing postmessage handlers at scale," in *CCS*, 2020.

[135] S. Khodayari and G. Pellegrino, "Jaw: Studying client-side csrf with hybrid property graphs and declarative traversals," in *USENIX Security*, 2021.

[136] Z. Kang, S. Li, and Y. Cao, "Probe the proto: Measuring client-side prototype pollution vulnerabilities of one million real-world websites," 2022.

[137] L. Knittel, C. Mainka, M. Niemietz, D. T. Noß, and J. Schwenk, "Xsinator.com: From a formal model to the automatic evaluation of cross-site leaks in web browsers," in *CCS*, 2021.

[138] A. Shusterman, A. Agarwal, S. O'Connell, D. Genkin, Y. Oren, and Y. Yarom, "Prime+Probe 1, JavaScript 0: Overcoming browser-based Side-Channel defenses," in *USENIX Security*, 2021.

[139] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," in *IEEE symposium on Security and privacy*. IEEE, 2010.

[140] W. Halfond, A. Orso, and P. Manolios, "Wasp: Protecting web applications using positive tainting and syntax-aware evaluation," *IEEE transactions on Software Engineering*, vol. 34, no. 1, 2008.

[141] A. Nguyen-Tuong, S. Guarnieri, D. Greene, J. Shirley, and D. Evans, "Automatically hardening web applications using precise tainting," in *IFIP International Information Security Conference*. Springer, 2005.

[142] Google, "Lighthouse," https://developers.google.com/web/tools/lighthouse.

[143] BuiltWith, "Javascript usage distribution in the top 1 million sites," https://trends.builtwith.com/javascript.

[144] M. Hanson, P. Lawler, and S. Macbeth, "The tracker tax: the impact of third-party trackers on website speed in the united states," Tech. Rep., 2018.

[145] "Web browser javascript benchmark," http://celtickane.com/labs/web-browser-javascript-benchmark.

[146] "Adblock," https://chrome.google.com/webstore/detail/adblock-%E2%80%94-best-ad-blocker/gighmmpiobklfepjocnamgkkbiglidom.

[147] "ublock origin," https://chrome.google.com/webstore/detail/ublock-origin/cjpalhdlnbpafiamejdnhcphjbkeiagm.

[148] G. G. Gulyas, D. F. Some, N. Bielova, and C. Castelluccia, "To extend or not to extend: on the uniqueness of browser extensions and web logins," in *Proceedings of the 2018 Workshop on Privacy in the Electronic Society*, 2018.

[149] R. Dingledine and N. Mathewson, "Anonymity loves company: Usability and the network effect." in *WEIS*, 2006.

[150] "Tor project — anonymity online," https://www.torproject.org.

[151] T. Van Goethem and W. Joosen, "One side-channel to bring them all and in the darkness bind them: Associating isolated browsing sessions," in *WOOT*, 2017.

[152] I. Sanchez-Rola, I. Santos, and D. Balzarotti, "Extension Breakdown: Security Analysis of Browsers Extension Resources Control Policies," in *USENIX Security*, 2017.

[153] O. Starov, P. Laperdrix, A. Kapravelos, and N. Nikiforakis, "Unnecessarily identifiable: Quantifying the fingerprintability of browser extensions due to bloat," in *WWW*, 2019.

[154] A. Sjösten, S. Van Acker, P. Picazo-Sanchez, and A. Sabelfeld, "Latex gloves: Protecting browser extensions from probing and revelation attacks," in *NDSS*, 2019.

[155] O. Starov and N. Nikiforakis, "Extended tracking powers: Measuring the privacy diffusion enabled by browser extensions," in *WWW*, 2017.

[156] MDN, "Mdn web docs - using shadow dom," https://developer.mozilla.org/en-US/docs/Web/Web_Components/Using_shadow_DOM.

[157] E. Lavoie, B. Dufour, and M. Feeley, "Portable and efficient run-time monitoring of javascript applications using virtual machine layering," in *European Conference on Object-Oriented Programming*, 2014.

[158] J. W. Mickens, J. Elson, and J. Howell, "Mugshot: Deterministic capture and replay for javascript applications." in *NSDI*, 2010.

[159] P. Snyder, L. Ansari, C. Taylor, and C. Kanich, "Browser feature usage on the modern web," in *IMC*, 2016.

[160] J. Onaolapo, E. Mariconti, and G. Stringhini, "What happens after you are pwnd: Understanding the use of leaked webmail credentials in the wild," in *Proceedings of the 2016 Internet Measurement Conference*, 2016, pp. 65–79.

[161] K. Thomas, F. Li, A. Zand, J. Barrett, J. Ranieri, L. Invernizzi, Y. Markov, O. Comanescu, V. Eranti, A. Moscicki *et al.*, "Data breaches, phishing, or malware? understanding the risks of stolen credentials," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2017.

[162] M. Zaeifi, F. Kalantari, A. Oest, Z. Sun, G.-J. Ahn, Y. Shoshitaishvili, T. Bao, R. Wang, and A. Doupé, "Nothing personal: Understanding the spread and use of personally identifiable information in the financial ecosystem," in *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy*, 2024, pp. 55–65.

[163] G. Milka, "Anatomy of account takeover," in *Enigma 2018 (Enigma 2018)*, 2018.

[164] D. Freeman, S. Jain, M. Dürmuth, B. Biggio, and G. Giacinto, "Who are you? a statistical approach to measuring user authenticity." in *NDSS*, vol. 16, 2016, pp. 21–24.

[165] A. Durey, P. Laperdrix, W. Rudametkin, and R. Rouvoy, "Fp-redemption: Studying browser fingerprinting adoption for the sake of web security," in *Detection of Intrusions and Malware, and Vulnerability Assessment: 18th International Conference, DIMVA 2021, Virtual Event, July 14–16, 2021, Proceedings 18*. Springer, 2021, pp. 237–257.

[166] A. Senol, A. Ukani, D. Cutler, and I. Bilogrevic, "The double edged sword: Identifying authentication pages and their fingerprinting behavior," in *Proceedings of the ACM on Web Conference 2024*, 2024, pp. 1690–1701.

[167] E. Bursztein, A. Malyshev, T. Pietraszek, and K. Thomas, "Picasso: Lightweight device class fingerprinting for web clients," in *Proceedings of the 6th Workshop on Security and Privacy in Smartphones and Mobile Devices*, 2016.

[168] Z. Liu, P. Shrestha, and N. Saxena, "Gummy browsers: targeted browser spoofing against state-of-the-art fingerprinting techniques," in *International Conference on Applied Cryptography and Network Security*. Springer, 2022, pp. 147–169.

[169] X. Lin, P. Ilia, S. Solanki, and J. Polakis, "Phish in sheep's clothing: Exploring the authentication pitfalls of browser fingerprinting," in *USENIX*, 2022.

[170] M. Campobasso and L. Allodi, "Impersonation-as-a-service: Characterizing the emerging criminal infrastructure for user impersonation at scale," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1665–1680.

[171] G. Advisory, "Illicit antidetect platforms vs. anti-fraud solutions," https://geminiadvisory.io/antidetect-platforms-vs-anti-fraud-solutions, 2023.

[172] B. A. Azad, O. Starov, P. Laperdrix, and N. Nikiforakis, "Taming the shape shifter: Detecting anti-fingerprinting browsers," in *DIMVA*, 2020.

[173] N. Andriamilanto, T. Allard, and G. Le Guelvouit, "Fpselect: low-cost browser fingerprints for mitigating dictionary attacks against web authentication mechanisms," in *Proceedings of the 36th Annual Computer Security Applications Conference*, 2020, pp. 627–642.

[174] C. Team, "ClonBrowser," https://www.clonbrowser.com, 2024.

[175] A. Team, "AntBrowser," https://antbrowser.pro, 2024.

[176] I. Team, "incogniton," https://incogniton.com, 2024.

[177] G. Team, "GoLogin," https://gologin.com, 2024.

[178] O. Team, "Octo Browser," https://octobrowser.net, 2024.

[179] Tenebris, "Linken Sphere," https://ls.tenebris.cc, 2024.

[180] S. Team, "Sphere – A Browser for Anonymity," https://www.geeksforgeeks.org/sphere-a-browser-for-anonymity/y, 2024.

[181] V. Team, "VMLogin," https://www.vmlogin.us, 2024.

[182] C. Team, "CheBrowser," https://chebrowser.site, 2024.

[183] A. Team, "AdsPower," https://www.adspower.com, 2024.

[184] F. Team, "FingerprintJS," https://github.com/fingerprintjs/fingerprintjs, 2024.

[185] A. Team, "AmIUnique," https://amiunique.org, 2024.

[186] C. Team, "ClientJS," http://clientjs.org, 2024.

[187] W. Team, "Browser fingerprinting market share," https://www.wappalyzer.com/technologies/browser-fingerprinting, 2024.

[188] I. Sanchez-Rola, L. Bilge, D. Balzarotti, A. Buescher, and P. Efstathopoulos, "Rods with laser beams: understanding browser fingerprinting on phishing pages," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4157–4173.

[189] S. A. Akhavani, J. Jueckstock, J. Su, A. Kapravelos, E. Kirda, and L. Lu, "Browserprint: An analysis of the impact of browser features on fingerprintability and web privacy," in *International Conference on Information Security*, 2021.

[190] MDN, "MDN Web API," https://developer.mozilla.org/en-US/docs/Web/API, 2022.

[191] P. Picazo-Sanchez, G. Schneider, and A. Sabelfeld, "Hmac and "secure preferences": Revisiting chromium-based browsers security," in *Cryptology and Network Security: 19th International Conference, CANS 2020, Vienna, Austria, December 14–16, 2020, Proceedings 19*. Springer, 2020, pp. 107–126.

[192] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth ieee international conference on data mining*. IEEE, 2008, pp. 413–422.

[193] I. T. Jolliffe, *Principal component analysis for special types of data*. Springer, 2002.

[194] H. Humaira and R. Rasyidah, "Determining the appropriate cluster number using elbow method for k-means algorithm," in *Proceedings of the 2nd Workshop on Multidisciplinary and Applications (WMA) 2018*, 2020.

[195] D. Varmedja, M. Karanovic, S. Sladojevic, M. Arsenovic, and A. Anderla, "Credit card fraud detection-machine learning methods," in *2019 18th International Symposium INFOTEH-JAHORINA (INFOTEH)*. IEEE, 2019, pp. 1–5.

[196] Kaggle, "Credit card fraud detection," https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, Feb 2024.

[197] Radware, "Detecting and mitigating highly distributed sophisticated bot attacks," https://www.radware.com/blog/application-protection/2023/11/detecting-and-mitigating-highly-distributed-sophisticated-bot-attacks, 2024.

[198] S. L. Lohr, *Sampling: Design and Analysis*. Cengage Learning, 2019.

[199] R. Kawase, F. Diana, M. Czeladka, M. Schüler, and M. Faust, "Internet fraud: the case of account takeover in online marketplace," in *Proceedings of the 30th ACM Conference on Hypertext and Social Media*, 2019, pp. 181–190.

[200] S. Wu, P. Sun, Y. Zhao, and Y. Cao, "Him of many faces: Characterizing billion-scale adversarial and benign browser fingerprints on commercial websites." in *NDSS*, 2023.

[201] P. Laperdrix, G. Avoine, B. Baudry, and N. Nikiforakis, "Morellian analysis for browsers: Making web authentication stronger with canvas fingerprinting," in *DIMVA*, 2019.

[202] Google, "Privacy sandbox," 2024, accessed: August 19, 2024. [Online]. Available: https://privacysandbox.com

[203] B. Wiser, "Web Environment Integrity," https://rupertbenwiser.github.io/Web-Environment-Integrity, 2023.

[204] Cloudflare, "Privacy pass," 2024, accessed: August 19, 2024. [Online]. Available: https://developers.cloudflare.com/waf/tools/privacy-pass/

[205] Akhavani, Seyed Ali, "bowserprint github," https://github.com/sa-akhavani/browserprint, Jan. 2024.

# APPENDIX A

# SIMULACRUM DETAILS

**Overriding prototype methods.** We employ the following steps: keeping a reference to the original function (Line 1); declaring and defining the new wrapper function (Lines 2-7); implementing the overriding logic (Lines 3, 5); executing the original function on proper inputs (Line 4); returning the desired value from the function (Line 6); and assigning the wrapper function to the object's prototype to overwrite the original method (Line 8).

```
  1  GET /?id=Admin";alert(1);" HTTP/1.1
...  Host: vulnerable.com
```

Listing 5: Overriding `function` from `Parent`'s prototype.

**Overriding object properties.** Listing 6 shows how we use `Object.defineProperty()` to override the `setter` and `getter` functions for the `id` property of the `parentObj` prototype.

```
  1  GET /?id=Admin";alert(1);" HTTP/1.1
...  Host: vulnerable.com
```

Listing 6: Overriding `id` property from `parentObj`'s prototype; `obj` and `args` values change based on our logic.

**Recording function invocations.** We created the extension VisibleJS to gather information related to DOM usage by websites. Listing 7 shows the script VisibleJS uses to wrap all the JavaScript interfaces. While the webpage runs VisibleJS reports the interfaces used by the website.

```
  1 let orig = original[<interface.method>];
... interface.prototype['method'] = function(){
 15     let toReturn = orig.apply(this, arguments);
 16     collect(this, arguments, toReturn);
 17     return toReturn;
... }
```

Listing 7: VisibleJS general strategy.

**Observer interfaces.** Listing 8 shows simple examples that demonstrate using three different types of observers. `ResizeObserver` does not need a configuration parameter. `IntersectionObserver` receives the configuration and callback parameters when it is constructed. `MutationObserver` receives the callback during construction and receives the target and the configuration details when the observe method is invoked.

```
  1 let orig = original[<interface.method>];
... interface.prototype['method'] = function(){
 15     let toReturn = orig.apply(this, arguments);
 16     collect(this, arguments, toReturn);
 17     return toReturn;
... }
```

Listing 8: Observer instantiation examples.

**Table A.1:** The number of overridden functions for noteworthy interfaces. * Element interface and all other interfaces that inherit from it.

Note: Some properties have both a setter and getter, thus the sum of used strategies is more than the total.

| | | | | Overridden Functions | | | | |
|---|---|---|---|---|---|---|---|---|
| Interface | Props | Methods | Total | Simp Setter | Simp Getter | Act Setter | Act Getter | Args |
| Node | 14 | 15 | 29 | 7 | 23 | 0 | 1 | 8 |
| Document | 264 | 40 | 304 | 20 | 34 | 115 | 153 | 6 |
| Element* | 1,283 | 189 | 1,472 | 687 | 968 | 84 | 193 | 14 |

**MutationObserver.** Listing 9 shows the wrapper code used in the `MutationObserver`'s constructor. Simulacrum overrides `MutationObserver`'s constructor to modify the incoming callback. Before calling the original observer method, the wrapper filters out the mutations that originate from an extension.

```
 1  let orig = original[<interface.method>];
...  interface.prototype['method'] = function(){
15      let toReturn = orig.apply(this, arguments);
16      collect(this, arguments, toReturn);
17      return toReturn;
...  }
```

Listing 9: Wrapping the constructor of `MutationObserver`.

**Overriding functions.** Table A.1 shows the number of overridden functions. The first column lists the noteworthy interfaces. The leftmost grouping breaks the interfaces down by properties and methods overridden. On the right side, the table breaks the interfaces down by the type of wrapper used, with the most common category being the simple setter and getter categories.

**Examples of overridden functions.** Table A.2 displays different function examples for each category with their overridden counterpart. For readability reasons we omit details in the "Overridden Function" column. For example, we use the "original" hash table (§2.4.3.1) for calling the functions but we do not include them in this table for simplicity. That is, instead of `original["Element.innerHTML_setter"].call(e,"text")`, we write `e.innerHTML="text"`.

**Inheritance hierarchy.** In Figure A.1 we provide a partial representation of the inheritance hierarchy in respect to the DOM API and some of the interfaces pertinent to our work.

**Breakage tests.** We manually assessed Simulacrum's effect on extension and website functionality by testing common operations with Simulacrum installed. We had a user freely explore websites with and without Simulacrum installed for a few minutes and then report any effects to the browsing experience when Simulacrum was present. This included assessing each website's functionality (e.g., login and payment processes, playing videos, interacting with website-specific tools, etc.) and was not limited to a visual inspection. For the scenarios testing the impact on other extensions, a user first installed the candidate extension and learned their expected functionality on a few target websites (either the ones advertised in the extension's overview page in the Chrome web store or on websites of user's choice). Subsequently,

**Table A.2:** Examples of different functions and our strategy for wrapping them.

| Category | Example | Overridden Function |
|---|---|---|
| Simple getter | `document.getElementsByTagName("div")` | `parallelDOM.getElementsByTagName("div")` |
| Simple getter | `e.hasAttribute("src");` | `parallelDOM.getElementById(e.id).hasAttribute('src')` |
| Active getter | `e.scrollTop` | `if (parallelDOM.getElementById(e.id))`<br>`  userDOM.getElementById(e.id).scrollTop` |
| Active getter | `document.activeElement` | `result = userDOM.activeElement;`<br>`return parallelDOM.getElementById(result.id);` |
| Simple setter | `e.innerHTML = "text"` | `equivalentElement = getEquivalent(e);`<br>`e.innerHTML = "text";`<br>`equivalentElement.innerHTML = "text";` |
| Simple setter | `document.write("text")` | `document.write("text")`<br>`parallelDOM.write("text");` |
| Active setter | `e.requestFullscreen()` | `if (parallelDOM.getElementById(e.id))`<br>`  userDOM.getElementById(e.id).requestFullscreen()` |
| Arguments | `n.insertBefore(newNode, refNode)` | `equivNewNode = deepClone(newNode);`<br>`equivRefNode = getEquivalent(refNode);`<br>`equivNode = getEquivalent(n);`<br>`n.insertBefore(newNode, refNode);`<br>`equivNode.insertBefore(equivNewNode, equivRefNode);` |

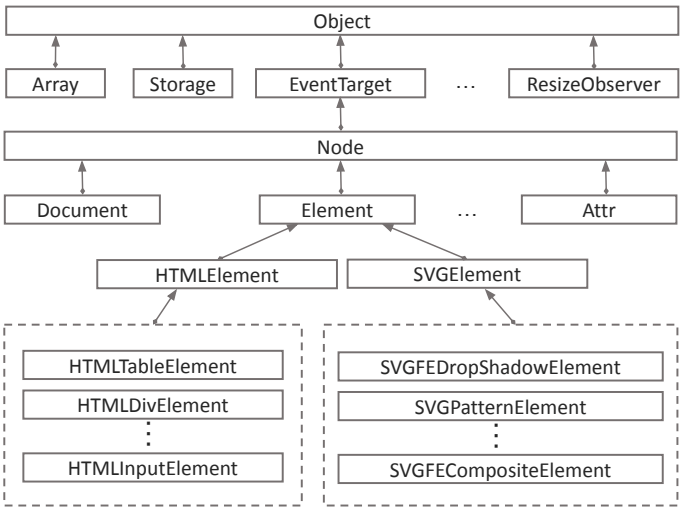


**Figure A.1:** Inheritance hierarchy in the DOM API.

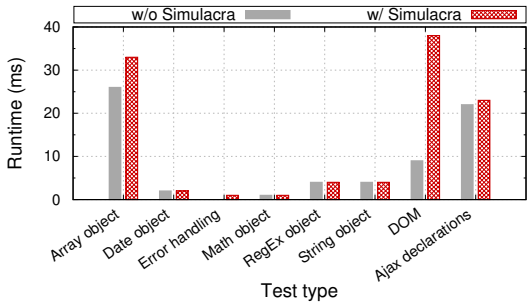


**Figure A.2:** Run-time overhead when evaluating Simulacrum with the Celtic Kane open-source benchmark speed tests.

the expected functionality was reevaluated with Simulacrum installed. In the case of privacy extensions (Privacy Badger [87], AdBlock [146] and uBlock [147]) we specifically chose five popular news websites (`nytimes.com`, `foxnews.com`, `abcnews.com`, `cnn.com` and `bbc.com`) which usually involve tracking scripts and verified that these privacy extensions continued to operate as expected and their functionality was not affected by Simulacrum.

**Benchmarking tests.** In Figure A.2 we present the results from our performance test using the Celtic Kane open-source benchmark. We find that in most cases Simulacrum does not affect performance, while for the DOM test our system introduces a 30 millisecond overhead.

**Noise-based defense against position-based attacks.** In Figure A.3 we provide an example screenshot that shows a page's visual appearance when Simulacrum introduces noisy elements as a countermeasure against position-based attacks. These elements are simply empty spaces that change the position of other elements on the page.

figures/noise.png

**Figure A.3:** Example of site with added noise elements.

# APPENDIX B

# CONTEXT-AUDITOR DETAILS

In the following two pages, we have demonstrated two simplified automata for parsing shell scripts and HTML responses.

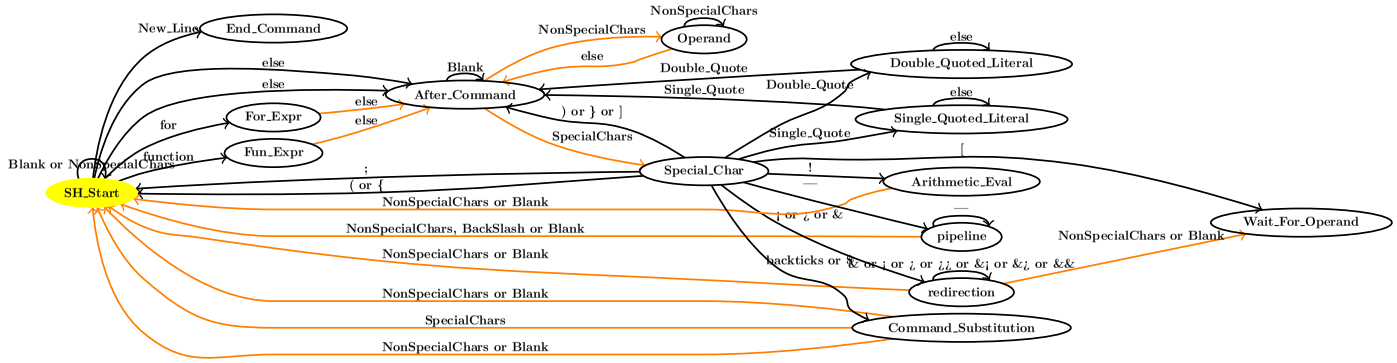


**Figure B.1:** A simplified 2PDA that parses shell scripts. `SH_Start` is the starting state. Orange edges demonstrate the revisiting functionality.

**Figure B.2:** A simplified 2PDA that represents state transitions between HTML, CSS, and JavaScript. `HTML_Start` is the starting state. `Script_Start` and `CSS_Start` represent the states where first characters of JavaScript and CSS code are consumed by the automaton, respectively. The automaton transitions back to `HTML_Start` from JavaScript or CSS states whenever it consumes a sequence of characters that represent an end tag in the corresponding language.

# APPENDIX C

# BROWSER POLYGRAPH FEATURES

Here is the list of features used for training of Browser Polygraph. The *deviation-based* features focus on counting JavaScript properties from the list of all MDN prototypes. These are initially filtered by standard deviation across all tested browsers during the candidate fingerprint generation stage, and then further filtered during the data pre-processing stage due to inconsistencies in actual browser data. The *time-based* features were initially derived from BrowserPrint [205]'s work, which examines the presence or absence of properties in browser releases. However, many of these features were subsequently filtered out as they did not track browser changes after 2020.

**Table C.1:** Features set used for training of Browser Polygraph.

| Num | Feature | Type |
|---|---|---|
| 1 | `Object.getOwnPropertyNames(Element.prototype).length` | *deviation-based* |
| 2 | `Object.getOwnPropertyNames(Document.prototype).length` | *deviation-based* |
| 3 | `Object.getOwnPropertyNames(HTMLElement.prototype).length` | *deviation-based* |
| 4 | `Object.getOwnPropertyNames(SVGElement.prototype).length` | *deviation-based* |
| 5 | `Object.getOwnPropertyNames(SVGFEBlendElement.prototype).length` | *deviation-based* |
| 6 | `Object.getOwnPropertyNames(TextMetrics.prototype).length` | *deviation-based* |
| 7 | `Object.getOwnPropertyNames(Range.prototype).length` | *deviation-based* |
| 8 | `Object.getOwnPropertyNames(StaticRange.prototype).length` | *deviation-based* |
| 9 | `Object.getOwnPropertyNames(AuthenticatorAttestationResponse.prototype).length` | *deviation-based* |
| 10 | `Object.getOwnPropertyNames(HTMLVideoElement.prototype).length` | *deviation-based* |
| 11 | `Object.getOwnPropertyNames(ResizeObserverEntry.prototype).length` | *deviation-based* |
| 12 | `Object.getOwnPropertyNames(ShadowRoot.prototype).length` | *deviation-based* |
| 13 | `Object.getOwnPropertyNames(PointerEvent.prototype).length` | *deviation-based* |
| 14 | `Object.getOwnPropertyNames(IntersectionObserver.prototype).length` | *deviation-based* |
| 15 | `Object.getOwnPropertyNames(CanvasRenderingContext2D.prototype).length` | *deviation-based* |
| 16 | `Object.getOwnPropertyNames(CSSStyleSheet.prototype).length` | *deviation-based* |
| 17 | `Object.getOwnPropertyNames(AudioContext.prototype).length` | *deviation-based* |
| 18 | `Object.getOwnPropertyNames(HTMLLinkElement.prototype).length` | *deviation-based* |
| 19 | `Object.getOwnPropertyNames(HTMLMediaElement.prototype).length` | *deviation-based* |
| 20 | `Object.getOwnPropertyNames(WebGL2RenderingContext.prototype).length` | *deviation-based* |
| 21 | `Object.getOwnPropertyNames(WebGLRenderingContext.prototype).length` | *deviation-based* |
| 22 | `Object.getOwnPropertyNames(CSSRule.prototype).length` | *deviation-based* |
| 23 | `Navigator.prototype.hasOwnProperty('deviceMemory')` | *time-based* |
| 24 | `BaseAudioContext.prototype.hasOwnProperty('currentTime')` | *time-based* |
| 25 | `HTMLVideoElement.prototype.hasOwnProperty('webkitDisplayingFullscreen')` | *time-based* |
| 26 | `Screen.prototype.hasOwnProperty('orientation')` | *time-based* |
| 27 | `Window.prototype.hasOwnProperty('speechSynthesis')` | *time-based* |
| 28 | `CSSStyleDeclaration.prototype.hasOwnProperty('getPropertyValue')` | *time-based* |

# APPENDIX D

# ALL FEATURES USED IN THE REAL-WORLD DATA COLLECTION STAGE

Here are the list of 200 features used for the *Real-World Data Collection* stage besides 313 features from browserprint's code [205]:

```
Object.getOwnPropertyNames(Element.prototype).length
Object.getOwnPropertyNames(Document.prototype).length
Object.getOwnPropertyNames(HTMLElement.prototype).length
Object.getOwnPropertyNames(SVGElement.prototype).length
Object.getOwnPropertyNames(Navigator.prototype).length
Object.getOwnPropertyNames(RTCIceCandidate.prototype).length
Object.getOwnPropertyNames(SVGFEBlendElement.prototype).length
Object.getOwnPropertyNames(TextMetrics.prototype).length
Object.getOwnPropertyNames(Range.prototype).length
Object.getOwnPropertyNames(StaticRange.prototype).length
Object.getOwnPropertyNames(RTCRtpReceiver.prototype).length
Object.getOwnPropertyNames(RTCPeerConnection.prototype).length
Object.getOwnPropertyNames(AuthenticatorAttestationResponse.prototype).length
Object.getOwnPropertyNames(FontFace.prototype).length
Object.getOwnPropertyNames(HTMLVideoElement.prototype).length
Object.getOwnPropertyNames(ResizeObserverEntry.prototype).length
Object.getOwnPropertyNames(ShadowRoot.prototype).length
Object.getOwnPropertyNames(RTCRtpSender.prototype).length
Object.getOwnPropertyNames(PointerEvent.prototype).length
Object.getOwnPropertyNames(Blob.prototype).length
Object.getOwnPropertyNames(ServiceWorkerRegistration.prototype).length
Object.getOwnPropertyNames(MediaSession.prototype).length
Object.getOwnPropertyNames(PaymentResponse.prototype).length
Object.getOwnPropertyNames(HTMLSourceElement.prototype).length
Object.getOwnPropertyNames(Clipboard.prototype).length
Object.getOwnPropertyNames(IDBTransaction.prototype).length
Object.getOwnPropertyNames(Performance.prototype).length
Object.getOwnPropertyNames(ServiceWorkerContainer.prototype).length
Object.getOwnPropertyNames(HTMLIFrameElement.prototype).length
Object.getOwnPropertyNames(PaymentRequest.prototype).length
Object.getOwnPropertyNames(RTCRtpTransceiver.prototype).length
Object.getOwnPropertyNames(IntersectionObserver.prototype).length
Object.getOwnPropertyNames(CanvasRenderingContext2D.prototype).length
Object.getOwnPropertyNames(CSSStyleSheet.prototype).length
Object.getOwnPropertyNames(BaseAudioContext.prototype).length
Object.getOwnPropertyNames(AudioContext.prototype).length
Object.getOwnPropertyNames(HTMLLinkElement.prototype).length
Object.getOwnPropertyNames(RTCDataChannel.prototype).length
Object.getOwnPropertyNames(WritableStream.prototype).length
Object.getOwnPropertyNames(DataTransferItem.prototype).length
Object.getOwnPropertyNames(DocumentFragment.prototype).length
Object.getOwnPropertyNames(HTMLMediaElement.prototype).length
Object.getOwnPropertyNames(StorageManager.prototype).length
Object.getOwnPropertyNames(HTMLSlotElement.prototype).length
Object.getOwnPropertyNames(Text.prototype).length
Object.getOwnPropertyNames(WebGL2RenderingContext.prototype).length
Object.getOwnPropertyNames(HTMLInputElement.prototype).length
```

```
Object.getOwnPropertyNames(WebGLRenderingContext.prototype).length
Object.getOwnPropertyNames(HTMLButtonElement.prototype).length
Object.getOwnPropertyNames(HTMLTextAreaElement.prototype).length
Object.getOwnPropertyNames(HTMLSelectElement.prototype).length
Object.getOwnPropertyNames(MediaRecorder.prototype).length
Object.getOwnPropertyNames(CountQueuingStrategy.prototype).length
Object.getOwnPropertyNames(ByteLengthQueuingStrategy.prototype).length
Object.getOwnPropertyNames(PerformanceMark.prototype).length
Object.getOwnPropertyNames(PerformanceMeasure.prototype).length
Object.getOwnPropertyNames(HTMLImageElement.prototype).length
Object.getOwnPropertyNames(SpeechSynthesisEvent.prototype).length
Object.getOwnPropertyNames(HTMLFormElement.prototype).length
Object.getOwnPropertyNames(IDBCursor.prototype).length
Object.getOwnPropertyNames(HTMLTemplateElement.prototype).length
Object.getOwnPropertyNames(CSSRule.prototype).length
Object.getOwnPropertyNames(Location.prototype).length
Object.getOwnPropertyNames(PaymentAddress.prototype).length
Object.getOwnPropertyNames(IntersectionObserverEntry.prototype).length
Object.getOwnPropertyNames(TextEncoder.prototype).length
Object.getOwnPropertyNames(ImageData.prototype).length
Object.getOwnPropertyNames(HTMLMetaElement.prototype).length
Object.getOwnPropertyNames(Crypto.prototype).length
Object.getOwnPropertyNames(GamepadButton.prototype).length
Object.getOwnPropertyNames(DOMMatrixReadOnly.prototype).length
Object.getOwnPropertyNames(MediaKeys.prototype).length
Object.getOwnPropertyNames(MessageEvent.prototype).length
Object.getOwnPropertyNames(IDBFactory.prototype).length
Object.getOwnPropertyNames(MediaDevices.prototype).length
Object.getOwnPropertyNames(OfflineAudioContext.prototype).length
Object.getOwnPropertyNames(URL.prototype).length
Object.getOwnPropertyNames(ScriptProcessorNode.prototype).length
Object.getOwnPropertyNames(SVGAnimatedNumberList.prototype).length
Object.getOwnPropertyNames(ServiceWorker.prototype).length
Object.getOwnPropertyNames(SensorErrorEvent.prototype).length
Object.getOwnPropertyNames(SVGAnimatedPreserveAspectRatio.prototype).length
Object.getOwnPropertyNames(Sensor.prototype).length
Object.getOwnPropertyNames(SVGAnimatedRect.prototype).length
Object.getOwnPropertyNames(SVGAnimatedString.prototype).length
Object.getOwnPropertyNames(Selection.prototype).length
Object.getOwnPropertyNames(SecurityPolicyViolationEvent.prototype).length
Object.getOwnPropertyNames(XPathExpression.prototype).length
Object.getOwnPropertyNames(SVGAnimatedNumber.prototype).length
Object.getOwnPropertyNames(SVGAnimatedTransformList.prototype).length
Object.getOwnPropertyNames(Screen.prototype).length
Object.getOwnPropertyNames(RTCTrackEvent.prototype).length
Object.getOwnPropertyNames(SVGAnimateElement.prototype).length
Object.getOwnPropertyNames(SVGAnimateMotionElement.prototype).length
Object.getOwnPropertyNames(RTCStatsReport.prototype).length
Object.getOwnPropertyNames(RTCSessionDescription.prototype).length
Object.getOwnPropertyNames(SVGAnimateTransformElement.prototype).length
```

```
Object.getOwnPropertyNames(ScreenOrientation.prototype).length
Object.getOwnPropertyNames(SVGAnimatedlengthList.prototype).length
Object.getOwnPropertyNames(XPathResult.prototype).length
Object.getOwnPropertyNames(SVGAngle.prototype).length
Object.getOwnPropertyNames(SVGAElement.prototype).length
Object.getOwnPropertyNames(SubtleCrypto.prototype).length
Object.getOwnPropertyNames(SVGAnimatedAngle.prototype).length
Object.getOwnPropertyNames(StyleSheetList.prototype).length
Object.getOwnPropertyNames(StyleSheet.prototype).length
Object.getOwnPropertyNames(StylePropertyMapReadOnly.prototype).length
Object.getOwnPropertyNames(StylePropertyMap.prototype).length
Object.getOwnPropertyNames(XPathEvaluator.prototype).length
Object.getOwnPropertyNames(SVGAnimatedBoolean.prototype).length
Object.getOwnPropertyNames(SharedWorker.prototype).length
Object.getOwnPropertyNames(StorageEvent.prototype).length
Object.getOwnPropertyNames(Storage.prototype).length
Object.getOwnPropertyNames(StereoPannerNode.prototype).length
Object.getOwnPropertyNames(SVGAnimatedEnumeration.prototype).length
Object.getOwnPropertyNames(SpeechSynthesisUtterance.prototype).length
Object.getOwnPropertyNames(SVGAnimatedInteger.prototype).length
Object.getOwnPropertyNames(SVGAnimatedlength.prototype).length
Object.getOwnPropertyNames(SpeechSynthesisErrorEvent.prototype).length
Object.getOwnPropertyNames(SourceBufferList.prototype).length
Object.getOwnPropertyNames(SourceBuffer.prototype).length
Object.getOwnPropertyNames(WebGLFramebuffer.prototype).length
Object.getOwnPropertyNames(PresentationConnection.prototype).length
Object.getOwnPropertyNames(Plugin.prototype).length
Object.getOwnPropertyNames(PluginArray.prototype).length
Object.getOwnPropertyNames(PopStateEvent.prototype).length
Object.getOwnPropertyNames(Presentation.prototype).length
Object.getOwnPropertyNames(PresentationAvailability.prototype).length
Object.getOwnPropertyNames(PresentationConnectionAvailableEvent.prototype).length
Object.getOwnPropertyNames(PresentationConnectionCloseEvent.prototype).length
Object.getOwnPropertyNames(PresentationConnectionList.prototype).length
Object.getOwnPropertyNames(PresentationReceiver.prototype).length
Object.getOwnPropertyNames(PresentationRequest.prototype).length
Object.getOwnPropertyNames(ProcessingInstruction.prototype).length
Object.getOwnPropertyNames(PictureInPictureWindow.prototype).length
Object.getOwnPropertyNames(PermissionStatus.prototype).length
Object.getOwnPropertyNames(PromiseRejectionEvent.prototype).length
Object.getOwnPropertyNames(PerformanceNavigationTiming.prototype).length
Object.getOwnPropertyNames(PerformanceObserver.prototype).length
Object.getOwnPropertyNames(PerformanceObserverEntryList.prototype).length
Object.getOwnPropertyNames(PerformancePaintTiming.prototype).length
Object.getOwnPropertyNames(Permissions.prototype).length
Object.getOwnPropertyNames(PerformanceResourceTiming.prototype).length
Object.getOwnPropertyNames(PerformanceServerTiming.prototype).length
Object.getOwnPropertyNames(PerformanceTiming.prototype).length
Object.getOwnPropertyNames(PeriodicWave.prototype).length
Object.getOwnPropertyNames(ProgressEvent.prototype).length
```

```
Object.getOwnPropertyNames(PublicKeyCredential.prototype).length
Object.getOwnPropertyNames(RTCDTMFToneChangeEvent.prototype).length
Object.getOwnPropertyNames(RTCCertificate.prototype).length
Object.getOwnPropertyNames(RTCDataChannelEvent.prototype).length
Object.getOwnPropertyNames(RTCDTMFSender.prototype).length
Object.getOwnPropertyNames(RTCPeerConnectionIceEvent.prototype).length
Object.getOwnPropertyNames(Response.prototype).length
Object.getOwnPropertyNames(PushManager.prototype).length
Object.getOwnPropertyNames(PushSubscription.prototype).length
Object.getOwnPropertyNames(PushSubscriptionOptions.prototype).length
Object.getOwnPropertyNames(RadioNodeList.prototype).length
Object.getOwnPropertyNames(ReadableStream.prototype).length
Object.getOwnPropertyNames(ResizeObserver.prototype).length
Object.getOwnPropertyNames(RelativeOrientationSensor.prototype).length
Object.getOwnPropertyNames(RemotePlayback.prototype).length
Object.getOwnPropertyNames(ReportingObserver.prototype).length
Object.getOwnPropertyNames(Request.prototype).length
Object.getOwnPropertyNames(SVGAnimationElement.prototype).length
Object.getOwnPropertyNames(XMLHttpRequestEventTarget.prototype).length
Object.getOwnPropertyNames(SVGCircleElement.prototype).length
Object.getOwnPropertyNames(TreeWalker.prototype).length
Object.getOwnPropertyNames(WebGLTexture.prototype).length
Object.getOwnPropertyNames(TextDecoderStream.prototype).length
Object.getOwnPropertyNames(TextEncoderStream.prototype).length
Object.getOwnPropertyNames(WebGLSync.prototype).length
Object.getOwnPropertyNames(TextTrack.prototype).length
Object.getOwnPropertyNames(TextTrackCue.prototype).length
Object.getOwnPropertyNames(TextTrackCueList.prototype).length
Object.getOwnPropertyNames(WebGLShaderPrecisionFormat.prototype).length
Object.getOwnPropertyNames(TextTrackList.prototype).length
Object.getOwnPropertyNames(TimeRanges.prototype).length
Object.getOwnPropertyNames(Touch.prototype).length
Object.getOwnPropertyNames(TouchEvent.prototype).length
Object.getOwnPropertyNames(TouchList.prototype).length
Object.getOwnPropertyNames(TrackEvent.prototype).length
Object.getOwnPropertyNames(TransformStream.prototype).length
Object.getOwnPropertyNames(WebGLTransformFeedback.prototype).length
Object.getOwnPropertyNames(TextDecoder.prototype).length
Object.getOwnPropertyNames(WebGLUniformLocation.prototype).length
Object.getOwnPropertyNames(SVGTitleElement.prototype).length
Object.getOwnPropertyNames(WebGLVertexArrayObject.prototype).length
Object.getOwnPropertyNames(SVGSymbolElement.prototype).length
Object.getOwnPropertyNames(SVGTextContentElement.prototype).length
Object.getOwnPropertyNames(SVGTextElement.prototype).length
Object.getOwnPropertyNames(SVGTextPathElement.prototype).length
Object.getOwnPropertyNames(SVGTextPositioningElement.prototype).length
Object.getOwnPropertyNames(SVGTransform.prototype).length
Object.getOwnPropertyNames(TaskAttributionTiming.prototype).length
Object.getOwnPropertyNames(SVGTransformList.prototype).length
Object.getOwnPropertyNames(SVGTSpanElement.prototype).length
```

```
Object.getOwnPropertyNames(SVGUnitTypes.prototype).length
Object.getOwnPropertyNames(SVGUseElement.prototype).length
Object.getOwnPropertyNames(SVGViewElement.prototype).length
```

# APPENDIX E

# BROWSER POLYGRAPH WITH DIFFERENT NUMBER OF CLUSTERS

**Table E.1:** `user-agents` assigned to clusters by Browser Polygraph with a less optimal clusters' number choice, k=6.

| Cluster | user-agents |
|---|---|
| 0 | Chrome 114, Edge 114 |
| 1 | Chrome 90-96, Edge 90-96, Firefox 101-114 |
| 2 | Chrome 97-113, Edge 97-113 |
| 3 | Chrome 64-89, Edge 79-89 |
| 4 | Chrome 59-63, Firefox 51-100 |
| 5 | Edge 17,18, Firefox 46-50 |

# APPENDIX F

# SENSITIVITY ANALYSIS OF BROWSER POLYGRAPH

**Table F.1:** Sensitivity analysis for increased number of clusters with 28 features and 7 PCA components.

| **Number of clusters** | **5** | **7** | **9** | **11** | **13** | **15** | **17** | **19** |
|---|---|---|---|---|---|---|---|---|
| **Model accuracy** | 99.88% | 99.69% | 99.58% | 99.60% | 99.40% | 99.31% | 99.29% | 99.26% |

To conduct an internal performance analysis on our trained model, it is essential to first define our accuracy metric. As mentioned in this paper, we are using a semi-supervised learning method (i.e. clustering). Our defined accuracy metric is how accurately our clusters represent the `user-agent` instances. For this evaluation, we consider a cluster assignment to be accurate if all instances of the same `user-agent` are mapped to the same cluster number (i.e., the one with the majority of data points for that `user-agent` string).

$$\text{Model accuracy} = \frac{\text{Number of user-agents mapped to their corresponding clusters}}{\text{Total number of user-agents}} \tag{F.1}$$

For example, if the majority of the data points with the `user-agent` of Chrome110 are assigned to cluster 0, we consider cluster 0 as the corresponding cluster for Chrome110. Any data point with the `user-agent` of Chrome110 that is assigned to a different cluster number is considered misclustered. Thus, when discussing model accuracy in the context of the following sensitivity analysis, we refer to the percentage of data in the dataset that is correctly assigned to the appropriate cluster numbers (Formula F.1).

For the Browser Polygraph model trained in this paper, as discussed in Section 5.6.4, we have determined that *28 features, 7 PCA components, and 11 clusters (k=11)* are optimal for our fraud detection objectives. With these parameters, our model achieves a 99.6% accuracy in correctly assigning `user-agent` instances to clusters. We will demonstrate how changes to the number of clusters, PCA components, and features impact the accuracy of the trained model in Browser Polygraph.

• **Number of clusters:** to evaluate the impact of varying the number of clusters on model accuracy, we trained our model using different numbers of clusters (5, 7, 9, 11, 13, 15, 17, and 19) while maintaining a consistent feature set of 28 attributes and 7 principal components. As shown in Table F.1, model accuracy generally decreases as the number of clusters increases. With a higher number of clusters, browser instances with close versions are more likely to be misclustered. This occurs because increasing the number of clusters separates instances with closely aligned feature values, whereby assigning them to distinct clusters.

An important consideration arises: why not use fewer clusters (i.e., fewer than k=11)? In practice, selecting too few clusters provides attackers with greater flexibility to manipulate browser attributes, enabling them to evade detection by Browser Polygraph. From a fraud detection standpoint, while using a larger number of clusters might detect more potential fraud cases, it is essential not to compromise accuracy, as this can lead to an increase in false positives, i.e. incorrectly identifying a non-fraudulent browser instance as a fraud case. As illustrated in Table F.1, there is a noticeable decrease in accuracy (0.2%) from k=11 to k=13, indicating that k=11 is an optimal choice for balancing cluster count and model accuracy.

• **Number of PCA components:** here, we evaluate the influence of the number of PCA components on our model's accuracy, while keeping the number of features at 28. Table F.2 presents our analysis for PCA components equal to 6 and greater

**Table F.2:** Sensitivity analysis for different number of PCA components with 28 features.

| **Number of PCA components** | **6** | **7** | **8** | **9** | **10** |
|---|---|---|---|---|---|
| **Optimal number of clusters** | 11 | 11 | 11 | 11 | 11 |
| **Model accuracy** | 99.54% | 99.60% | 99.46% | 99.46% | 99.46% |

**Table F.3:** Sensitivity analysis for different number of features.

| Features | Names of added features | PCA | k | Model accuracy |
|---|---|---|---|---|
| 28 | Appendix-2 | 7 | 11 | 99.60% |
| 32 | `Object.getOwnPropertyNames(HTMLIFrameElement.prototype).length`<br>`Object.getOwnPropertyNames(SVGAElement.prototype).length`<br>`Object.getOwnPropertyNames(RemotePlayback.prototype).length`<br>`Object.getOwnPropertyNames(StylePropertyMapReadOnly.prototype).length` | 7 | 11 | 99.52% |
| 36 | `Object.getOwnPropertyNames(Screen.prototype).length`<br>`Object.getOwnPropertyNames(Request.prototype).length`<br>`Object.getOwnPropertyNames(TouchEvent.prototype).length`<br>`Object.getOwnPropertyNames(TaskAttributionTiming.prototype).length` | 7 | 12 | 99.41% |
| 42 | `Object.getOwnPropertyNames(PictureInPictureWindow.prototype).length`<br>`Object.getOwnPropertyNames(ReportingObserver.prototype).length`<br>`Object.getOwnPropertyNames(HTMLTemplateElement.prototype).length`<br>`Object.getOwnPropertyNames(MediaSession.prototype).length` | 7 | 14 | 99.41% |

than 6. Each experiment is followed by a similar analysis as detailed in Section 5.6.4 to determine the optimal number of clusters. We observe an increase in accuracy from PCA=6 to PCA=7, indicating that the additional component provides valuable information for accurately clustering browser instances. However, adding more components beyond 7 leads to a decrease in accuracy due to the introduction of noise. This observation can be attributed to the "curse of dimensionality," a phenomenon that occurs when too many features—particularly irrelevant or noisy ones—are added to a model. In high-dimensional spaces, data points become more sparse, leading to reduced clustering accuracy. Therefore, 7 number of PCA components lead to the most accurate clustering.

• **Number of features:** here, we explore the influence of selecting more than 28 features on the training process. We analyzed all 513 features from our dataset of 205k rows. The values were sorted based on standard deviation (excluding those deemed improper in Section 5.6.3), and at each step, we selected 4 additional features from those with higher standard deviations. Out of the 4 features added in each step, two were present across the releases of Chrome, Edge, and Firefox browsers, highlighting differences between browser versions. Meanwhile, the other two features, which are absent in Firefox, were selected to emphasize the differences between Firefox and Chrome/Edge browsers. For each step, we extracted the optimal number of PCA components (similar to Figure 5.2) based on the analysis in Section 5.6.4 and also determined the optimal number of clusters according to the logic in Section 5.6.4, by observing the highest relative WSS. The list of added features and the results of our analysis for each step are shown in Table F.3. Moving from 28 to 32 features does not affect the number of PCA components or clusters; however, it decreases model accuracy. We believe this accuracy reduction is due to the introduction of noise and the curse of dimensionality from the added features. However, increasing the number of features in the other steps (i.e., steps with 36 and 42 features) does not impact the number of PCA components, but it increases the number of optimal clusters. This is because the added features introduce additional dimensions to our data points,

potentially leading to the formation of more clusters, which, in turn, reduces model accuracy. To summarize, the analysis in Table F.3 shows a decrease in accuracy as the number of features and clusters increases. For instance, increasing the number of features from 28 to 32 causes the accuracy to drop by 0.08%, which, for a dataset of 200k rows, could result in up to 160 additional false positives in the fraud detection system (a 13% increase compared to a model with 28 features).

The three evaluations above clearly demonstrate that our chosen parameters in Browser Polygraph (28 features, 7 PCA components, and k=11 clusters) lead to optimal accuracy in clustering browser instances.

# APPENDIX G

# COMPARISON BETWEEN BROWSER POLYGRAPH AND FINE-GRAINED FINGERPRINTING TECHNIQUES

In this section, a performance comparison between Browser Polygraph and fine-grained fingerprinting techniques is provided. Synthetic data is being used to compare Browser Polygraph coarse-grained fingerprints and two of the fine-grained fingerprinting techniques (FingerprintJS and ClientJS).

Before diving into the experiments conducted, we would like to reiterate that fine-grained fingerprinting techniques were originally designed for user tracking by collecting detailed information about a user's OS, browser, configuration, and underlying hardware. These techniques typically generate a JSON object containing these details, which is hashed to create a user identifier for tracking purposes. However, to provide a testbed for comparison between fine-grained techniques and our proposed method, we have to further process the JSON object generated by the fine-grained fingerprinting data to extract relevant information that can be used as the inputs of a clustering method.

In this synthetic setup, we used BrowserStack to launch Chrome, Edge, and Firefox browser instances on Windows 10 and 11. The synthetic data are collected from (1) our developed website for Browser Polygraph, (2) FingerprintJS website, and (3) a custom website for ClientJS. In this experiment, we extracted 430 fingerprints from Browser Polygraph, 382 fingerprints from FingerprintJS, and 391 fingerprints from ClientJS along with their corresponding `user-agent` strings. To prepare the FingerprintJS and ClientJS data for clustering, we had to interpret the fingerprints provided in the JSON format. To do so, for nested objects within the JSON, we flattened the data by creating separate columns for each key. Then, we converted all values into numerical formats: numeric values were left unchanged, boolean values were mapped to 0 and 1, and strings were encoded as numerical categories. Any missing values were assigned a default value of -1. Subsequently, columns with unique values across all data points were excluded. Additionally, for ClientJS, since some features were directly extracted from the `user-agent` string, we excluded those features as well. All other features were retained for clustering. Ultimately, 268 features were extracted for clustering FingerprintJS data while only 7 useful features from ClientJS were obtained, as ClientJS does not provide many descriptive browser features. We also used 28 features from Browser Polygraph as discussed in Section 5.6.4.

To perform clustering, we followed features scaling, PCA components selection, and cluster numbers selection as described in Section 5.6.4. Details and results of clustering for these three datasets are shown in Table G.1. As can be seen, clustering model in Browser Polygraph achieved 100% accuracy as it did not miscluster any browser instance. However, FingerprintJS had 99.21% model accuracy and ClientJS had model accuracy of 93.63% which falls behind the other two techniques. The results show that the coarse-grained features of Browser Polygraph outperform fine-grained techniques in clustering. That is because the coarse-grained features of Browser Polygraph provide broad insights into the behavior and attributes of browsers. In fact, broad features offer a more comprehensive view of the data, aiding in effectively grouping similar browsers. This causes the clustering based on coarse-grained features to outperform clustering that relies on fine-grained fingerprinting features, which collect detailed and individualistic user tracking information.

Similar to Windows 10 and 11 as mentioned above, we repeated the synthetic experiment to collect the data for Browser Polygraph, FingerprintJS, and ClientJS on macOS Sequoia and macOS Sonoma. The results are shown in Table G.2. As can

**Table G.1:** Comparison of clustering performance between Browser Polygraph and fine-grained fingerprinting techniques using synthetic data generated by BrowserStack across Windows 10 and Windows 11 (for Chrome, Edge, and Firefox).

| Technique | Size of dataset | Features | PCA | k | Model accuracy |
|---|---|---|---|---|---|
| **Browser Polygraph** | 430 | 28 | 13 | 14 | 100% |
| **FingerprintJS** | 382 | 268 | 55 | 16 | 99.21% |
| **ClientJS** | 391 | 7 | 2 | 5 | 93.60% |

be seen, the performance of clustering for the three methods on macOS is similar to the performances observed for Windows.

**Table G.2:** Comparison of clustering performance between Browser Polygraph and fine-grained fingerprinting techniques using synthetic data generated by BrowserStack across macOS Sequoia and macOS Sonoma (for Chrome, Edge, and Firefox).

| Technique | Size of dataset | Features | PCA | k | Model accuracy |
|---|---|---|---|---|---|
| **Browser Polygraph** | 320 | 28 | 11 | 14 | 100% |
| **FingerprintJS** | 325 | 589 | 36 | 9 | 99.38% |
| **ClientJS** | 327 | 4 | 2 | 15 | 85.93% |

The analysis in this section highlights that the explored fine-grained fingerprinting techniques, in their current form and without additional preprocessing, may not be well-suited for clustering purposes.